

Indice

1	Introducción.....	3
1.1	El dominio del problema.....	3
1.1.1	Software de nivel industrial.....	3
1.1.2	El software es costoso.....	4
1.1.3	Tardío e In-fiabile.....	5
1.1.4	Mantenimiento y Re-trabajo.....	6
1.2	Los Desafíos de la Ingeniería de Software.....	8
1.2.1	Escala.....	8
1.2.2	Calidad y Productividad.....	10
1.2.3	Consistencia y Repetibilidad.....	12
1.2.4	Cambio.....	13
1.3	El Enfoque de la Ingeniería de Software.....	13
1.3.1	Proceso de Desarrollo por Fases.....	14
	Análisis de Requisitos.....	15
	Diseño de Software.....	15
	Codificación.....	16
	Pruebas.....	16
1.3.2	Gestión del Proceso.....	17
1.4	Resumen.....	18
3	Análisis y Especificación de Requerimientos de Software.....	20
3.1	Requerimientos de Software.....	21
3.1.1	Necesidad del SRS.....	21
	Errores en los requerimientos y su impacto.....	22
	Impacto en costos.....	23
	Requerimientos cambiantes y re-trabajo.....	24
	Conclusión.....	25
3.1.2	Proceso de Requerimientos.....	25
3.2	Análisis del Problema.....	28
3.2.1	Enfoque Informal.....	29
3.2.2	Modelado de Flujo de Datos.....	29
	Diagramas de Flujo de Datos y Diccionario de Datos.....	30
	Verificación de un DFD y su diccionario de datos.....	33
	El Método de Análisis Estructurado.....	34
	Etapas del método.....	34
	Recomendaciones de construcción.....	36
	Fortalezas y limitaciones.....	36
3.2.3	Modelado Orientado a Objetos.....	36
	Conceptos Básicos y Notación.....	37
3.2.4	Prototipado.....	38
3.3	Especificación de Requisitos.....	41
3.3.1	Características de un SRS.....	42
3.3.2	Componentes de un SRS.....	44
	Requisitos Funcionales.....	44
	Requisitos de Rendimiento.....	45
3.3.3	Lenguaje de Especificación.....	46
3.4	Especificación Funcional con Casos de Uso.....	47
3.5	Validación.....	47
3.6	Métricas.....	49
4	Arquitectura de Software.....	50
4.1	El Rol de la Arquitectura de Software.....	50

Principales usos de la descripción arquitectónica.....	51
5 Planificación de un Proyecto de Software.....	52
6 Diseño Orientado a Funciones.....	52
7 Diseño Orientado a Objetos.....	52
8 Diseño Detallado.....	53
8.1 Diseño Detallado y PDL.....	53
8.1.1 PDL.....	53
8.1.2 Diseño de lógica o de algoritmos.....	54
8.1.3 Modelado de estado de las clases.....	57
8.2 Verificación.....	59
8.2.1 Revisiones de diseño (Design Walkthroughs).....	59
8.2.2 Revisión Crítica del Diseño.....	60
8.2.3 Verificadores de Consistencia.....	61
8.3 Métricas.....	61
8.3.1 Complejidad Ciclomática.....	62
8.3.2 Vínculos de datos.....	63
8.3.3 Métrica de Cohesión.....	64
9 Codificación.....	65
9.1 Principios y guías de programación.....	66
9.1.1 Errores comunes de codificación.....	67
Fugas de memoria (Memory Leaks).....	67
Liberar un recurso ya liberado.....	67
Desreferenciación de NULL.....	68
Falta de direcciones únicas.....	69
Errores de sincronización.....	69
9.1.2 Programación Estructurada.....	69
9.1.3 Ocultamiento de Información.....	72
9.2 Proceso de Codificación.....	73
9.2.1 Un Proceso de Codificación Incremental.....	73
9.2.2 Desarrollo Guiado por Pruebas (Test Driven Development).....	74
9.2.3 Programación en Pareja (Pair Programming).....	76
9.2.4 Control del Código Fuente y Construcción.....	77
9.3 Refactorización.....	79
9.3.1 Conceptos Básicos.....	79
9.3.2 Un ejemplo.....	81
9.3.3 Malos olores.....	81
9.3.4 Refactorizaciones comunes.....	82
Mejorando los métodos.....	82
Mejorando las clases.....	83
Mejorando las jerarquías.....	84
9.4 Verification.....	85
9.4.1 Inspección de código (Code Inspections).....	86

Ingeniería del Software

La Ingeniería de Software es la aplicación de un enfoque sistemático, disciplinado, y cuantificable al desarrollo, operación, y mantenimiento del software.

Software

Colección de programas, procedimientos, y la documentación y datos asociados que determinan la operación de un sistema de computación.

1 Introducción

1.1 El dominio del problema

En ingeniería de software no tratamos con programas que la gente desarrolla para ilustrar un concepto o como pasatiempo (a lo que aquí llamamos sistemas de estudiante). En cambio, el dominio de problemas se centra en software que resuelve necesidades reales de ciertos usuarios, en contextos donde sistemas más grandes o incluso negocios completos pueden depender de ese software, y donde los fallos pueden provocar pérdidas significativas, ya sea de forma directa o indirecta.

A este tipo de software lo llamamos software de nivel industrial.

Comencemos primero hablando de la diferencia clave entre el software de estudiante y el software de nivel industrial.

1.1.1 Software de nivel industrial

Un sistema desarrollado por un estudiante está pensado principalmente con fines de demostración; por lo general, no se utiliza para resolver un problema real de alguna organización. En consecuencia, nada verdaderamente importante depende de que funcione correctamente. Y como nada de peso depende de ese software, la existencia de “bugs” (defectos o fallos) no es un gran problema. Por eso, normalmente no se diseña pensando en aspectos de calidad como portabilidad, robustez, fiabilidad o usabilidad.

Además, el software de estudiante suele ser utilizado por el mismo desarrollador que lo creó, así que no hay necesidad de documentación, y nuevamente los errores no son críticos, ya que el propio usuario–desarrollador puede corregirlos en cuanto los detecta.

En cambio, un sistema de software de nivel industrial se construye para resolver un problema real de un cliente, y es utilizado por la organización del cliente para operar alguna parte de su negocio (entendiendo “negocio” en un sentido muy amplio: puede ser administrar inventarios, finanzas, monitorear pacientes, controlar el tráfico aéreo, etc.). Dicho de otro modo, actividades importantes dependen de que el sistema funcione correctamente. Y una falla en este tipo de sistemas puede tener un impacto enorme: pérdidas económicas o comerciales, molestias para los usuarios, e incluso pérdida de bienes o vidas humanas.

Por lo tanto, este software necesita ser de alta calidad en propiedades como confiabilidad, robustez y facilidad de uso.

Este requisito de alta calidad tiene muchas implicaciones:

- **Primero**, el software debe probarse exhaustivamente antes de ponerse en uso. Esta necesidad de pruebas rigurosas aumenta considerablemente el costo. En un proyecto de software industrial, entre un 30% y un 50% del esfuerzo total puede dedicarse a las pruebas (mientras que en un proyecto de estudiante, ¡incluso un 5% podría considerarse mucho!).
- **Segundo**, desarrollar software de alta calidad requiere dividir el proceso en fases, de manera que la salida de cada una sea revisada y evaluada para eliminar defectos lo antes posible. Esta necesidad de dividir el problema y detectar errores temprano implica más documentación, más estándares y más procesos. Todo esto aumenta el esfuerzo necesario para construir el software, por lo que la productividad al desarrollar software industrial suele ser mucho menor que la de un software estudiantil.

El software de nivel industrial también presenta otras características que no existen en los sistemas de estudiante. Generalmente, para un mismo problema, los requisitos detallados de lo que debe hacer el software aumentan considerablemente. Además de los requisitos de calidad, se suman necesidades de respaldo y recuperación, tolerancia a fallos, cumplimiento de estándares, portabilidad, etc. Todo esto, por lo general, hace que el sistema sea más complejo y más grande.

De hecho, para un mismo problema, el software industrial puede tener un tamaño al menos dos veces mayor que el sistema de estudiante. Si asumimos una productividad cinco veces menor y un tamaño dos veces mayor, construir un software de nivel industrial puede requerir unas **10 veces más esfuerzo** que un software de estudiante para el mismo problema. La “regla práctica” de Brooks también dice que el software industrial puede costar unas 10 veces más que el software estudiantil [25].

La industria del software está interesada principalmente en desarrollar software de nivel industrial, y el área de ingeniería de software se enfoca en cómo construir este tipo de sistemas.

En el resto del libro, cuando usemos la palabra *software*, nos estaremos refiriendo a *software de nivel industrial*.

El IEEE define software como: “*el conjunto de programas informáticos, procedimientos, reglas, y la documentación y datos asociados*” [91]. Esta definición deja claro que el software no son solo programas, sino que también incluye toda la documentación y datos relacionados. Esto implica que la disciplina que se ocupa de desarrollar software no debe centrarse únicamente en crear programas, sino en producir todos los elementos que lo componen.

1.1.2 El software es costoso

El software de nivel industrial es muy caro, principalmente porque su desarrollo es extremadamente intensivo en mano de obra. Para darnos una idea de los costos implicados, consideremos el estado actual de la práctica en la industria.

La métrica más utilizada para medir el tamaño del software en la industria es el número de líneas de código (LOC) o miles de líneas de código (KLOC) entregadas.

Dado que el principal costo de producir software es el trabajo humano, el **costo de desarrollo** se mide generalmente en **persona-meses** de esfuerzo invertido. Asimismo, la **productividad en la industria** suele expresarse en **LOC (o KLOC) por persona-mes**.

En el desarrollo de nuevo código, la productividad típica en la industria oscila entre **300 y 1.000 LOC por persona-mes**. Es decir, a lo largo de todo el ciclo de desarrollo, un programador promedio produce entre 300 y 1.000 LOC por mes.

Las empresas de software suelen cobrar al cliente para el que desarrollan a partir de **100.000 USD por persona-año**, o más de **8.000 USD por persona-mes** (aproximadamente 50 USD por hora). Con las cifras actuales de productividad, esto significa que **cada línea de código entregada cuesta entre 8 y 25 USD**. Dicho de otro modo, una sola línea de código puede costar entre 8 y 25 dólares con los niveles actuales de costos y productividad.

Incluso proyectos pequeños pueden fácilmente alcanzar **50.000 LOC**. Con esta productividad, un proyecto así costaría entre **0,5 y 1,25 millones de USD**.

Con la potencia actual de las computadoras, un software de este tamaño puede ejecutarse perfectamente en una estación de trabajo o un pequeño servidor. Esto implica que un software que puede costar más de un millón de dólares puede correr sobre un hardware cuyo precio no pasa de unas decenas de miles de dólares. Esto demuestra claramente que **el costo del hardware es solo una fracción del costo del software** que ejecuta.

Este ejemplo muestra que el software no solo es muy caro, sino que suele ser el **componente principal** del costo total de un sistema automatizado, siendo el hardware un elemento mucho menor. Esto se refleja en el gráfico clásico de inversión hardware–software mostrado en la Figura 1.1 [17].

Como se observa en la Figura 1.1, en los primeros tiempos el costo del hardware solía dominar el costo total del sistema. Pero, a medida que el precio del hardware ha ido disminuyendo y continúa bajando, y que su potencia se duplica aproximadamente cada 2 años (la Ley de Moore), permitiendo ejecutar sistemas de software cada vez más grandes, **el costo del software ha pasado a ser el factor dominante en los sistemas**.

1.1.3 Tardío e In-fiable

A pesar de los importantes avances en las técnicas de desarrollo de software, esta sigue siendo un área débil. En una encuesta realizada a más de 600 empresas, más del **35%** informó tener algún proyecto de desarrollo relacionado con computadoras que clasificaron como “*runaway*” [131].

Un *runaway* no es simplemente un proyecto que se retrasa un poco o que supera ligeramente el presupuesto, sino uno en el que **el presupuesto y el cronograma están completamente fuera de control**. El problema se ha vuelto tan grave que incluso ha generado una industria dedicada a resolverlo: existen empresas consultoras que asesoran sobre cómo recuperar estos proyectos, y una de ellas obtuvo más de **30 millones de dólares** en ingresos provenientes de más de 20 clientes [131].

De forma similar, se han documentado numerosos casos de **falta de fiabilidad** en el software: programas que no hacen lo que deberían hacer o que hacen cosas que no deberían. En una encuesta

de defensa, se informó que **más del 70% de todas las fallas de equipos** se debieron al software. Y esto en sistemas que también cuentan con componentes eléctricos, hidráulicos y mecánicos.

Este dato deja claro que **otras disciplinas de la ingeniería han avanzado mucho más que la ingeniería de software**, y que, en un sistema que combina productos de varias ramas de la ingeniería, el software suele ser el componente más débil. El fallo de un vuelo temprano del programa Apolo se atribuyó al software. De forma parecida, la falla en una prueba de lanzamiento de un misil en India se debió a problemas de software. Incluso muchos bancos han perdido millones de dólares debido a errores e imprecisiones en su software [122].

Un apunte sobre la **causa de la falta de fiabilidad en el software**: las fallas de software son distintas de las que ocurren, por ejemplo, en sistemas mecánicos o eléctricos. En estos últimos, los productos fallan por cambios en sus propiedades físicas o eléctricas debido al envejecimiento.

En cambio, un producto de software **nunca se desgasta con el tiempo**. Sus fallos se producen por *bugs* o errores introducidos durante el diseño o el desarrollo. Así, aunque un sistema de software pueda fallar después de funcionar correctamente durante un tiempo, **el error que provoca esa falla ya estaba presente desde el principio**; simplemente, se ejecutó en el momento del fallo. Esto es muy diferente a otros sistemas, en los que un fallo suele significar que, en algún momento antes de producirse, surgió un problema (por desgaste) que antes no existía.

1.1.4 Mantenimiento y Re-trabajo

Una vez que el software se entrega y se pone en funcionamiento, entra en la fase de mantenimiento. ¿Por qué es necesario el mantenimiento si el software no envejece?

El mantenimiento del software no se debe a que sus componentes se desgasten y necesiten reemplazo, sino a que normalmente permanecen **errores residuales** en el sistema que deben corregirse a medida que se descubren. Se cree comúnmente que, con el estado actual de la tecnología, prácticamente **todo el software desarrollado contiene errores o bugs**. Muchos de ellos solo aparecen después de que el sistema ha estado en operación, incluso durante mucho tiempo.

Estos errores, una vez detectados, deben eliminarse, lo que implica cambiar el software. A esto se le llama **mantenimiento correctivo**.

Incluso sin *bugs*, el software suele cambiar con frecuencia. La principal razón es que el software a menudo debe **actualizarse y mejorarse** para incluir más funcionalidades y ofrecer más servicios. Esto también requiere modificaciones.

Se ha argumentado que, una vez desplegado, el software opera en un **entorno cambiante**. Por lo tanto, las necesidades que motivaron su desarrollo también cambian para adaptarse a ese nuevo entorno. Esto obliga a que el software se adapte a las nuevas necesidades. A su vez, el software modificado cambia el entorno, lo que genera nuevos cambios. Este fenómeno se conoce como **ley de la evolución del software** y el mantenimiento que ocasiona se llama **mantenimiento adaptativo**.

Aunque el mantenimiento no se considera parte directa del desarrollo, es una actividad crucial en la vida de un producto de software. Si consideramos todo el ciclo de vida, **el costo del mantenimiento**

suele superar el costo del desarrollo. La proporción mantenimiento/desarrollo se ha estimado en 80:20, 70:30 o 60:40.

El trabajo de mantenimiento se basa en software existente, mientras que el desarrollo crea software nuevo. Por ello, el mantenimiento gira en torno a **entender el software existente**, lo que incluye no solo el código, sino también la documentación relacionada.

Al modificar el software, el mantenedor debe comprender bien **los efectos del cambio**, ya que es fácil introducir efectos secundarios no deseados. Para comprobar que las partes no modificadas siguen funcionando igual, se realiza **pruebas de regresión** (*regression testing*), que consisten en ejecutar casos de prueba antiguos para asegurarse de que no se han introducido nuevos errores.

En resumen, el mantenimiento implica:

- Comprender el código y la documentación existente
- Analizar el impacto del cambio
- Modificar código y documentación
- Probar las nuevas partes
- Reprobar las partes antiguas

Como muchas veces durante el desarrollo no se piensa en las necesidades de los futuros mantenedores, suele haber poca documentación de apoyo, lo que, junto con la complejidad de la tarea, convierte al mantenimiento en la **actividad más costosa** de la vida del software.

El mantenimiento es un tipo de cambio que se realiza **después** de completar el desarrollo y desplegar el software. Sin embargo, existen otros cambios que generan **re-trabajo** (*rework*) durante el propio desarrollo.

Uno de los mayores problemas, especialmente en sistemas grandes y complejos, es que los **requisitos** del software no se entienden del todo. Para especificarlos completamente, todas las funcionalidades, interfaces y restricciones deben definirse antes de empezar el desarrollo. Esto obliga tanto a clientes como desarrolladores a **imaginar cómo debería comportarse el software terminado**, algo difícil de hacer, sobre todo en proyectos grandes.

En la práctica, el desarrollo comienza cuando se cree que los requisitos están bien definidos. Pero con el tiempo y una mejor comprensión del sistema, los clientes suelen descubrir requisitos adicionales no especificados al inicio. Esto lleva a cambios de requisitos, que obligan a modificar también el diseño y el código, generando re-trabajo.

Además, no es solo por requisitos mal entendidos: en proyectos largos, las **necesidades del cliente cambian con el tiempo**. Los requerimientos originales reflejan el contexto actual en el que se inició el proyecto, pero, al cambiar el contexto, cambian también las necesidades. Obviamente, el cliente quiere que el sistema final satisfaga sus necesidades más recientes, lo que provoca más cambios y re-trabajo.

Se estima que el re-trabajo representa **entre el 30% y el 40% del costo de desarrollo** [22]. Es decir, de todo el esfuerzo invertido en desarrollo, entre un tercio y casi la mitad se dedica a rehacer trabajo debido a cambios. Por eso, los cambios y el re-trabajo son uno de los principales contribuyentes a la **crisis del software**.

A diferencia de otros problemas ya mencionados, el re-trabajo no siempre refleja deficiencias técnicas del desarrollo: muchas veces es el propio cliente quien impulsa los cambios debido a la evolución de sus necesidades.

1.2 Los Desafíos de la Ingeniería de Software

Ahora que tenemos una mejor comprensión del dominio del problema con el que trata la ingeniería de software, orientemos nuestra discusión hacia la propia Ingeniería de Software.

La ingeniería de software se define como el enfoque sistemático para el desarrollo, operación, mantenimiento y retiro del software [91]. En este libro nos enfocaremos principalmente en el desarrollo.

El uso del término **enfoque sistemático** para el desarrollo de software implica que se utilizan metodologías para desarrollarlo que son **repetibles**. Es decir, si las metodologías son aplicadas por diferentes grupos de personas, se producirá un software similar. En esencia, el objetivo de la ingeniería de software es acercar el desarrollo de software a la ciencia y la ingeniería y alejarlo de los enfoques ad-hoc para el desarrollo, cuyos resultados no son predecibles, pero que han sido usados ampliamente en el pasado y que aún se siguen utilizando para desarrollar software.

Como se mencionó, el software de tipo industrial está destinado a resolver algún problema del cliente. (Usamos el término **cliente** en un sentido muy general, para referirnos a las personas cuyas necesidades deben ser satisfechas por el software). Por lo tanto, el problema es desarrollar (de manera sistemática) un software que satisfaga las necesidades de algunos usuarios o clientes. Este problema fundamental con el que trata la ingeniería de software se muestra en la Figura 1.2.

Aunque el problema básico es desarrollar de manera sistemática un software que satisfaga al cliente, existen algunos factores que afectan los enfoques seleccionados para resolver el problema. Estos factores son las fuerzas primarias que impulsan el progreso y el desarrollo en el campo de la ingeniería de software. Nosotros consideramos estos como los **principales desafíos de la ingeniería de software** y aquí discutiremos algunos de los más importantes.

1.2.1 Escala

Un factor fundamental con el que la ingeniería de software debe lidiar es el tema de la **escala**; el desarrollo de un sistema muy grande requiere un conjunto de métodos muy diferente al que se usa para desarrollar un sistema pequeño. En otras palabras, los métodos que se utilizan para desarrollar sistemas pequeños, en general, no escalan a sistemas grandes.

Un ejemplo ilustra este punto. Consideremos el problema de contar personas en una sala frente a realizar un censo de un país. Ambos son esencialmente problemas de conteo. Pero los métodos utilizados para contar personas en una sala (probablemente ir fila por fila o columna por columna) simplemente no funcionarían para realizar un censo. Se deberá usar un conjunto de métodos distinto para llevar adelante un censo, y este problema requerirá considerablemente más gestión, organización y validación, además del simple conteo.

De manera similar, los métodos que se pueden usar para desarrollar programas de unos pocos cientos de líneas no se pueden esperar que funcionen cuando se necesita desarrollar software de unos pocos cientos de miles de líneas. Se debe usar un conjunto diferente de métodos para desarrollar software de gran tamaño. Cualquier proyecto grande implica el uso de **ingeniería** y de **gestión de proyectos**. En proyectos de software, por ingeniería entendemos los métodos, procedimientos y herramientas que se utilizan. En proyectos pequeños, pueden usarse métodos informales tanto para el desarrollo como para la gestión. Sin embargo, para proyectos grandes, ambos tienen que ser mucho más formales, como se muestra en la Figura 1.3.

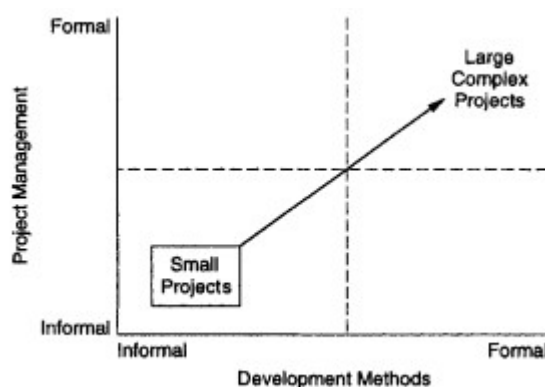


Figura 1.3 : El problema de la escala.

Tal como muestra la figura, al tratar con un proyecto pequeño de software, la capacidad de ingeniería requerida es baja (todo lo que necesitas saber es programar y hacer un poco de pruebas) y el requerimiento de gestión de proyectos también es bajo. Sin embargo, cuando la escala cambia a grande, para resolver esos problemas adecuadamente es esencial avanzar en ambas direcciones: los métodos de ingeniería utilizados para el desarrollo deben ser más formales, y la gestión del proyecto de desarrollo también debe ser más formal.

Por ejemplo, si juntamos a 50 programadores brillantes (que saben cómo desarrollar programas pequeños correctamente) sin una gestión y procedimientos de desarrollo formales, y les pedimos que desarrollen un sistema de control de inventario en línea para un fabricante de automóviles, es muy poco probable que produzcan algo útil. Para ejecutar el proyecto con éxito, se debe utilizar un método adecuado de ingeniería del sistema y el proyecto debe estar fuertemente gestionado para asegurarse de que los métodos efectivamente se están siguiendo y de que el costo, el cronograma y la calidad están bajo control.

No existe una definición universalmente aceptada de qué es un proyecto "pequeño" y qué es un proyecto "grande", y las escalas cambian claramente con el tiempo. Sin embargo, de manera informal, podemos usar órdenes de magnitud y decir que un proyecto es **pequeño** si su tamaño es menor a 10 KLOC, **mediano** si el tamaño es menor a 100 KLOC (y mayor a 10), **grande** si el tamaño es menor a un millón de LOC, y **muy grande** si el tamaño es de muchos millones de LOC.

Para darnos una idea del tamaño de algunos productos de software reales, en la Tabla 1.1 se muestran los tamaños aproximados de algunos productos conocidos.

Size (KLOC)	Software	Languages
980	gcc	ansic, cpp, yacc
320	perl	perl, ansic, sh
305	teTeX	ansic, perl
200	openssl	ansic, cpp, perl
200	Python	python, ansic
100	apache	ansic, sh
90	cvs	ansic, sh
65	sendmail	ansic
60	xfig	ansic
45	gnuplot	ansic, lisp
38	openssh	ansic
30,000	Red Hat Linux	ansic, cpp
40,000	Windows XP	ansic, cpp

Tabla 1.1 : Tamaño en KLOC de algunos productos conocidos.

1.2.2 Calidad y Productividad

Una disciplina de ingeniería, casi por definición, está guiada por parámetros prácticos de **costo**, **cronograma** y **calidad**. Una solución que demande enormes recursos y muchos años puede no ser aceptable. De la misma manera, una solución de baja calidad, incluso si tiene un costo bajo, puede no ser de mucha utilidad. Como todas las disciplinas de ingeniería, la ingeniería de software está impulsada por estos tres factores principales: costo, cronograma y calidad.

El costo de desarrollar un sistema corresponde al costo de los recursos utilizados para el sistema, lo cual, en el caso del software, está dominado por el costo de la mano de obra, ya que el desarrollo es en gran medida una actividad intensiva en trabajo humano. Por lo tanto, el costo de un proyecto de software suele medirse en **persona-meses**; es decir, el costo se considera el número total de persona-meses invertidos en el proyecto. (Los persona-meses pueden convertirse a un monto en dólares multiplicándolos por el costo promedio en dólares —incluyendo costos indirectos como hardware y herramientas— de un persona-mes).

El **cronograma** es un factor importante en muchos proyectos. Las tendencias del mercado exigen que el *time-to-market* de un producto se reduzca; es decir, que el ciclo desde el concepto hasta la entrega sea más corto. Para el software, esto significa que debe desarrollarse más rápido.

La **productividad**, medida en términos de salida (*KLOC*) por persona-mes, puede capturar adecuadamente tanto las preocupaciones de costo como de cronograma. Si la productividad es mayor, está claro que el costo en términos de persona-meses será menor (el mismo trabajo se puede realizar con menos persona-meses). De manera similar, si la productividad es mayor, la posibilidad de desarrollar el software en menos tiempo mejora: un equipo con alta productividad terminará un trabajo en menos tiempo que un equipo del mismo tamaño con baja productividad. (El tiempo real que tomará el proyecto, por supuesto, también depende del número de personas asignadas al mismo). En otras palabras, la productividad es un factor clave en todos los negocios, y el deseo de alta productividad dicta, en gran medida, cómo se hacen las cosas.

El otro gran factor que guía cualquier disciplina de producción es la **calidad**. Hoy en día, la calidad es un mantra central, y las estrategias de negocio se diseñan en torno a ella. Claramente, desarrollar

software de alta calidad es otro objetivo fundamental de la ingeniería de software. Sin embargo, mientras que el costo es un concepto generalmente bien entendido, la calidad en el contexto del software necesita una discusión más profunda. Aquí utilizamos el estándar internacional de calidad de productos de software como base [94].



Figura 1.4: Atributos de calidad del software

Según el modelo de calidad adoptado por este estándar, la calidad del software se compone de **seis atributos principales** (llamados características), tal como se muestra en la Figura 1.4 [94]. Estos seis atributos tienen características detalladas que se consideran las básicas y que pueden (y deben) medirse usando métricas adecuadas. A nivel superior, para un producto de software, estos atributos se definen de la siguiente manera [94]:

- **Funcionalidad:** La capacidad de proporcionar funciones que satisfacen necesidades explícitas e implícitas cuando se utiliza el software.
- **Confiabilidad:** La capacidad de mantener un nivel de desempeño especificado.
- **Usabilidad:** La capacidad de ser entendido, aprendido y utilizado.
- **Eficiencia:** La capacidad de ofrecer un rendimiento adecuado en relación con la cantidad de recursos utilizados.
- **Mantenibilidad:** La capacidad de ser modificado con fines de corrección, mejora o adaptación.
- **Portabilidad:** La capacidad de adaptarse a diferentes entornos especificados sin necesidad de acciones o medios adicionales más allá de los previstos en el producto.

Las características de cada atributo brindan mayor detalle. Por ejemplo, la **usabilidad** incluye entendibilidad, capacidad de aprendizaje y operabilidad; la **mantenibilidad** incluye capacidad de cambio, capacidad de prueba y estabilidad; mientras que la **portabilidad** incluye adaptabilidad, instalabilidad, etc. La **funcionalidad** incluye adecuación (si se proporciona el conjunto apropiado de funciones), exactitud (los resultados son correctos) y seguridad. Es importante notar que, en esta clasificación, la **seguridad** se considera una característica de la funcionalidad y se define como: “la capacidad de proteger información y datos de manera que personas o sistemas no autorizados no puedan leerlos ni modificarlos, y que personas o sistemas autorizados no vean denegado su acceso”.

Existen dos consecuencias importantes de que la calidad tenga múltiples dimensiones. Primero, la calidad del software no puede reducirse a un solo número (o un solo parámetro). Y segundo, el concepto de calidad es específico de cada proyecto. En un proyecto ultra-sensible, la confiabilidad puede ser de máxima importancia y no así la usabilidad; mientras que en un paquete comercial de juegos para PC, la usabilidad puede ser la prioridad principal y no la confiabilidad. Por lo tanto, para cada proyecto de desarrollo de software debe especificarse un objetivo de calidad antes de

comenzar el desarrollo, y la meta del proceso de desarrollo debe ser satisfacer ese objetivo de calidad.

A pesar de que existen muchos factores de calidad, la **confiabilidad** suele aceptarse como el criterio de calidad más importante. Dado que la falta de confiabilidad del software se debe a la presencia de defectos en el mismo, una medida de la calidad es el número de defectos en el software entregado por unidad de tamaño (generalmente miles de líneas de código, o KLOC). Con este criterio principal de calidad, el objetivo es reducir lo más posible el número de defectos por KLOC. Las mejores prácticas actuales en ingeniería de software han logrado reducir la densidad de defectos a menos de 1 por KLOC.

Es importante señalar que, para usar esta definición de calidad, debe estar claramente definido qué se considera un **defecto**. Un defecto puede ser un problema en el software que lo hace colapsar, un problema que provoca que una salida no se alinee correctamente, o incluso un error de ortografía en una palabra. La definición exacta de lo que se considera un defecto dependerá claramente del proyecto o de los estándares de la organización que lo desarrolla (generalmente es esto último).

1.2.3 Consistencia y Repetibilidad

Ha habido muchos casos en los que se desarrolló software de alta calidad con una productividad muy elevada. Pero, han existido muchos más casos en los que se desarrolló software con baja calidad o baja productividad. Un desafío clave al que se enfrenta la ingeniería de software es cómo asegurar que los resultados exitosos puedan repetirse, y que pueda existir cierto grado de consistencia tanto en la calidad como en la productividad.

Podemos decir que una organización que desarrolla un sistema con alta calidad y una productividad razonable, pero que no es capaz de mantener esos niveles de calidad y productividad en otros proyectos, no conoce verdaderamente la buena ingeniería de software. Uno de los objetivos de los métodos de la ingeniería de software es que sistema tras sistema pueda ser producido con alta calidad y productividad. Es decir, que los métodos que se usan sean repetibles a lo largo de los proyectos, lo que conduce a una consistencia en la calidad del software producido.

Una organización dedicada al desarrollo de software no solo quiere alta calidad y productividad, sino que quiere obtenerlas de manera consistente. En otras palabras, una organización de desarrollo de software desea producir software de calidad consistente con productividad consistente. La consistencia en el desempeño es un factor importante para cualquier organización; le permite predecir el resultado de un proyecto con una precisión razonable, y mejorar sus procesos para producir productos de mayor calidad e incrementar su productividad. Sin consistencia, incluso estimar el costo de un proyecto se vuelve difícil.

Lograr la consistencia es un problema importante que la ingeniería de software debe abordar. Como puede imaginarse, este requisito de consistencia obliga a seguir ciertos procedimientos estandarizados para desarrollar software. No existen metodologías aceptadas de manera global y diferentes organizaciones utilizan distintas. Sin embargo, dentro de una organización, la consistencia se logra utilizando sus metodologías elegidas de manera uniforme. Marcos como **ISO9001** y el **Modelo de Madurez de Capacidades (CMM)** alientan a las organizaciones a estandarizar metodologías, usarlas de forma consistente y mejorarlas con base en la experiencia. Discutiremos este tema un poco más en el próximo capítulo.

1.2.4 Cambio

Hemos mencionado antes cómo el mantenimiento y el re-trabajo son muy costosos y cómo forman parte integral del dominio de problemas con los que trata la ingeniería de software. En el mundo actual, el cambio en los negocios es muy rápido. A medida que los negocios cambian, requieren que el software que los respalda también cambie. En general, a medida que el mundo cambia más rápido, el software también debe cambiar más rápido.

El cambio rápido tiene un impacto especial en el software. Como el software es fácil de modificar debido a la ausencia de propiedades físicas que dificulten el cambio, la expectativa hacia el software para adaptarse es mucho mayor.

Por lo tanto, uno de los desafíos de la ingeniería de software es **acomodar y aceptar el cambio**. Como veremos, se utilizan distintos enfoques para manejar el cambio. Pero el cambio es hoy uno de los principales motores de la ingeniería de software. Los enfoques que logran producir software de alta calidad con alta productividad pero que no pueden aceptar ni adaptarse al cambio tienen muy poca utilidad actualmente, ya que solo pueden resolver un número reducido de problemas que son resistentes al cambio.

1.3 El Enfoque de la Ingeniería de Software

Ahora entendemos el dominio del problema y los factores básicos que impulsan la ingeniería de software. Podemos considerar la **alta calidad y productividad (Q&P)** como el objetivo fundamental que **debe lograrse de manera consistente** para **problemas a gran escala** y **bajo la dinámica de los cambios**. La Q&P alcanzada durante un proyecto dependerá claramente de muchos factores, pero las tres fuerzas principales que gobiernan la Q&P son las **personas**, los **procesos** y la **tecnología**, a menudo llamadas el **Triángulo de Hierro**, como se muestra en la Figura 1.5.



Figura 1.5: Triangulo de hierro

Por lo tanto, **para lograr una alta Q&P se debe usar buena tecnología, se deben aplicar buenos procesos o métodos, y las personas que realicen el trabajo deben estar debidamente capacitadas**. En la ingeniería de software, el enfoque está principalmente en los **procesos**, a lo que anteriormente nos referimos como un enfoque sistemático en la definición dada. Como los procesos constituyen el corazón de la ingeniería de software (con herramientas y tecnología que brindan apoyo para ejecutar los procesos de manera eficiente), en este libro nos centraremos principalmente en los procesos. El proceso es lo que nos lleva desde las necesidades del usuario hasta el software que satisface dichas necesidades.

El enfoque básico de la ingeniería de software es separar el **proceso de desarrollo de software** del **producto desarrollado** (es decir, el software). La premisa es que, en gran medida, el proceso de software determina la calidad del producto y la productividad alcanzada. Por lo tanto, para abordar el dominio del problema y enfrentar con éxito los desafíos que afronta la ingeniería de software, se debe enfocar en el proceso de software. El diseño de procesos de software adecuados y su control se convierten entonces en un objetivo clave de la investigación en ingeniería de software.

Es este enfoque en el **proceso** lo que distingue a la ingeniería de software de la mayoría de las otras disciplinas de la computación. La mayoría de las otras disciplinas de la computación se enfocan en algún tipo de producto—algoritmos, sistemas operativos, bases de datos, etc.—mientras que la ingeniería de software se centra en el **proceso para producir los productos**. Es, esencialmente, el equivalente en software de la “ingeniería de manufactura”.

Aunque discutiremos más sobre los procesos en el próximo capítulo, aquí hablaremos brevemente de dos aspectos clave: **el proceso de desarrollo** y **la gestión del proceso de desarrollo**.

1.3.1 Proceso de Desarrollo por Fases

Un proceso de desarrollo consiste en varias fases, cada fase terminando con una salida definida. Las fases se realizan en un orden especificado por el modelo de proceso que se está siguiendo. La razón principal de tener un proceso por fases es que divide el problema de desarrollar software en realizar con éxito un conjunto de fases, cada una manejando una preocupación diferente del desarrollo de software.

Esto asegura que el costo del desarrollo sea más bajo de lo que hubiera sido si todo el problema se abordara de una sola vez. Además, un proceso por fases permite una verificación adecuada de calidad y de progreso en algunos puntos definidos durante el desarrollo (al final de las fases). Sin esto, uno tendría que esperar hasta el final para ver qué software ha sido producido. Claramente, esto no funcionará para sistemas grandes.

Por lo tanto, para manejar la complejidad, el seguimiento del proyecto y la calidad, todos los procesos de desarrollo consisten en un conjunto de fases. Un proceso de desarrollo por fases es central para el enfoque de ingeniería de software para resolver la crisis del software.

Se han propuesto varios modelos de proceso para desarrollar software. De hecho, la mayoría de las organizaciones que siguen un proceso tienen su propia versión. Discutiremos algunos de los modelos comunes en el próximo capítulo. En general, sin embargo, podemos decir que cualquier resolución de problemas en software debe consistir en:

- la **especificación de requisitos** para entender y declarar claramente el problema,
- el **diseño** para decidir un plan de solución,
- la **codificación** para implementar la solución planificada, y
- la **prueba** para verificar los programas.

Para problemas pequeños, estas actividades pueden no hacerse explícitamente, los límites de inicio y fin de estas actividades pueden no estar claramente definidos, y no se puede mantener un registro

escrito de las actividades. Sin embargo, los enfoques sistemáticos requieren que cada una de estas cuatro actividades de resolución de problemas se realice formalmente.

De hecho, para sistemas grandes, cada actividad en sí misma puede ser extremadamente compleja, y se necesitan metodologías y procedimientos para realizarlas de manera eficiente y correcta. Aunque diferentes modelos de proceso llevarán a cabo estas fases de manera diferente, ellas existen en todos los procesos. Discutiremos diferentes modelos de proceso en el próximo capítulo. Aquí discutimos brevemente estas fases básicas; cada una de ellas será tratada en más detalle a lo largo del libro (hay al menos un capítulo para cada una de estas fases).

Análisis de Requisitos

El análisis de requisitos se realiza para **entender el problema** que el sistema de software debe resolver.

El énfasis en el análisis de requisitos está en **identificar qué se necesita del sistema, no cómo el sistema alcanzará sus objetivos**. Para sistemas complejos, incluso determinar qué se necesita es una tarea difícil. El objetivo de esta actividad es documentar los requisitos en un **documento de especificación de requisitos de software**.

Hay dos actividades principales en esta fase:

1. **Análisis del problema** → comprender el problema, su contexto y los requisitos del nuevo sistema.
2. **Especificación de requisitos** → detallar lo comprendido en un documento formal.

Entender los requisitos de un sistema que aún no existe es complicado y requiere pensamiento creativo. El problema se vuelve más complejo porque un sistema automatizado ofrece posibilidades que no existirían de otra manera. En consecuencia, incluso los propios usuarios pueden no saber con claridad cuáles son sus necesidades.

Una vez analizado el problema y comprendidos los aspectos esenciales, los requisitos deben especificarse en un documento de requisitos, el cual debe incluir:

- todos los **requisitos funcionales** y de **rendimiento**;
- los formatos de entradas y salidas;
- todas las **restricciones de diseño** existentes (políticas, económicas, ambientales, de seguridad, etc.).

En otras palabras, además de la funcionalidad requerida, todos los factores que puedan afectar el diseño y el correcto funcionamiento del sistema deben especificarse en el documento de requisitos. Un **manual preliminar de usuario**, que describa las interfaces principales, suele formar parte del documento de requisitos.

Diseño de Software

El propósito de la fase de diseño es **planificar una solución** al problema especificado en el documento de requisitos. Esta fase es el primer paso en el paso del **dominio del problema** al

dominio de la solución. Dicho de otra manera: comenzando con lo que se necesita, el diseño nos lleva hacia cómo satisfacer esas necesidades.

El diseño de un sistema es quizá el factor más crítico que afecta la **calidad del software**, ya que tiene un gran impacto en las fases posteriores, especialmente en **pruebas** y **mantenimiento**.

La actividad de diseño suele producir tres salidas:

1. **Diseño de arquitectura** → define el sistema como un conjunto de componentes y cómo interactúan entre sí.
2. **Diseño de alto nivel** → identifica los módulos a construir y sus especificaciones.
3. **Diseño detallado** → define la lógica interna de cada módulo.

En resumen:

- En **arquitectura**, el foco está en qué grandes componentes se necesitan.
- En **diseño de alto nivel**, el foco está en qué módulos se necesitan.
- En **diseño detallado**, el foco está en cómo implementar esos módulos en software.

Una **metodología de diseño** es un enfoque sistemático para crear un diseño aplicando un conjunto de técnicas y guías. La mayoría de las metodologías se centran en el **diseño de alto nivel**.

Codificación

Una vez que el diseño está completo, la mayoría de las decisiones principales sobre el sistema ya se han tomado. Sin embargo, muchos detalles sobre la codificación —que dependen del lenguaje de programación elegido— no están especificados en el diseño.

El objetivo de la fase de codificación es **traducir el diseño en código** en un lenguaje de programación. El propósito es implementar el diseño de la mejor manera posible.

La fase de codificación afecta profundamente tanto a las pruebas como al mantenimiento. **Un código bien escrito** puede reducir significativamente el esfuerzo en pruebas y mantenimiento. Dado que los costos de pruebas y mantenimiento son mucho más altos que los de codificación, el objetivo debe ser reducir el esfuerzo posterior.

Por ello, durante la codificación el énfasis debe estar en **desarrollar programas fáciles de leer y entender**, no simplemente fáciles de escribir. La **simplicidad y claridad** deben ser prioridades.

Pruebas

Las pruebas son la principal medida de **control de calidad** durante el desarrollo de software. Su función básica es **detectar defectos** en el software.

Durante el análisis de requisitos y el diseño, la salida es un documento textual y no ejecutable. Después de la codificación, ya existen programas ejecutables que pueden probarse. Esto implica que las pruebas deben descubrir no solo errores introducidos durante la codificación, sino también errores de requisitos y de diseño.

El objetivo de las pruebas es, por lo tanto, **detectar errores de requisitos, diseño y codificación**.

El proceso suele seguir estas etapas:

- **Pruebas unitarias** → verificar módulos o componentes individuales.
- **Pruebas de integración** → verificar interacciones entre módulos.
- **Pruebas de sistema** → verificar que se cumplen todos los requisitos.
- **Pruebas de aceptación** → demostrar al cliente, con datos reales, que el sistema funciona.

Las pruebas son una actividad extremadamente crítica y que consume mucho tiempo. Requieren **planificación adecuada**, usualmente mediante un **plan de pruebas**, que define actividades, cronograma, recursos y guías.

El plan de pruebas incluye:

- condiciones a probar,
- unidades a verificar,
- cómo se integrarán los módulos.

Luego, para cada unidad de prueba se produce un **documento de especificación de casos de prueba**, con los casos y los resultados esperados.

Durante la ejecución, se comparan los resultados obtenidos con los esperados.

La salida final de la fase de pruebas es el **informe de pruebas** y el **informe de errores**, donde se detallan los casos ejecutados, resultados, errores encontrados y las acciones tomadas para corregirlos.

1.3.2 Gestión del Proceso

Como se mencionó antes, un proceso de desarrollo por fases es central para el enfoque de la ingeniería de software. Sin embargo, un proceso de desarrollo no especifica cómo asignar recursos a las diferentes actividades del proceso. Tampoco especifica cosas como el cronograma de actividades, cómo dividir el trabajo dentro de una fase, cómo asegurar que cada fase se realice correctamente, o cuáles son los riesgos del proyecto y cómo mitigarlos.

Sin una gestión adecuada de estos aspectos relacionados con el proceso, es poco probable que se puedan cumplir los objetivos de costo y calidad. Estas cuestiones relacionadas con la **gestión del proceso de desarrollo de un proyecto** se manejan mediante **la gestión de proyectos**.

Las actividades de gestión suelen girar en torno a un **plan**. Un plan de software forma la base que se usa intensamente para **monitorear y controlar** el proceso de desarrollo del proyecto. Esto convierte a la planificación en la actividad más importante de la gestión de proyectos. Se puede afirmar con seguridad que, sin una adecuada planificación, es muy poco probable que un proyecto de software cumpla sus objetivos. A la planificación del proyecto le dedicaremos un capítulo completo.

Gestionar un proceso requiere **información** sobre la cual basar las decisiones de gestión. De lo contrario, ni siquiera preguntas esenciales como: *¿se está cumpliendo el cronograma del proyecto?*, *¿cuál es el grado de sobre costo?*, *¿se están cumpliendo los objetivos de calidad?* pueden responderse.

Además, la información subjetiva es solo un poco mejor que no tener ninguna (ejemplo:

—P: ¿Qué tan cerca están de terminar?

—R: Ya casi lo logramos).

Por eso, para gestionar un proceso de manera efectiva, se necesita **información objetiva**. Para esto se usan las **métricas de software**.

Las **métricas de software** son medidas **cuantificables** que pueden usarse para medir diferentes características de un sistema de software o del proceso de desarrollo. Existen dos tipos:

- **Métricas de producto:** cuantifican características del producto en desarrollo (el software).
- **Métricas de proceso:** cuantifican características del proceso usado para desarrollar el software. Estas buscan medir aspectos como la productividad, costos y necesidades de recursos, efectividad de las medidas de control de calidad, y el efecto de las técnicas y herramientas de desarrollo.

Las **métricas y la medición** son aspectos necesarios para gestionar un proyecto de software. Para un monitoreo efectivo, la gestión necesita información sobre el proyecto: cuánto ha progresado, cuánto desarrollo se ha realizado, qué tan retrasado está respecto al cronograma, y cuál es la calidad alcanzada hasta el momento.

Con esta información, pueden tomarse decisiones sobre el proyecto. Sin métricas adecuadas para cuantificar esta información, habría que basarse en opiniones subjetivas, que a menudo son poco confiables y van en contra de los principios de la ingeniería.

Por lo tanto, podemos decir que la **gestión basada en métricas** también es un componente clave en la estrategia de la ingeniería de software para alcanzar sus objetivos.

Aunque nos hemos enfocado en la **gestión del proceso de desarrollo de un proyecto**, existen otros aspectos de la gestión de procesos de software. Algunos de estos se discutirán en el próximo capítulo.

1.4 Resumen

El costo del software ahora constituye la principal parte del costo de un sistema de cómputo. Actualmente, el software es **extremadamente costoso** de desarrollar y con frecuencia es **poco confiable**.

En este capítulo, hemos discutido algunos temas sobre el software y la ingeniería de software:

1. El dominio del problema de la ingeniería de software es el **software de nivel industrial**.
2. Este software no es solo un conjunto de programas, sino que incluye programas, datos asociados y documentación.

El software de nivel industrial es:

- Costoso y difícil de construir,
 - Costoso de mantener debido a cambios y reprocesamiento,
 - Con altos requisitos de calidad.
3. La **ingeniería de software** es la disciplina que busca proporcionar métodos y procedimientos para desarrollar software de nivel industrial de manera sistemática.

Las principales fuerzas que impulsan a la ingeniería de software son: **el problema de escala, la calidad y productividad (Q&P), la consistencia y el cambio.**

Alcanzar una alta Q&P de forma consistente, en problemas de gran escala y donde los cambios ocurren de manera continua, es el principal desafío de la ingeniería de software.

4. El **enfoque fundamental** de la ingeniería de software para lograr sus objetivos es separar el **proceso de desarrollo** de los **productos**.

La ingeniería de software se centra en el proceso porque la calidad de los productos desarrollados y la productividad alcanzada dependen en gran medida de él.

Para enfrentar los retos de la ingeniería de software, este proceso de desarrollo es **por fases**.

Otro enfoque clave es **gestionar el proceso de forma eficaz y proactiva usando métricas**.

3 Análisis y Especificación de Requerimientos de Software

La complejidad y el tamaño de los sistemas de software están aumentando continuamente.

A medida que la escala cambia hacia sistemas de software más grandes y complejos, surgen nuevos problemas que no existían en sistemas más pequeños (o que eran de menor importancia), lo que conduce a una redefinición de las prioridades de las actividades que intervienen en el desarrollo de software.

Los requerimientos de software son una de esas áreas, a las que se les daba poca importancia en los primeros días del desarrollo de software, ya que el énfasis estaba en la codificación y el diseño. La suposición tácita era que los desarrolladores entendían claramente el problema cuando se les explicaba, generalmente de manera informal.

A medida que los sistemas se hicieron más complejos, se volvió evidente que los objetivos de todo el sistema no podían ser comprendidos con facilidad. De ahí surgió la necesidad de un análisis de requerimientos más riguroso. Hoy en día, para los grandes sistemas de software, el análisis de requerimientos es quizás la actividad más difícil e intratable; también es muy propensa a errores. Muchos creen que la disciplina de la ingeniería de software es más débil en esta área crítica.

Parte de la dificultad se debe al alcance de esta actividad. El proyecto de software se inicia a partir de las necesidades del cliente. Al comienzo, estas necesidades están en la mente de distintas personas dentro de la organización del cliente. El analista de requerimientos debe identificar estos requerimientos hablando con dichas personas y comprendiendo sus necesidades.

En situaciones donde el software busca automatizar un proceso actualmente manual, muchas de las necesidades pueden entenderse observando la práctica actual. Pero no existen métodos similares para sistemas en los que los procesos manuales no existen, o para las “nuevas funcionalidades” que suelen agregarse al automatizar un proceso existente. Para esos sistemas, el problema de los requerimientos se complica aún más por el hecho de que las necesidades y los requerimientos del sistema pueden no ser conocidos ni siquiera por el propio usuario —deben ser visualizados y creados.

Por lo tanto, identificar requerimientos implica necesariamente especificar lo que algunas personas tienen en sus mentes (o lo que llegará a sus mentes cuando lo visualicen). Como la información en sus mentes no está formalmente declarada ni organizada, la entrada a la fase de especificación de requerimientos de software es, por naturaleza, informal e imprecisa, y es probable que esté incompleta. Cuando se deben recopilar entradas de múltiples personas, estas también tienden a ser inconsistentes.

La fase de requerimientos traduce las ideas en la mente de los clientes (la entrada) en un documento formal (la salida de la fase de requerimientos). Así, el resultado de la fase es un conjunto de requerimientos especificados con precisión, que idealmente sean completos y consistentes, mientras que la entrada no posee ninguna de estas propiedades.

Es evidente que el proceso de especificación de requerimientos no puede ser totalmente formal; cualquier proceso de traducción formal que produzca una salida formal debe tener una entrada precisa y no ambigua. Esta es la razón por la cual la actividad de requerimientos de software no

puede ser completamente automatizada, y cualquier método para identificarlos puede ser, como máximo, un conjunto de guías.

En este capítulo discutiremos qué son los requerimientos, por qué la especificación de requerimientos es importante, cómo se analizan y especifican los requerimientos, cómo se validan y algunas métricas que se pueden aplicar a los requerimientos. Finaliza con una discusión sobre el **SRS** (Software Requirements Specification) de los casos de estudio utilizados en el libro.

3.1 Requerimientos de Software

El **IEEE** define un requerimiento como:

(1) Una *condición o capacidad que un usuario necesita para resolver un problema o alcanzar un objetivo*;

(2) Una *condición o capacidad que debe cumplir o poseer un sistema... para satisfacer un contrato, estándar, especificación u otro documento formalmente impuesto*. [91]

Es importante notar que en los **requerimientos de software** estamos tratando con los requerimientos del sistema propuesto, es decir, *las capacidades que el sistema —que aún no se ha desarrollado— debería tener*. Es precisamente porque se trata de especificar un sistema que no existe todavía que el problema de los requerimientos se vuelve complicado.

El objetivo de la actividad de requerimientos es producir la **Especificación de Requerimientos de Software (SRS)**, la cual describe **qué debe hacer el software propuesto**, sin describir **cómo lo hará**.

Producir el SRS es más fácil de decir que de hacer. Una limitación básica es que las necesidades del usuario cambian constantemente, a medida que cambia el entorno en el cual el sistema debe funcionar. Aunque es inevitable aceptar que algunos cambios en los requerimientos ocurrirán, aún existen razones de peso por las cuales debe hacerse un trabajo minucioso en la fase de requerimientos, de modo que el SRS resultante sea de alta calidad y relativamente estable.

3.1.1 Necesidad del SRS

El origen de la mayoría de los sistemas de software está en las **necesidades de los clientes**. El sistema de software es creado por los **desarrolladores**, y finalmente será usado por los **usuarios finales**. Así, existen tres partes principales interesadas en un nuevo sistema:

- el **cliente**,
- el **desarrollador**,
- y los **usuarios**.

De algún modo, los requerimientos del sistema que satisfagan las necesidades del cliente y las preocupaciones de los usuarios deben ser comunicados al desarrollador.

El problema es que:

- el cliente usualmente **no entiende de software ni del proceso de desarrollo**,
- y el desarrollador a menudo **no comprende el problema ni el área de aplicación del cliente**.

Esto genera una **brecha de comunicación** entre las partes. Un propósito fundamental de la especificación de requerimientos de software es **cerrar esa brecha**.

El SRS es el medio a través del cual las necesidades del cliente y del usuario se especifican con precisión para el desarrollador.

De allí se derivan varias ventajas importantes:

- **Un SRS establece la base de acuerdo entre el cliente y el proveedor sobre lo que hará el producto de software.**

Este acuerdo suele formalizarse en un **contrato legal** entre el cliente (o consumidor) y el desarrollador (el proveedor). A través del SRS, el cliente expresa claramente lo que espera, y el desarrollador entiende con claridad qué capacidades debe construir en el software.

Sin un acuerdo de este tipo, casi con seguridad, al finalizar el desarrollo habrá un **cliente insatisfecho**, lo que suele llevar a **desarrolladores insatisfechos**.

(La situación clásica: Cliente: “¡Oye! esto es un error”; Desarrollador: “No, es una característica del software”).

- **Un SRS provee una referencia para la validación del producto final.**

Es decir, ayuda al cliente a determinar si el software cumple con los requerimientos.

Sin un SRS adecuado, el cliente no tiene manera de comprobar si el software entregado es el que se pidió, ni el desarrollador tiene cómo convencer al cliente de que todos los requerimientos han sido satisfechos.

Estas dos razones —**acuerdo y validación**— deberían ser suficientes para justificar la necesidad de un SRS riguroso. Sin embargo, hay razones aún más prácticas y urgentes.

Errores en los requerimientos y su impacto

Sabemos que los principales factores que impulsan un proyecto son: **costo, cronograma y calidad**.

En un estudio (Boehm, citado en [45]):

- En algunos proyectos, el **54% de todos los errores detectados** se encontraron después de la codificación y pruebas unitarias.
- El **45% de esos errores** se originaron en la etapa de requerimientos y en el diseño temprano.
- En otras palabras, **aprox. 25% de los errores ocurren en las etapas iniciales de requerimientos y diseño**.

Ejemplos reportados:

- En el proyecto A-7 se encontraron unos **80 errores** en el documento de requerimientos en pocos meses, lo que generó solicitudes de cambio [8].
- Otro informe indica que se encontraron más de **500 errores** en un SRS previamente aprobado [45].
- En otro proyecto, más de **250 errores** fueron encontrados en un SRS revisado anteriormente, al reescribirlo de forma estructurada y usar herramientas de análisis [47].

Está claro que muchos errores se cometen en la fase de requerimientos.

Y un error en el SRS **casi siempre se manifestará en el sistema final**, porque si el documento

especifica un sistema incorrecto (que no satisface los objetivos del cliente), entonces incluso una implementación “correcta” del SRS dará como resultado un sistema que no sirve al cliente.

De ahí que podamos concluir:

- **Un SRS de alta calidad es un requisito previo para obtener un software de alta calidad.**

Impacto en costos

El costo de corregir un error **crece exponencialmente** a medida que pasa el tiempo.

Un error en los requerimientos, si se detecta después del desarrollo, puede costar **hasta 100 veces más** que corregirlo en la fase de requerimientos.

Ejemplo con datos:

- Corregir un error de requerimientos en la fase temprana: **~2 horas-persona** en promedio.
- Si se detecta más tarde, el costo crece de acuerdo con el avance del proyecto.

Phase	Cost (person-hours)
Requirements	2
Design	5
Coding	15
Acceptance test	50
Operation and maint.	150

Tabla 3.1: Costo de corregir errores de requisitos

Tabla 3.1 resume los costos relativos de corregir errores dependiendo de la fase (según estudios multicorporativos [17, 20]).

Ejemplo simplificado:

- Invertir **100 horas-persona adicionales** en la fase de requerimientos permite detectar unos **50 errores nuevos**.
- Si esos errores no se corrigen ahí, serán encontrados más adelante, con el siguiente patrón (como en el proyecto A-7 [8]):
 - 65% en diseño
 - 2% en codificación
 - 30% en pruebas
 - 3% en operación y mantenimiento

El costo total de corregirlos más tarde sería:

$$32.5 * 5 + 1 * 15 + 15 * 50 + 1.5 * 150 = 1152 \text{ horas-persona}$$

En otras palabras: invertir **100 horas-persona adicionales en requerimientos** reduce el costo total en **~1052 horas-persona**.

Otra manera de verlo:

La probabilidad de detectar un error en fases posteriores es:

- 0.4 en diseño,
- 0.1 en codificación y pruebas unitarias,
- 0.4 en pruebas de aceptación,
- 0.1 en operación/mantenimiento.

El **costo esperado** de corregir un error no eliminado en requerimientos es:

$$0.4 * 5 + 0.1 * 15 + 0.4 * 50 + 0.1 * 150 = 38.5 \text{ horas-persona}$$
$$0.4 * 5 + 0.1 * 15 + 0.4 * 50 + 0.1 * 150 = 38.5 \text{ horas-persona}$$

Entonces, **si detectar y corregir un error en requerimientos cuesta menos que 38.5 horas, conviene hacerlo ahí mismo.**

Requerimientos cambiantes y re-trabajo

Los requerimientos cambian con frecuencia. Aunque algunos cambios son inevitables (porque cambian las necesidades y percepciones), **muchos cambios ocurren porque los requerimientos no fueron bien analizados ni validados.**

Un SRS de alta calidad reduce significativamente esos cambios innecesarios.

Esto es crítico porque:

- los cambios aumentan los costos,
- y pueden descontrolar el cronograma del proyecto.

Ejemplo simplificado:

- Se estima que entre **20% y 40% del esfuerzo total** en un proyecto de software se debe a **re-trabajo**, gran parte causado por cambios en requerimientos.
- Según el modelo **COCOMO** [20], el costo de la fase de requerimientos suele ser de ~6% del costo total del proyecto.

Supongamos un proyecto de **50 meses-persona**:

- Requerimientos consumen **3 meses-persona**.
- Si invertimos un **33% adicional de esfuerzo** en esa fase, y logramos reducir los cambios en requerimientos en 33%, el retrabajo bajará de **10–20 meses-persona** a **6–12 meses-persona**.
- Esto implica un ahorro de **5–11 meses-persona**, es decir, un **10%–22% del costo total del proyecto**.

Conclusión

De todo lo anterior se desprende que:

- **Un SRS de alta calidad reduce los costos de desarrollo.**

Por lo tanto, la calidad del SRS impacta directamente en:

- la **satisfacción del cliente (y del desarrollador)**,
- la **validación del sistema**,
- la **calidad del software final**,
- y el **costo del desarrollo**.

El papel crítico que desempeña el SRS en un proyecto de desarrollo de software debería ser evidente.

3.1.2 Proceso de Requerimientos

El proceso de requerimientos es la secuencia de actividades que deben realizarse en la fase de requerimientos y que culminan en la producción de un documento de alta calidad que contiene la **Especificación de Requerimientos de Software (ERS o SRS por sus siglas en inglés)**.

El proceso de requerimientos típicamente consiste en tres tareas básicas:

- **análisis del problema o requerimientos**,
- **especificación de requerimientos**, y
- **validación de requerimientos**.

El **análisis del problema** suele comenzar con una "declaración del problema" a alto nivel. Durante el análisis, se modela el dominio del problema y el entorno en un esfuerzo por entender el comportamiento del sistema, las restricciones sobre el sistema, sus entradas y salidas, etc. El propósito básico de esta actividad es obtener una comprensión profunda de lo que el software debe proveer.

La comprensión obtenida mediante el análisis del problema forma la base para la **especificación de requerimientos**, en la cual el enfoque está en especificar claramente los requerimientos en un documento. En esta actividad se abordan temas como la representación, los lenguajes de especificación y las herramientas. Dado que el análisis produce grandes cantidades de información y conocimiento con posibles redundancias, **organizar y describir los requerimientos de manera adecuada** es un objetivo importante de esta actividad.

La **validación de requerimientos** se centra en asegurar que lo especificado en la ERS son efectivamente todos los requerimientos del software y en garantizar que la ERS sea de buena calidad. El proceso de requerimientos termina con la producción de la **ERS validada**.

Aunque parece que el proceso de requerimientos es una secuencia lineal de estas tres actividades, en realidad no es así, excepto para sistemas triviales. En la mayoría de sistemas reales, hay una superposición considerable y retroalimentación entre estas actividades. Así, algunas partes del sistema se analizan y luego se especifican mientras el análisis de otras partes sigue en curso.

Además, si las actividades de validación revelan problemas en la ERS, es probable que conduzcan a más análisis y especificación. Sin embargo, en general, para una parte del sistema, **el análisis precede a la especificación y la especificación precede a la validación**.

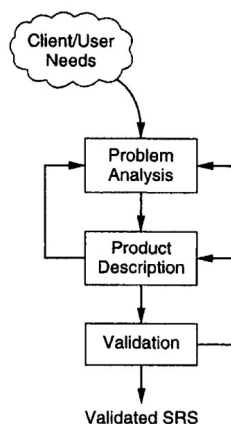


Figura 3.1: El proceso de requerimientos

Este proceso de requerimientos se muestra en la **Figura 3.1**.

Como se muestra en la figura, desde la actividad de especificación podemos regresar a la actividad de análisis. Esto sucede con frecuencia, ya que algunas partes del problema se analizan y especifican antes de que otras lo sean. Asimismo, el proceso de especificación suele mostrar deficiencias en el conocimiento del problema, lo que requiere más análisis. Una vez que la especificación está "completa", pasa a la validación. Esta actividad puede revelar problemas en las especificaciones mismas (lo que requiere regresar a la especificación) o deficiencias en la comprensión del problema (lo que requiere regresar al análisis).

Durante el **análisis de requerimientos**, el enfoque está en **comprender el sistema y sus requerimientos**. Para un sistema complejo, esta es una tarea difícil, y el método probado en el tiempo de **“dividir y conquistar”** (es decir, descomponer el problema o sistema en partes más pequeñas y luego entender esas partes y sus relaciones) se aplica inevitablemente para manejar la complejidad. Además, para manejar la complejidad y el gran volumen de información disponible durante el análisis, se usan diversas **estructuras de representación** que ayudan a visualizar el sistema como una serie de abstracciones. Ejemplos de estas estructuras son los **diagramas de flujo de datos** y los **diagramas de objetos** (más sobre esto en la próxima sección). Para una parte del sistema, el análisis normalmente precede a la especificación. Una vez finalizado el análisis y construidas las estructuras, esa parte del sistema debe especificarse.

La transición del **análisis** a la **especificación**, sin embargo, aunque parece que debería ser sencilla, no lo es. De hecho, puede ser bastante difícil. La razón es que los objetivos de ambas actividades son distintos. En la **especificación** debemos indicar únicamente lo que el software debe hacer, es decir, enfocarnos en el **comportamiento externo** del sistema. Para identificar todos los comportamientos externos, es necesario comprender claramente la estructura del problema y sus componentes, además de sus entradas y salidas. Sin embargo, la estructura en sí puede no ser muy útil en la especificación, ya que el enfoque está exclusivamente en el comportamiento externo o en el sistema final, no en la estructura interna del dominio del problema. Por esto, no se debe esperar que, una vez realizado el análisis, la especificación sea directa.

Además, muchos de los “resultados” del análisis no se utilizan directamente en la ERS. Esto no significa que no sean útiles; son esenciales para modelar el problema, lo cual conduce a una

comprensión adecuada de los requerimientos, que es un **prerrequisito** para la especificación. Por tanto, el uso de las actividades de análisis y las estructuras que se construyen pueden ser **indirectos**, ayudando a la comprensión más que directamente a la especificación.

Vale la pena señalar que existen similitudes entre la **actividad de análisis** y la **actividad de diseño**. Como señaló Davis [45], el problema básico durante el diseño de software es el mismo: **manejar la complejidad**. El enfoque usado es similar: descomposición y construcción de estructuras para representar el sistema como una serie de abstracciones. Debido a esta similitud, los enfoques usados para el análisis del problema y el diseño son frecuentemente similares (por ejemplo, los diagramas de flujo de datos y diagramas de objetos se utilizan en análisis y también en diseño). Sin embargo, aunque los enfoques son similares, los objetivos de ambas actividades son completamente diferentes:

- el **análisis** trata con el dominio del problema, con el objetivo de entenderlo,
- el **diseño** trata con el dominio de la solución, con el objetivo de **optimizar el diseño** [45].

Debido a esto, la aplicación de enfoques similares produce estructuras diferentes durante el análisis y durante el diseño. A veces se cree erróneamente que las estructuras producidas en el análisis se deben arrastrar hasta el diseño. Esto proviene de una **mala comprensión** de los objetivos de ambas actividades. Aunque algunas estructuras eventualmente se usen en el diseño, esto debe hacerse solo si son **consistentes con el objetivo del diseño**.

Finalmente, está el tema del **nivel de detalle** que debe descubrirse y especificarse en el proceso de requerimientos. Este es un asunto que no puede resolverse fácilmente y que depende del objetivo de la fase de especificación de requerimientos.

- Si el objetivo es **definir las necesidades generales del sistema**, los requerimientos pueden expresarse en forma abstracta. Estos se usan generalmente para realizar un **análisis de factibilidad** o para **licitaciones competitivas**.
- En el otro extremo están los requerimientos en los que todo el comportamiento y las interfaces externas del software están claramente especificados. Estos son muy detallados y adecuados para el **desarrollo del software**.

Un ejemplo puede ilustrar este punto. Supongamos que un fabricante de automóviles desea tener un sistema de **control de inventario**.

- A nivel abstracto, los requerimientos podrían expresarse en términos del número de piezas a rastrear, el nivel de concurrencia que debe soportar, si será en línea o por lotes, qué tipo de información y reportes debe proporcionar (por ejemplo: estado de cada ítem bajo demanda, órdenes de compra de artículos con inventario bajo, patrones de consumo), etc. Estos requerimientos abstractos pueden usarse para **estimar costos y realizar un análisis costo-beneficio**, así como para invitar a distintos desarrolladores a presentar propuestas.
- Sin embargo, tal especificación de requerimientos es de poca utilidad para un desarrollador contratado para construir el software. Ese desarrollador necesita saber el **formato exacto de los reportes**, todas las **consultas posibles y su estructura**, el **número total de terminales** que el sistema debe soportar, la **estructura de las bases de datos principales**, etc.

Todo esto sigue describiendo el **comportamiento externo del software**, pero desde un nivel más bajo de abstracción que define cómo se satisfacen los requerimientos generales.

Como debería quedar claro, el nivel abstracto **no es adecuado para el desarrollo de software**. Por eso, nos enfocaremos principalmente en los requerimientos de **nivel detallado**, en los cuales se especifica toda la información sobre el comportamiento externo que el desarrollador necesita para construir el sistema. Es decir, consideramos la ERS como un documento que **proporciona toda la información necesaria para que el desarrollador construya el sistema correctamente**.

Sin embargo, es importante señalar que incluso cuando el objetivo es obtener los requerimientos detallados, los **requerimientos abstractos** pueden desempeñar un papel útil en sistemas complejos. Dado que el análisis del problema comienza con una descripción inicial del comportamiento o necesidades del sistema, los requerimientos abstractos pueden desempeñar este papel (además de servir para licitaciones competitivas o análisis de factibilidad).

En otras palabras, **especificar los requerimientos a nivel abstracto puede ser útil para producir la ERS** que contiene los requerimientos detallados del sistema.

Las siguientes secciones estarán dedicadas a proporcionar una descripción más detallada de las actividades en las tres principales de la fase de requerimientos:

- análisis,
- especificación, y
- verificación.

3.2 Análisis del Problema

El objetivo básico del análisis del problema es obtener una comprensión clara de las necesidades de los clientes y los usuarios, qué es exactamente lo que se desea del software, y cuáles son las restricciones de la solución [45].

Con frecuencia, el cliente y los usuarios no entienden o no conocen todas sus necesidades, porque el potencial del nuevo sistema no suele apreciarse completamente. Los analistas deben asegurarse de descubrir las verdaderas necesidades de los clientes y usuarios, incluso si ellos mismos no las conocen con claridad. Es decir, los analistas no solo recopilan y organizan información sobre la organización del cliente y sus procesos, sino que también actúan como consultores que desempeñan un papel activo ayudando a los clientes y usuarios a identificar sus necesidades.

El principio básico utilizado en el análisis es el mismo que en cualquier tarea compleja: **divide y vencerás**. Es decir, dividir el problema en subproblemas e intentar comprender cada subproblema y su relación con los demás, en un esfuerzo por entender el problema total.

Los conceptos de *estado* y *proyección* también pueden usarse eficazmente en el proceso de partición. Un estado de un sistema representa ciertas condiciones sobre el mismo. Frecuentemente, al usar estados, se ve primero al sistema como si operara en uno de los varios estados posibles, y luego se realiza un análisis detallado de cada estado. Este enfoque se utiliza a veces en software en tiempo real o en software de control de procesos.

En la *proyección*, un sistema se define desde múltiples puntos de vista [152]. Al usar proyección, se definen diferentes perspectivas del sistema y luego se analiza el sistema desde esas diferentes perspectivas. Las distintas “proyecciones” obtenidas se combinan para formar el análisis completo

del sistema. Analizar el sistema desde distintas perspectivas suele ser más sencillo, ya que limita y enfoca el alcance del estudio.

En el resto de esta sección discutiremos algunos métodos para el análisis del problema. Como el objetivo del análisis es comprender el dominio del problema, un analista debe estar familiarizado con diferentes métodos de análisis y elegir el enfoque que considere más adecuado para el problema en cuestión.

3.2.1 Enfoque Informal

El enfoque informal de análisis es aquel en el que no se utiliza ninguna metodología definida. Como en cualquier enfoque, la información sobre el sistema se obtiene mediante interacción con el cliente, usuarios finales, cuestionarios, estudio de documentos existentes, sesiones de *brainstorming*, etc.

Sin embargo, con este enfoque no se construye ningún modelo formal del sistema. El problema y el modelo del sistema se construyen esencialmente en la mente de los analistas (o los analistas pueden usar alguna notación informal para este fin) y se traducen directamente de la mente de los analistas al SRS (*Software Requirements Specification* o Especificación de Requerimientos de Software).

Con frecuencia, en este tipo de enfoque, el analista tendrá una serie de reuniones con los clientes y usuarios finales. En las primeras reuniones, los clientes y usuarios finales le explicarán al analista acerca de su trabajo, su entorno y sus necesidades según las perciben. Se pueden entregar documentos que describan el trabajo o la organización, junto con salidas de los métodos actuales de realización de tareas. En estas reuniones iniciales, el analista es básicamente un oyente, absorbiendo la información proporcionada.

Una vez que el analista entiende el sistema hasta cierto punto, usa las siguientes reuniones para aclarar las partes que no comprende. Puede documentar la información de alguna manera (incluso puede construir un modelo si lo desea) y puede realizar sesiones de *brainstorming* o reflexionar sobre lo que el sistema debería hacer. En las últimas reuniones, el analista básicamente explica al cliente lo que entiende que el sistema debe hacer y utiliza estas reuniones como un medio de verificación para comprobar si lo que propone que el sistema haga es consistente con los objetivos del cliente. En estas reuniones finales puede usarse un borrador inicial del SRS.

El enfoque informal de análisis se usa ampliamente y puede ser bastante útil. La razón de su utilidad es que los enfoques basados en modelado conceptual frecuentemente no modelan todos los aspectos del problema y no siempre son adecuados para todos los problemas. Además, dado que el SRS debe ser validado y la retroalimentación de la actividad de validación puede requerir más análisis o especificación (ver Figura 3.1), elegir un enfoque informal de análisis no es muy riesgoso: los errores que puedan introducirse no necesariamente pasarán desapercibidos en la fase de requerimientos. Por lo tanto, este tipo de enfoques pueden ser la alternativa más práctica para el análisis en algunas situaciones.

3.2.2 Modelado de Flujo de Datos

El modelado basado en flujo de datos, a menudo referido como la técnica de análisis estructurado, utiliza descomposición basada en funciones mientras modela el problema. Se enfoca en las funciones realizadas en el dominio del problema y en los datos consumidos y producidos por estas funciones. Es un enfoque de refinamiento de arriba hacia abajo, que originalmente se llamó análisis

y especificación estructurada, y fue propuesto para producir las especificaciones. Sin embargo, limitaremos nuestra atención al aspecto de análisis del enfoque. Antes de describir el enfoque, describamos el diagrama de flujo de datos y el diccionario de datos en los que la técnica se apoya fuertemente.

Diagramas de Flujo de Datos y Diccionario de Datos

Los **diagramas de flujo de datos** (también llamados *data flow graphs*) se utilizan comúnmente durante el análisis de problemas. Estos diagramas (DFD) son bastante generales y no están limitados únicamente al análisis de problemas para la especificación de requerimientos de software. De hecho, ya se utilizaban mucho antes de que la ingeniería de software existiera como disciplina. Los DFD resultan muy útiles para comprender un sistema y pueden emplearse eficazmente durante la etapa de análisis.

Un DFD muestra el **flujo de datos a través de un sistema**. Considera al sistema como una función que transforma las entradas en salidas deseadas. Un sistema complejo no realiza esta transformación en un “único paso”; normalmente, los datos pasan por una serie de transformaciones antes de convertirse en la salida final. El objetivo de un DFD es capturar esas transformaciones que ocurren dentro del sistema sobre los datos de entrada, hasta que finalmente se obtiene la salida.

El agente que realiza la transformación de los datos de un estado a otro se denomina **proceso** (o *burbuja*). Por lo tanto, un DFD muestra cómo se mueven los datos a través de las diferentes transformaciones o procesos del sistema. Los procesos se representan mediante **círculos con nombre**, mientras que los **flujos de datos** se representan con **flechas con nombre** que entran o salen de esas burbujas. Un **rectángulo** representa una fuente o un destino, que son los puntos donde los datos se originan o se consumen. Usualmente, una fuente o destino se encuentra fuera del sistema principal que se está estudiando.

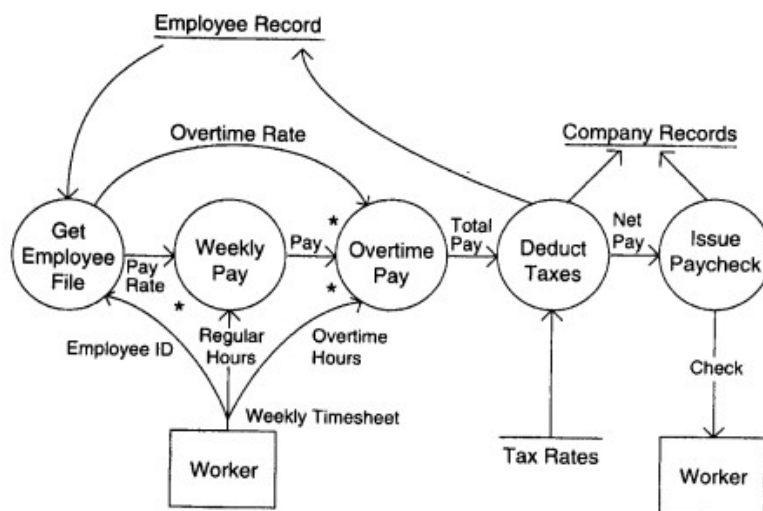


Figura 3.2: DFD de un sistema de pago de empleados

Un ejemplo de DFD para un sistema que paga a los empleados se muestra en la Figura 3.2.

En este DFD hay un flujo de entrada básico: la **planilla semanal de horas**, que proviene de la fuente “trabajador”. La salida principal es el **cheque de pago**, cuyo destino también es el trabajador.

En este sistema, primero se recupera el registro del empleado utilizando el número de identificación que aparece en la planilla. A partir de ese registro se obtienen la tarifa de pago y las horas extras. Con esa información, junto a las horas normales y extras de la planilla, se calcula el sueldo. Una vez determinado el pago total, se deducen los impuestos. Para calcular la deducción de impuestos se consulta el archivo de tasas impositivas. El monto descontado se registra tanto en los registros del empleado como en los de la compañía. Finalmente, se emite el cheque con el pago neto, y el monto abonado también queda registrado en los archivos de la empresa.

Algunas convenciones utilizadas en el dibujo de este DFD merecen explicación.

- Todos los **archivos externos** (como el registro de empleados, registro de la compañía y tasas de impuestos) se muestran como una **línea recta con etiqueta**.
- Cuando un proceso requiere **múltiples flujos de datos**, esto se indica con un ‘*’ entre los flujos. Este símbolo representa la relación **AND**. Por ejemplo, si un proceso tiene como entrada A * B, significa que necesita A **Y** B. En el DFD, para el proceso “pago semanal”, tanto las “horas” como la “tarifa” son necesarias, lo cual se muestra con este símbolo.
- De manera similar, la relación **OR** se representa con un ‘+’ entre los flujos de datos.

Este DFD es una **descripción abstracta** del sistema de pagos. No importa si el sistema es automático o manual. De hecho, este diagrama podría representar perfectamente un sistema manual donde los cálculos se hacen con calculadoras y los registros son carpetas físicas o libros contables. Los detalles y caminos de datos menores no aparecen en este nivel. Por ejemplo, no se muestra qué ocurre si hay errores en la planilla semanal. Esto se hace a propósito, para no atascarse con detalles al construir un DFD del sistema general. Si se desean más detalles, el DFD puede refinarse en niveles posteriores.

Es importante aclarar que un **DFD no es un diagrama de flujo (flowchart)**. El DFD representa **flujos de datos**, mientras que un flowchart representa **flujos de control**. Un DFD **no incluye información procedimental**. Por eso, al dibujar un DFD, no se debe pensar en detalles de procedimiento, y es necesario evitar conscientemente ese tipo de razonamiento. Consideraciones como bucles o decisiones deben ignorarse. Lo único que debe especificar el diseñador en el DFD son las **transformaciones principales** que los datos atraviesan desde la entrada hasta la salida. Cómo se realizan esas transformaciones no es relevante en este nivel.

No existen procedimientos estrictos para dibujar un DFD de un problema dado, solo **algunas pautas generales**.

Una manera de construir un DFD es comenzar identificando las **entradas y salidas principales**, ignorando inicialmente las menores (como mensajes de error). Después, partiendo desde las entradas, se trabaja hacia las salidas, identificando las principales transformaciones intermedias. Otra alternativa es trabajar en sentido inverso: desde las salidas hacia las entradas.

(Recuerda que la información procedimental, como bucles y decisiones, no debe aparecer en el DFD, y el diseñador no debe preocuparse por esos aspectos al dibujar el diagrama de flujo de datos).

A continuación, se presentan algunas sugerencias para la construcción de un DFD [154, 48]:

Trabaja de manera consistente desde las entradas hacia las salidas, o viceversa.

Si te atascas, cambia la dirección. Empieza con un **DFD de alto nivel**, con pocas transformaciones

principales que describan toda la conversión de entradas en salidas, y luego refina cada transformación en transformaciones más detalladas.

- **Nunca intentes mostrar lógica de control.**

Si te descubres pensando en términos de bucles o decisiones, es momento de detenerte y comenzar de nuevo.

- **Etiqueta cada flecha con los elementos de datos adecuados.**

Las entradas y salidas de cada transformación deben estar cuidadosamente identificadas.

- **Haz uso de las operaciones * y +, y muestra suficiente detalle en el diagrama de flujo de datos.**

- **Intenta dibujar DFDs alternativos antes de decidirte por uno.**

Muchos sistemas son demasiado grandes como para que un solo DFD pueda describir claramente el procesamiento de datos. En tales casos, es necesario utilizar algún mecanismo de **descomposición y abstracción**.

Los DFDs pueden organizarse jerárquicamente, lo que ayuda a ir particionando y analizando progresivamente sistemas grandes. A este conjunto de DFDs se le llama un **conjunto de DFDs nivelados** (*leveled DFD set* [48]).

Un **conjunto de DFDs nivelados** comienza con un DFD inicial, que es una representación muy abstracta del sistema, mostrando las entradas y salidas principales y los procesos más relevantes. Luego, cada proceso se refina, dibujando un nuevo DFD para ese proceso. En otras palabras, una **burbuja en un DFD se expande en otro DFD** durante el refinamiento.

Para que la jerarquía sea consistente, es importante que las **entradas y salidas netas** de un DFD de proceso coincidan con las entradas y salidas del proceso en el DFD de nivel superior. El refinamiento se detiene cuando cada burbuja se considera “atómica”, es decir, cuando puede especificarse o comprenderse fácilmente.

Es importante señalar que durante el refinamiento, aunque las entradas y salidas netas se preservan, también puede ocurrir una **descomposición de los datos**. Por ejemplo, una unidad de datos puede dividirse en sus componentes para el procesamiento cuando se dibuja el DFD detallado de un proceso. Así, a medida que se descomponen los procesos, también se descomponen los datos.

En un DFD, los **flujos de datos** se identifican con **nombres únicos**. Estos nombres deben transmitir algún significado sobre lo que representan. Sin embargo, la **estructura precisa** de los flujos de datos no se especifica en un DFD.

El **diccionario de datos** es un repositorio que contiene la definición de los diferentes flujos de datos presentes en un DFD. En él se establece con precisión la estructura de cada flujo. También se especifican los componentes de un flujo, así como la estructura de los archivos mostrados en el DFD.

Para definir la estructura de los datos se usan diferentes **notaciones**, similares a las de las expresiones regulares (que se discutirán más adelante en este capítulo). En esencia, además de la **secuencia o composición** (representada con ‘+’), se incluyen **selección e iteración**:

- La **selección** (representada con la barra vertical ‘|’) significa *uno u otro*.
- La **repetición** (representada con ‘*’) significa *una o más ocurrencias*.

En el DFD mostrado anteriormente, se utilizan los flujos de datos de la **planilla semanal**. El diccionario de datos correspondiente a este DFD se muestra en la Figura 3.3.

La mayoría de los flujos de datos del DFD están especificados allí. Algunos de los más obvios no se muestran. Por ejemplo, la entrada del diccionario para la **planilla semanal** indica que este flujo de datos está compuesto por tres entidades básicas:

1. nombre del empleado,
2. ID del empleado,
3. y múltiples ocurrencias de un par de datos que incluyen horas normales y horas extras.

La última entidad representa las horas de trabajo diarias del empleado. El diccionario de datos también contiene entradas que especifican los diferentes elementos de cada flujo de datos.

Verificación de un DFD y su diccionario de datos

Una vez que se ha construido un DFD y su correspondiente diccionario de datos, es necesario **verificar su corrección**.

No puede haber una verificación formal de un DFD, porque lo que se modela en él no está definido formalmente en ningún otro lugar contra lo cual pueda compararse. Por lo tanto, deben usarse **procesos humanos y reglas prácticas** para validarlo.

Además de la revisión con el cliente (*walkthrough*), el analista debe buscar errores comunes. Algunos de los más frecuentes son:

- Flujos de datos **sin etiqueta**.
- Flujos de datos **faltantes**: información que un proceso necesita pero que no está disponible.
- Flujos de datos **innecesarios**: información que no se usa en el proceso.
- Falta de **consistencia** durante el refinamiento.
- **Procesos faltantes**.
- Inclusión de información de control dentro de los flujos de datos.

Quizás el error más común es el **flujo de datos sin etiqueta**. Si un analista no puede nombrar un flujo de datos, es probable que no comprenda su propósito ni su estructura. Una buena prueba para este tipo de error es revisar que todas las entradas del diccionario de datos sean precisas para cada flujo de datos.

Para comprobar si faltan flujos de datos, el analista debe preguntar, para cada proceso:

“¿Puede este proceso generar las salidas mostradas a partir de las entradas dadas?”

De manera similar, para verificar flujos de datos redundantes, se debe preguntar:

“¿Todos los flujos de datos de entrada son realmente necesarios para calcular las salidas?”

En un conjunto de DFDs nivelados es esencial mantener la **consistencia**. Esta puede perderse fácilmente si se agregan flujos nuevos durante una modificación. En tal caso, se deben realizar los cambios correspondientes en el **DFD padre** o en el **DFD hijo**.

Es decir:

- Si se añade un flujo nuevo en un DFD de nivel bajo, este debe reflejarse también en los DFDs de niveles superiores.
- Si se añade un flujo nuevo en un DFD de nivel alto, los DFDs de los procesos afectados también deben modificarse.

Los DFDs deben ser cuidadosamente revisados para asegurar que **todos los procesos del entorno físico** estén representados. Ningún flujo de datos debe en realidad transportar **información de control**. Un flujo sin estructura o composición es un **candidato potencial** a contener información de control.

El Método de Análisis Estructurado

Ahora volvamos al método de análisis estructurado. La visión básica de este enfoque es que cada sistema puede verse como una función de transformación que opera dentro de un entorno, tomando algunas entradas de ese entorno y produciendo algunas salidas para él. Y como la función global de transformación de todo el sistema puede ser demasiado compleja para comprenderla como una sola función, ésta debe dividirse en **subfunciones** que, en conjunto, formen la función global.

Las subfunciones pueden dividirse aún más y el proceso repetirse hasta llegar a un punto en el que cada función pueda comprenderse con facilidad. El enfoque básico utilizado para descubrir las funciones que se realizan en el sistema (o las que forman parte de la función global del sistema) es rastrear los datos a medida que fluyen a través de éste: **desde la entrada hasta la salida**.

Se cree que, en cualquier sistema complejo, la transformación de los datos desde la entrada hasta la salida no ocurre en un solo paso; más bien, los datos se transforman en una serie de pasos, comenzando desde la entrada y culminando en la salida deseada. Al comprender los “**estados**” por los que pasan los datos en esa serie de transformaciones, pueden identificarse las funciones del sistema; cada transformación de los datos en esa serie es realizada por una función de transformación.

Por lo tanto, al rastrear los datos a través del sistema, pueden identificarse las diversas funciones que éste ejecuta. Como este enfoque puede modelarse fácilmente mediante **diagramas de flujo de datos (DFD)**, éstos se utilizan ampliamente en este método.

Etapas del método

1. Estudiar el entorno físico:

- En esta etapa se dibuja un **DFD del sistema actual no automatizado** (o parcialmente automatizado), mostrando los flujos de datos de entrada y salida, cómo circulan por el sistema y qué procesos actúan sobre ellos.
- Este DFD puede contener nombres específicos de flujos de datos y procesos usados en el entorno físico, como nombres de departamentos, personas, procedimientos locales o archivos organizacionales.
- Para elaborarlo, el analista interactúa con los usuarios para determinar el proceso general desde el punto de vista de los datos.

- Esta etapa se considera terminada cuando el usuario acepta que el DFD refleja fielmente el funcionamiento del sistema actual.
- Puede comenzar con un **diagrama de contexto**, donde el sistema entero se trata como un solo proceso y se identifican todas sus entradas, salidas, fuentes y destinos.

2. Obtener el DFD lógico del sistema actual:

- El objetivo es llegar a un DFD donde cada flujo de datos y cada proceso sean entidades **lógicas**, no físicas.
- El DFD físico sirve solo como punto de partida.
- En esta fase se reemplazan los nombres físicos por equivalentes lógicos.
 - Ejemplo: “archivo 12.3.2” → “archivo de salarios de empleados”.
 - Ejemplo: un proceso llamado “A la oficina de Juan” → “emitir cheques”.
- Además, se eliminan las burbujas que no transforman los datos.
- La fase concluye cuando el DFD lógico ha sido verificado por el usuario.

3. Diseñar el DFD lógico del nuevo sistema:

- Una vez comprendido el sistema actual, se pasa a modelar el **nuevo sistema** con los cambios incorporados.
- El analista trabaja en modo lógico, especificando **qué debe hacerse, no cómo se hará**.
- No se distingue aún entre procesos automatizados y manuales.
- Este paso requiere determinar cuáles partes del DFD actual cambiarán (definir los **límites de cambio**).
- El nuevo DFD reemplazará solo esa parte del existente, manteniendo las mismas entradas y salidas en los límites.

4. Definir la frontera hombre-máquina:

- Aquí se especifica qué procesos se automatizarán y cuáles permanecerán manuales.
- Incluso los procesos manuales pueden diferir de los originales en el nuevo sistema.
- Puede haber varias opciones según el grado de automatización posible.
- El analista debe explorar y presentar estas alternativas.

5. Evaluar opciones y presentar especificaciones:

- Una vez definidos los posibles escenarios de automatización, deben evaluarse y documentarse para presentar la mejor propuesta.

Recomendaciones de construcción

- El análisis estructurado sugiere un **enfoque de arriba hacia abajo (top-down)** para dibujar los DFD.
- Se comienza con un **diagrama de contexto**, que lista las entradas y salidas principales del sistema.
- Luego este diagrama se descompone en partes más detalladas, formando un **conjunto jerárquico de DFDs**.
- Durante la refinación debe mantenerse la **consistencia**, es decir, preservar entradas y salidas netas.

Fortalezas y limitaciones

- El análisis estructurado ofrece métodos claros para **organizar y representar información** sobre sistemas.
- También proporciona **guías para verificar la precisión** de la información obtenida.
- Es especialmente útil en las **primeras dos etapas**, cuando se construye el DFD del sistema actual.
- Sin embargo, para la tercera etapa (analizar el sistema objetivo y construir su DFD o diccionario de datos), la técnica no ofrece tanta ayuda.
- Aun así, el conocimiento adquirido del sistema actual sirve de base para diseñar el nuevo sistema.

3.2.3 Modelado Orientado a Objetos

En el modelado orientado a objetos, un sistema se ve como un conjunto de **objetos**. Los objetos interactúan entre sí a través de los servicios que proveen. Algunos objetos también interactúan con los usuarios a través de sus servicios de tal manera que los usuarios obtienen los servicios deseados. Por lo tanto, el objetivo del modelado es identificar los objetos (en realidad, las clases de objetos) que existen en el dominio del problema, definir las clases especificando qué información de estado encapsulan y qué servicios proveen, e identificar las relaciones que existen entre objetos de diferentes clases, de tal manera que el modelo en su conjunto apoye los servicios que el usuario desea.

El modelado orientado a objetos y los sistemas han recibido mucha atención en el pasado reciente. La razón básica para esto es la creencia de que los sistemas orientados a objetos van a ser más fáciles de construir y mantener. También se cree que la transición desde el análisis orientado a objetos al diseño (e implementación) orientado a objetos será fácil, y que el análisis orientado a objetos es más inmune al cambio porque los objetos son más estables que las funciones. Es decir, en un dominio de problema, es probable que los objetos permanezcan iguales incluso si la naturaleza exacta del problema cambia, mientras que esto no ocurre con el modelado orientado a funciones.

Algunos enfoques para el modelado y diseño orientado a objetos fueron propuestos tempranamente. Los objetivos de muchas de estas técnicas, respecto a qué producir, son bastante similares, y sus enfoques y notaciones también lo son. Aquí, describimos brevemente el enfoque propuesto en [36];

la notación que usamos es **UML** ([24, 64] o www.uml.org), que ahora es la notación estándar de facto para el modelado OO. Discutiremos UML más a fondo en el Capítulo 7 cuando hablemos de diseño OO; aquí discutimos algunos conceptos necesarios para hablar del análisis.

Conceptos Básicos y Notación

Al comprender o modelar un sistema utilizando una técnica de modelado orientado a objetos, el sistema se ve como compuesto de objetos. Cada objeto tiene ciertos **atributos**, que en conjunto definen al objeto. La separación de un objeto de sus atributos es un método natural que usamos para entender sistemas (un hombre está separado de sus atributos como altura, peso, etc.). En sistemas orientados a objetos, los atributos mantienen el estado (o definen el estado) de un objeto. Un atributo es un valor puro de datos (como entero, cadena, etc.), no un objeto.

Objetos de tipo similar se agrupan para formar una **clase de objetos** (o simplemente clase). Una clase es esencialmente una definición de tipo, que define el espacio de estado de los objetos de ese tipo y las operaciones (y su semántica) que pueden aplicarse a los objetos de ese tipo. La formación de clases también es una técnica general utilizada por los humanos para entender sistemas y diferenciar entre clases (por ejemplo, un manzano es una instancia de la clase de árboles, y la clase de árboles es diferente de la clase de aves).

Un objeto también provee algunos **servicios u operaciones**. Estos servicios son el único medio por el cual el estado del objeto puede ser modificado o visto desde el exterior. Para operar un servicio, se envía un mensaje al objeto para ese servicio. En general, estos servicios se definen para una clase y son provistos para cada objeto de esa clase. Encapsular servicios y atributos juntos en un objeto es una de las principales características que distingue un enfoque de modelado orientado a objetos de los enfoques de modelado de datos, como los diagramas ER.

Los **diagramas de clases** representan gráficamente la estructura del problema utilizando una notación precisa. En un diagrama de clases, una clase se representa como un rectángulo en formato vertical dividido en tres partes. La parte superior contiene el nombre de la clase. La parte del medio lista los atributos que poseen los objetos de esta clase. Y la tercera parte lista los servicios provistos por los objetos de esta clase.

Para modelar las **relaciones entre clases**, se usan algunas estructuras.

- La estructura de **generalización-especialización** puede ser usada por una clase para heredar todos o algunos atributos y servicios de una clase general y agregar más atributos y servicios. Esta estructura se modela en el modelado orientado a objetos mediante **herencia**. Usando una clase general e incorporando algunos de sus atributos y servicios y agregando más, se puede crear una clase que es una versión especializada de la clase general. Y muchas clases especializadas pueden crearse a partir de una clase general, dándonos jerarquías de clases.
- La estructura de **agregación** modela la relación todo-parte. Un objeto puede estar compuesto de muchos objetos; esto se modela mediante la estructura de agregación.

La representación de estas relaciones en un diagrama de clases se muestra en la Figura 3.8.

Además de estas, las **instancias** de una clase pueden estar relacionadas con objetos de alguna otra clase. Por ejemplo, un objeto de la clase **Employer** puede estar relacionado con muchos objetos de la clase **Employee**. Esta relación entre objetos también debe ser capturada si se quiere modelar

adecuadamente un sistema. Esto se captura a través de **asociaciones**. Una asociación se muestra en el diagrama de clases mediante una línea entre las dos clases. La **multiplicidad** de una asociación especifica cuántas instancias de una clase pueden relacionarse con instancias de la otra clase a través de esa asociación. Una asociación entre dos clases puede ser de **uno a uno** (es decir, una instancia de una clase está relacionada exactamente con una instancia de la otra clase), de **uno a muchos**, o de algunos otros casos especiales. La multiplicidad se especifica colocando una estrella (*) en la línea adyacente a la clase, representando que cero o más instancias de la clase pueden estar relacionadas con una instancia de la otra clase.

3.2.4 Prototipado

El **prototipado** adopta un enfoque diferente para el análisis de problemas en comparación con los enfoques basados en modelos. En el prototipado, se construye un **sistema parcial**, que luego es utilizado por el cliente, los usuarios y los desarrolladores para obtener una mejor comprensión del problema y de las necesidades. Por lo tanto, la experiencia real con un prototipo que implementa parte del sistema de software eventual se usa para analizar el problema y entender los requisitos del sistema final.

Un **prototipo de software** se puede definir como una implementación parcial de un sistema cuyo propósito es **aprender algo sobre el problema que se está resolviendo o sobre el enfoque de solución** [46]. Como indica esta definición, el prototipado también puede usarse para evaluar o verificar una alternativa de diseño (a este tipo se le llama **prototipo de diseño** [46]). Aquí nos enfocamos en el prototipado utilizado principalmente para comprender los requisitos.

La **justificación** de usar prototipado para el entendimiento y análisis de problemas es que los clientes y usuarios, a menudo, encuentran difícil visualizar cómo funcionará el sistema de software final en su entorno solo leyendo un documento de especificación. Imaginar la operación del software que aún no se ha construido, y si cumplirá con los objetivos finales, únicamente leyendo y discutiendo requisitos en papel, es muy difícil. Esto es particularmente cierto si el sistema es completamente nuevo y muchos usuarios o clientes no tienen una idea clara de sus necesidades.

La idea del prototipado es que **clientes y usuarios pueden evaluar mucho mejor sus necesidades si pueden ver un sistema en funcionamiento**, aunque este sea solo parcial. El prototipado enfatiza que la **experiencia práctica real es la mejor ayuda para comprender las necesidades**. Al experimentar con un sistema, la gente puede decir:

- “No quiero esta característica”,
- “Me gustaría que tuviera esta otra”, o
- “Esto es excelente”.

Existen dos enfoques de prototipado: **desechable y evolutivo** [44, 46]. En el **enfoque desechable**, el prototipo se construye con la idea de que será descartado después de completar el análisis, y el sistema final se construirá desde cero. En el **enfoque evolutivo**, el prototipo se construye con la idea de que eventualmente se convertirá en el sistema final. Desde el punto de vista del análisis de problemas y su comprensión, los **prototipos desechantes** son los más adecuados. En lo que sigue, nos enfocaremos en los prototipos desechantes.

La primera pregunta que debe responderse es **si conviene o no hacer un prototipo**. En otras palabras, es importante entender claramente cuándo debe hacerse. Los requisitos de un sistema pueden dividirse en tres conjuntos [46]:

- Los bien entendidos,
- Los poco entendidos, y
- Los desconocidos.

En un **prototipo desechable**, los requisitos poco entendidos son los que deben incorporarse. Con base en la experiencia obtenida con el prototipo, estos requisitos poco entendidos se transforman en **bien entendidos**, como se muestra en la Figura 3.12.

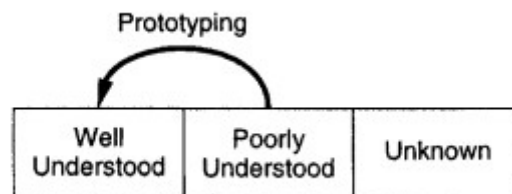


Figura 3.12: Prototipo desechable.

Es posible dividir aún más el conjunto de requisitos poco entendidos en dos subconjuntos:

- Los **críticos para el diseño**, y
- Los **no críticos para el diseño** [46].

Los requisitos que pueden incorporarse fácilmente al sistema más adelante se consideran **no críticos**. Si es posible determinar cuáles requisitos poco entendidos son críticos y cuáles no, entonces el prototipo desechable debe enfocarse principalmente en los **críticos**.

En general, podemos decir que si el conjunto de requisitos poco entendidos es sustancial (en particular, la parte de los críticos), entonces conviene construir un prototipo desechable.

El **proceso de desarrollo** usando un prototipo desechable ya fue descrito en el Capítulo 2. La actividad comienza con un **ERS (Especificación de Requerimientos de Software)** para el prototipo. Sin embargo, desarrollar este ERS requiere identificar qué funciones deben incluirse en el prototipo. Esta decisión depende de la aplicación. En general, como se mencionó antes, los requisitos que tienden a ser **inciertos, vagos o cambiantes** son los que deben implementarse en el prototipo.

Ejemplos comunes de estos son:

- La **interfaz de usuario**,
- Las **nuevas características** a agregar (más allá de automatizar lo que ya se hace), y
- Las características que podrían ser **inviables**.

Dependiendo de qué aspectos del sistema se incluyan en el prototipo, este puede clasificarse como **vertical** o **horizontal** [110].

- En el **prototipado horizontal**, el sistema se ve como una serie de capas y se selecciona una de ellas para el prototipo (ejemplo: la **interfaz de usuario**, donde casi toda la interfaz se incluye en el prototipo).
- En el **prototipado vertical**, se construye por completo una parte específica del sistema que no se entiende bien. Este enfoque se usa para validar alguna **funcionalidad o capacidad** del sistema.

El **desarrollo de un prototipo desechable** es fundamentalmente distinto del desarrollo de software de calidad final para producción. El foco principal durante el prototipado es **mantener bajos los costos y minimizar el tiempo de producción**.

Por esta razón:

- Muchas de las tareas de **documentación, control de calidad y gestión** se reducen al mínimo.
- Las **preocupaciones de eficiencia** pasan a un segundo plano.
- Con frecuencia se usan **lenguajes interpretados de alto nivel** para construir el prototipo.

Debido a esto, la **tentación de convertir el prototipo en el sistema final debe evitarse**.

La experiencia se obtiene al poner el prototipo en manos del cliente y los usuarios reales. Durante esta actividad es necesaria una **interacción constante** para comprender sus reacciones. Se pueden usar **cuestionarios e entrevistas** para recopilar retroalimentación.

El **ERS final** se desarrolla de manera similar a cualquier ERS, pero con la gran ventaja de que los clientes y usuarios podrán **responder mejor a las preguntas y explicar sus necesidades**, gracias a la experiencia con el prototipo.

Para que el prototipado en análisis de requisitos sea viable, su **costo debe mantenerse bajo**. Por lo tanto, solo se incluyen en el prototipo aquellas funciones cuyo **retorno en términos de experiencia del usuario sea valioso**.

En general, no se incluyen características como:

- Manejo de excepciones,
- Recuperación ante fallos,
- Conformidad con estándares y formatos.

Como el prototipo será desechado, la cantidad de documentos producidos se mantiene al mínimo: no se necesitan documentos de diseño, plan de pruebas ni especificaciones de casos de prueba. Otra forma importante de reducir costos es disminuir el **nivel de pruebas**, ya que las pruebas consumen gran parte del presupuesto en el desarrollo normal de software.

Gracias a estas medidas, es posible mantener el costo del prototipo en **menos de unos pocos puntos porcentuales** del costo total del desarrollo.

El costo de construir y ejecutar un prototipo puede estar alrededor del **10% del costo total del desarrollo** [72]. Sin embargo, esto **no significa** que el costo total aumente en esa proporción. La razón principal es que los **beneficios del prototipado** —en forma de reducción de errores de

requisitos y disminución en la cantidad de cambios posteriores— suelen ser muy significativos, reduciendo así el costo global del desarrollo.

3.3 Especificación de Requisitos

El resultado final es el documento de especificación de requisitos de software (SRS).

En problemas pequeños, o en aquellos que pueden comprenderse fácilmente, la actividad de especificación puede realizarse una vez que el análisis está completamente terminado. Sin embargo, lo más común es que el análisis del problema y la especificación se hagan de manera simultánea. Normalmente, un analista estudia una parte del problema y luego escribe los requisitos correspondientes a esa parte. En la práctica, las actividades de análisis y de especificación se superponen, y se avanza de una a otra constantemente, como se muestra en la Figura 3.1. No obstante, dado que toda la información de la especificación proviene del análisis, conceptualmente podemos ver la especificación como una etapa posterior al análisis.

La primera pregunta que surge es: si durante el análisis se realiza un modelado formal, ¿por qué no se consideran los resultados de ese modelado —las estructuras creadas (por ejemplo, DFD y DD, diagramas de objetos)— como la SRS?

La razón principal es que el modelado generalmente se centra en la estructura del problema, no en su comportamiento externo. Por eso, elementos como la **interfaz de usuario** rara vez se modelan, aunque suelen ser una parte fundamental de la SRS. De manera similar, para simplificar el modelado, a menudo no se consideran en detalle “situaciones menores” como los errores en la salida, mientras que en la SRS sí es necesario especificar cómo debe comportarse el sistema en esos casos. Además, aspectos como restricciones de rendimiento, limitaciones de diseño, cumplimiento de estándares, recuperación ante fallos, etc., no se incluyen en los modelos, pero deben estar claramente definidos en la SRS, ya que el diseñador necesita conocerlos para poder diseñar el sistema adecuadamente.

Por lo tanto, queda claro que los modelos no pueden reemplazar una SRS.

Por estas razones, la transición del análisis a la especificación no debe pensarse como un paso directo, incluso si se usaron modelos formales en el análisis. No se trata simplemente de tomar las estructuras del modelado y escribirlas de forma más formal. Una buena SRS debe cubrir muchos aspectos que el modelado no resuelve satisfactoriamente. Además, a veces las estructuras producidas en el análisis no sirven para traducirse directamente en especificaciones de comportamiento externo (que es justamente lo que se define en una SRS). Por ejemplo, un diagrama de objetos hecho durante un análisis orientado a objetos tiene una utilidad limitada a la hora de especificar el comportamiento externo del sistema deseado.

En esencia, lo que pasa del análisis de requisitos a la especificación es el **conocimiento adquirido sobre el sistema**. El modelado es una herramienta para ayudar a obtener un conocimiento profundo y completo del sistema propuesto. La SRS se redacta en base a ese conocimiento. Como transformar ese conocimiento en un documento estructurado no es algo simple, la especificación en sí misma es una tarea importante y relativamente independiente.

Una consecuencia de esto es que es menos crítico tener un modelo “completo” que una especificación completa. El objetivo principal del análisis es comprender el problema, mientras que el objetivo principal de la fase de requisitos es producir la SRS. Por lo tanto, los modelos de análisis exhaustivos y detallados no son esenciales. De hecho, es posible crear una SRS sin usar técnicas de modelado formal. El objetivo central de los modelos es apoyar la representación del conocimiento y la división del problema; los modelos no son un fin en sí mismos.

Con esto en mente, comencemos la discusión sobre la especificación de requisitos. Empezaremos analizando las características deseables de una SRS.

3.3.1 Características de un SRS

Para cumplir correctamente con los objetivos básicos, un **SRS** (Especificación de Requisitos de Software) debería tener ciertas propiedades y contener distintos tipos de requisitos. En esta sección vamos a hablar sobre algunas de las características deseables de un SRS y de los componentes que lo conforman.

Un buen SRS debe ser:

1. **Correcto**
2. **Completo**
3. **No ambiguo**
4. **Verificable**
5. **Consistente**
6. **Clasificado por importancia y/o estabilidad**
7. **Modificable**
8. **Trazable**

Un **SRS es correcto** si cada requisito incluido en él representa algo que realmente se necesita en el sistema final.

Un **SRS es completo** si especifica todo lo que el software debe hacer y también las respuestas del software para todas las clases de datos de entrada.

La **corrección** y la **completitud** van de la mano: la corrección asegura que lo que está especificado esté bien hecho, mientras que la completitud asegura que todo esté especificado.

La corrección es más fácil de comprobar que la completitud, ya que implica revisar cada requisito y asegurarse de que refleja lo que el usuario necesita. La completitud, en cambio, es la más difícil de garantizar, porque implica detectar lo que **falta**, y comprobar una ausencia es mucho más complicado que verificar lo que ya está presente.

Un **SRS es no ambiguo** si, y solo si, cada requisito tiene una sola interpretación posible. Como los requisitos suelen escribirse en lenguaje natural (que es inherentemente ambiguo), si se usan frases en lenguaje común el redactor del SRS debe tener especial cuidado en evitar ambigüedades. Una forma de reducirlas es usar un lenguaje formal de especificación de requisitos. Sin embargo, su

desventaja es que escribir un SRS en un lenguaje formal requiere mucho esfuerzo, tiene un alto costo y además es difícil de leer y entender (especialmente para clientes y usuarios).

Un **SRS es verificable** si, y solo si, cada requisito puede ser comprobado.

Un requisito es verificable si existe algún proceso, razonablemente económico, que pueda comprobar si el software final cumple con ese requisito. Esto implica que los requisitos deben ser lo menos subjetivos posible, porque lo subjetivo es difícil de verificar. La ausencia de ambigüedad es esencial para que algo sea verificable.

Dado que la verificación de requisitos suele hacerse mediante revisiones, también implica que un SRS debe ser comprensible, al menos para el desarrollador, el cliente y los usuarios. La **entendibilidad** es clave, porque uno de los objetivos de la fase de requisitos es producir un documento en el que cliente, usuarios y desarrolladores estén de acuerdo.

Un **SRS es consistente** si no existen requisitos que entren en conflicto entre sí. Las inconsistencias pueden aparecer por la terminología (cuando diferentes requisitos usan distintos términos para referirse a lo mismo) o por conflictos lógicos o temporales.

Esto ocurre si el SRS contiene dos o más requisitos que no pueden cumplirse juntos en ningún sistema de software.

Por ejemplo: si un requisito dice que un evento **e** debe ocurrir antes que un evento **f**, pero otro requisito (directo o indirecto) dice que **f** debe ocurrir antes que **e**, entonces hay una inconsistencia. Estas inconsistencias pueden reflejar problemas serios en la especificación.

En general, no todos los requisitos del software tienen la misma importancia. Algunos son críticos, otros son importantes pero no críticos, y otros simplemente deseables. Del mismo modo, algunos requisitos son “centrales” y poco probables de cambiar con el tiempo, mientras que otros dependen más de las circunstancias.

Un **SRS está clasificado por importancia y/o estabilidad** si para cada requisito se indica qué tan importante es y qué tan probable es que cambie en el futuro. La estabilidad de un requisito se puede expresar en términos del volumen esperado de cambios.

Escribir un SRS es un **proceso iterativo**. Incluso cuando los requisitos de un sistema se especifican, más tarde se modifican a medida que cambian las necesidades del cliente. Por eso, un **SRS debe ser fácil de modificar**.

Un SRS es modificable si su estructura y estilo permiten hacer cambios fácilmente, sin perder completitud ni consistencia. La redundancia es un gran obstáculo para la modificabilidad, ya que puede generar errores.

Por ejemplo: si un requisito está escrito en dos lugares distintos y luego hay que cambiarlo, pero solo se modifica en uno de los dos, el SRS resultante queda inconsistente.

Un **SRS es trazable** si se conoce claramente el origen de cada requisito y si facilita la referencia de cada requisito durante el desarrollo posterior [91].

La **trazabilidad hacia adelante** significa que cada requisito puede vincularse con elementos de diseño y código.

La **trazabilidad hacia atrás** implica que es posible rastrear elementos de diseño y código hacia los requisitos que soportan. La trazabilidad facilita la verificación y la validación.

De todas estas características, la **completitud** es quizás la más importante (y la más difícil de garantizar).

Uno de los problemas más comunes en la especificación de requisitos ocurre cuando algunos

requisitos del cliente no se incluyen. Esto obliga a hacer agregados y modificaciones más adelante en el ciclo de desarrollo, y esos cambios suelen ser costosos.

La incompletitud también es una fuente importante de conflictos entre cliente y proveedor.

Por eso, la importancia de tener requisitos completos no puede enfatizarse lo suficiente.

3.3.2 Componentes de un SRS

La completitud de las especificaciones es difícil de lograr y aún más difícil de verificar.

Tener guías sobre qué cosas debería especificar un SRS ayuda a detallar los requisitos de manera completa.

Aquí describimos algunas de las propiedades del sistema que un SRS debería especificar.

Los aspectos básicos que un SRS debe cubrir son:

- **Funcionalidad**
- **Desempeño (Performance)**
- **Restricciones de diseño impuestas en la implementación**
- **Interfaces externas**

Conceptualmente, cualquier SRS debería tener estos componentes.

Si se sigue el enfoque tradicional de análisis de requisitos, entonces el SRS incluso podría tener secciones que correspondan a cada uno de ellos.

Sin embargo, los requisitos funcionales pueden especificarse de manera indirecta, ya sea indicando los servicios sobre los objetos o describiendo los casos de uso.

Requisitos Funcionales

Los **requisitos funcionales** especifican qué salidas (outputs) deben producirse a partir de las entradas (inputs) dadas.

Describen la relación entre la entrada y la salida del sistema.

Para cada requisito funcional, se debe incluir una descripción detallada de:

- Todos los datos de entrada y su origen.
- Las unidades de medida.
- El rango de valores válidos para las entradas.

También deben especificarse todas las operaciones que se realizan sobre los datos de entrada para obtener la salida.

Esto incluye:

- Las verificaciones de validez en los datos de entrada y salida.
- Los parámetros afectados por la operación.
- Las ecuaciones u otras operaciones lógicas que deben usarse para transformar las entradas en las salidas correspondientes.

Por ejemplo, si existe una fórmula para calcular una salida, debe estar especificada.

Es importante **no especificar algoritmos de implementación** que no formen parte del sistema en sí, aunque puedan ser necesarios para implementarlo.

Las decisiones de este tipo deben quedar a cargo del diseñador.

Un aspecto crucial de la especificación es el **comportamiento del sistema en situaciones anormales**, como:

- Entradas inválidas (que pueden presentarse de muchas formas).
- Errores durante el cómputo.

El requisito funcional debe indicar claramente qué debe hacer el sistema si ocurren estas situaciones.

En particular, debe especificar:

- El comportamiento frente a entradas inválidas.
- El comportamiento frente a salidas inválidas.
- El comportamiento cuando la entrada es válida, pero la operación normal no puede realizarse.

Ejemplo: en un sistema de reservas de vuelos, aunque un pasajero válido intente reservar, la operación no puede completarse si el avión ya está totalmente ocupado.

En resumen:

El comportamiento del sistema debe especificarse para **todas las entradas previstas** y para **todos los estados posibles del sistema**.

Estas condiciones especiales suelen pasarse por alto, lo que termina generando sistemas poco robustos.

Requisitos de Rendimiento

Esta parte de un SRS especifica las restricciones de rendimiento del sistema de software. Todos los requisitos relacionados con las características de rendimiento del sistema deben estar claramente definidos. Existen dos tipos de requisitos de rendimiento: estáticos y dinámicos.

Los **requisitos estáticos** son aquellos que no imponen restricciones sobre las características de ejecución del sistema. Entre ellos se incluyen, por ejemplo, el número de terminales que el sistema debe soportar, el número de usuarios simultáneos que puede manejar y el número de archivos que debe procesar, así como sus tamaños. Estos también se conocen como requisitos de capacidad del sistema.

Por otro lado, los **requisitos dinámicos** especifican restricciones sobre el comportamiento en ejecución del sistema. Normalmente incluyen el tiempo de respuesta y el rendimiento o throughput. El tiempo de respuesta se refiere al tiempo esperado para completar una operación bajo ciertas circunstancias, mientras que el throughput indica el número esperado de operaciones que se pueden realizar en una unidad de tiempo. Por ejemplo, un SRS puede especificar cuántas transacciones deben procesarse por unidad de tiempo o cuál debe ser el tiempo de respuesta de un comando específico. Además, se deben definir los rangos aceptables para estos parámetros de rendimiento, considerando tanto condiciones de carga normales como máximas.

Es fundamental que todos estos requisitos estén expresados en términos medibles. Requisitos vagos como “el tiempo de respuesta debe ser bueno” o “el sistema debe procesar todas las transacciones rápidamente” no son deseables, ya que son imprecisos y no verificables. En cambio, se deben usar declaraciones claras y cuantificables, por ejemplo: “El tiempo de respuesta del comando X debe ser menor a un segundo el 90% de las veces” o “Una transacción debe procesarse en menos de un segundo el 98% de las veces”. Esto asegura que los requisitos de rendimiento sean objetivos y verificables.

3.3.3 Lenguaje de Especificación

La especificación de requisitos requiere el uso de algún **lenguaje de especificación**. Este lenguaje debe soportar las cualidades deseadas del SRS, como modificabilidad, comprensibilidad, ausencia de ambigüedad, entre otras. Además, debe ser fácil de aprender y usar. Como es de esperar, muchas de estas características pueden entrar en conflicto al momento de elegir un lenguaje de especificación. Por ejemplo, para evitar ambigüedad, lo ideal sería usar un lenguaje formal. Pero para facilitar la comprensión, un lenguaje natural podría ser preferible.

Aunque existen notaciones formales para especificar propiedades específicas del sistema, hoy en día los **lenguajes naturales** son los más utilizados para especificar requisitos. Si se usan lenguajes formales, normalmente se emplean para especificar propiedades particulares o partes específicas del sistema, y estas especificaciones formales generalmente se incluyen dentro del SRS general, que está escrito en lenguaje natural. En otras palabras, el SRS completo suele estar en lenguaje natural y, cuando es factible y conveniente, algunas partes pueden usar notaciones formales.

La ventaja principal de usar un lenguaje natural es que tanto el cliente como el proveedor lo entienden. Sin embargo, por la naturaleza misma del lenguaje natural, este es impreciso y puede ser ambiguo. Para reducir estos inconvenientes, generalmente se usa el **lenguaje natural de manera estructurada**. Por ejemplo, en inglés estructurado, los requisitos se dividen en secciones y párrafos, y cada párrafo se subdivide en subpárrafos. Muchas organizaciones también establecen reglas estrictas para el uso de palabras como "shall", "perhaps" o "should", y restringen ciertas frases comunes para mejorar la precisión y reducir la ambigüedad y la verbosidad. Una regla general al usar lenguaje natural es ser preciso, factual y conciso, organizando los requisitos jerárquicamente siempre que sea posible y asignando un número único a cada requisito.

En un SRS, como se discutió, algunas partes pueden especificarse mejor usando notaciones formales. Por ejemplo, para definir los formatos de entradas o salidas, las expresiones regulares son muy útiles. De manera similar, para sistemas como protocolos de comunicación, se pueden usar autómatas finitos. Las **tablas de decisión** son útiles para especificar formalmente el comportamiento del sistema ante diferentes combinaciones de entradas o configuraciones. Además, algunos aspectos del sistema pueden especificarse o explicarse mediante los modelos que se hayan construido durante el análisis del problema. A veces, estos modelos se incluyen como documentos de apoyo para clarificar mejor los requisitos y su motivación.

3.4 Especificación Funcional con Casos de Uso

Los **requisitos funcionales** a menudo constituyen el núcleo de un documento de requisitos.

El enfoque tradicional para especificar la funcionalidad consiste en describir cada función que el sistema debe proporcionar.

Los **casos de uso** especifican la funcionalidad de un sistema describiendo su **comportamiento**, capturado como las **interacciones entre los usuarios y el sistema**. Los casos de uso pueden utilizarse para describir los **procesos de negocio** de una organización que despliega el software, o simplemente para describir el comportamiento del sistema de software en sí. En este caso, nos centraremos en describir el comportamiento de los sistemas de software que se van a construir.

Aunque los casos de uso se utilizan principalmente para especificar comportamiento, también pueden ser efectivos durante la fase de análisis. Más adelante, al discutir cómo desarrollar casos de uso, veremos cómo pueden ayudar también a **obtener los requisitos**.

Los casos de uso ganaron relevancia después de ser utilizados como parte del enfoque de **modelado orientado a objetos** propuesto por Jacobson. Debido a esta conexión con el enfoque orientado a objetos, a veces se consideran parte de este enfoque para el desarrollo de software. Sin embargo, los casos de uso son un **método general** para describir la interacción de un sistema, incluso de sistemas no tecnológicos.

La discusión sobre casos de uso aquí se basa en los conceptos y procesos presentados en las referencias mencionadas.

3.5 Validación

El desarrollo de software comienza con un **documento de requisitos**, que también se utiliza posteriormente para determinar si el software entregado es aceptable o no. Por lo tanto, es importante que la especificación de requisitos no contenga errores y refleje correctamente las necesidades del cliente. Además, como hemos visto, cuanto más tiempo permanezca un error sin ser detectado, mayor será el costo de corregirlo. Por esta razón, es muy deseable detectar los errores en los requisitos **antes** de que comience el diseño y desarrollo del software.

Debido a la naturaleza de la fase de especificación de requisitos, existe mucho margen para malentendidos y errores, y es bastante posible que la especificación de requisitos no represente con exactitud las necesidades del cliente. El objetivo principal de la **validación de requisitos** es garantizar que el SRS refleje los requisitos reales de manera precisa y clara. Un objetivo relacionado es comprobar que el documento SRS en sí sea de **“buena calidad”**.

Antes de discutir la validación, conviene analizar los tipos de errores que típicamente ocurren en un SRS. Existen muchos tipos de errores posibles, pero los más comunes se pueden clasificar en cuatro categorías: **omisión, inconsistencia, hecho incorrecto y ambigüedad**.

- **Omisión:** Es un error común donde algún requisito del usuario simplemente no se incluye en el SRS. El requisito omitido puede estar relacionado con el comportamiento del sistema, su rendimiento, restricciones u otros factores. La omisión afecta directamente la **completitud externa** del SRS.

- **Inconsistencia:** Puede deberse a contradicciones dentro de los propios requisitos o a la incompatibilidad entre los requisitos declarados y los reales del cliente o del entorno en que operará el sistema.
- **Hecho incorrecto:** Ocurre cuando algún dato registrado en el SRS no es correcto.
- **Ambigüedad:** Ocurre cuando algún requisito tiene múltiples interpretaciones posibles, es decir, su interpretación no es única.

Algunos proyectos han recopilado datos sobre los errores en los requisitos. Por ejemplo, se han detectado más de 250 errores en aplicaciones de procesamiento de datos, y en el proyecto A-7 (software de control de vuelo en tiempo real) se detectaron alrededor de 80 errores, de los cuales aproximadamente un 23% eran de naturaleza administrativa. A pesar de que la distribución de errores varía según la aplicación y el método de detección, los principales problemas, además de los errores administrativos, son **omisiones, hechos incorrectos, inconsistencias y ambigüedades**. Esto indica que la validación debe centrarse en descubrir estos tipos de errores además de mejorar la calidad del SRS.

Como los requisitos generalmente son documentos textuales que no se pueden ejecutar, las **inspecciones** son muy adecuadas para la validación de requisitos. Las inspecciones del SRS, frecuentemente llamadas **revisiones de requisitos**, son el método más común de validación. Dado que la especificación de requisitos formaliza algo que originalmente existía de manera informal en la mente de las personas, la validación debe involucrar tanto a los clientes como a los usuarios. Por ello, el equipo de revisión suele incluir representantes del cliente y de los usuarios.

La **revisión de requisitos** es un proceso en el que un grupo de personas busca errores y señala otros aspectos importantes en el SRS. El grupo de revisión debería incluir: el autor del documento de requisitos, alguien que conozca las necesidades del cliente, un miembro del equipo de diseño y las personas responsables de mantener el documento de requisitos. También es recomendable incluir a alguien que no esté directamente involucrado en el desarrollo, como un ingeniero de calidad de software.

Aunque el objetivo principal del proceso de revisión es detectar errores, también se considera la calidad del SRS, como su **capacidad de prueba y legibilidad**. Durante la revisión, los revisores deben identificar requisitos demasiado subjetivos o difíciles de probar.

Se suelen usar **listas de verificación** para enfocar el esfuerzo de revisión y asegurar que no se pase por alto ninguna fuente importante de errores. Algunos ejemplos de preguntas de la lista de verificación son:

- ¿Se han definido todos los recursos de hardware?
- ¿Se han especificado los tiempos de respuesta de las funciones?
- ¿Se han definido todas las interfaces de hardware, software externo y datos?
- ¿Se han especificado todas las funciones requeridas por el cliente?
- ¿Cada requisito es verificable?
- ¿Se ha definido el estado inicial del sistema?
- ¿Se han especificado las respuestas a condiciones excepcionales?

- ¿Contiene el requisito restricciones controlables por el diseñador?
- ¿Se han especificado posibles modificaciones futuras?

Las revisiones de requisitos son probablemente el medio más efectivo para detectar errores. En el proyecto A-7, aproximadamente el 33% de los errores se detectaron mediante revisiones, y alrededor del 45% durante la fase de diseño, cuando el documento de requisitos se utilizó como referencia. Esto sugiere que revisar los requisitos no solo detecta una fracción sustancial de errores, sino que la mayoría de los errores restantes se detectan poco después en la fase de diseño.

Aunque las revisiones siguen siendo el método más común y viable, existen otras posibilidades si se utilizan herramientas especiales de modelado y análisis. Por ejemplo, si los requisitos están escritos en un lenguaje formal o diseñado para procesamiento por máquina, es posible usar herramientas para verificar algunas propiedades, como **consistencia interna** y **completitud**, aunque no pueden comprobar directamente la **completitud externa** (por ejemplo, no pueden detectar requisitos completamente omitidos). Por esta razón, incluso cuando se usan herramientas o notaciones formales, las revisiones siguen siendo necesarias.

3.6 Métricas

Como mencionamos anteriormente, el **propósito básico de las métricas** en cualquier momento de un proyecto de desarrollo es **proporcionar información cuantitativa** al proceso de gestión, de manera que esta información pueda ser utilizada para **controlar eficazmente el proceso de desarrollo**.

A menos que una métrica sea útil de alguna forma para **monitorear o controlar el costo, el cronograma o la calidad del proyecto**, tiene poco valor para el proyecto.

Existen muy pocas métricas definidas para los **requisitos**, y se ha realizado poco trabajo para estudiar la relación entre los valores de las métricas y las propiedades del proyecto que son de interés. Esto refleja más el **estado actual del arte de las métricas de software** que la utilidad de contar con dichas métricas.

4 Arquitectura de Software

Un **sistema** es una entidad que provee algún tipo de **comportamiento a su entorno**, el cual puede estar formado por personas u otros sistemas. En el capítulo anterior vimos que el **comportamiento esperado** de un sistema de software propuesto se define a través de un documento de **especificación de requisitos de software (SRS)**. Para construir el sistema especificado, un paso clave es el **diseño de la arquitectura de software**, tema central de este capítulo.

Cualquier sistema complejo está compuesto por **subsistemas** que interactúan bajo el control del diseño del sistema, de modo que el conjunto provea el comportamiento esperado. Al diseñar un sistema, el enfoque lógico es **identificar los subsistemas que lo compondrán, las interfaces entre ellos y las reglas de interacción**. Justamente esto es lo que busca lograr la **arquitectura de software**.

La arquitectura de software es un área relativamente reciente. A medida que los sistemas de software se vuelven **más distribuidos y complejos**, la arquitectura se convierte en un paso esencial para su construcción. Debido a la **gran cantidad de opciones** disponibles para configurar y conectar un sistema, el diseño cuidadoso de la arquitectura resulta muy importante. Es durante este proceso donde se toman decisiones como el uso de cierto **middleware**, de una **base de datos**, de un **servidor** o de un **componente de seguridad**. No es posible diseñar todos los detalles y luego intentar incorporar estas decisiones; la arquitectura debe contemplarlas desde el inicio en la **estructura del sistema**.

Además, la arquitectura es el punto más temprano en el que se pueden evaluar propiedades como **fiabilidad** y **rendimiento**, algo cada vez más relevante en los sistemas modernos.

En este capítulo nos enfocaremos principalmente en los **conceptos de arquitectura** y en cierta **notación para describirla**. El tema de la **metodología** (es decir, cómo debe crearse la arquitectura) no se aborda, ya que se trata de una **actividad creativa de alto nivel** para la cual no existen metodologías estrictas. Sin embargo, discutiremos algunos **estilos arquitectónicos**, que sugieren formas de estructurar sistemas. Una combinación o variación de estos estilos puede resultar útil para muchos casos.

También trataremos temas relacionados con la **documentación de arquitecturas**, su **relación con el diseño** y una forma de **analizar arquitecturas**. Finalmente, cerraremos el capítulo con una discusión sobre la arquitectura en los **dos estudios de caso**.

4.1 El Rol de la Arquitectura de Software

Antes de hablar de su papel en la construcción de un sistema, debemos responder: **¿qué es la arquitectura?**

A un nivel alto, la **arquitectura** es el diseño de un sistema que ofrece una **vista general de sus partes y de cómo se relacionan** para formar el sistema completo. Esto implica **particionar** el sistema en componentes lógicos que pueden entenderse de forma independiente, y luego describir tanto esas partes como las relaciones entre ellas.

En un sistema complejo puede haber muchas maneras de dividirlo, y lo mismo ocurre con un software: **no existe una única arquitectura posible**, sino múltiples estructuras válidas.

Una de las definiciones más aceptadas es:

La arquitectura de software de un sistema es la estructura o estructuras del sistema, compuestas por elementos de software, las propiedades visibles externamente de esos elementos y las relaciones entre ellos.

Esto significa que la arquitectura se enfoca en las **abstracciones necesarias para especificar interacciones**: qué servicios brinda cada componente, qué propiedades de calidad ofrece (rendimiento, seguridad, fiabilidad, etc.) y cómo se relaciona con otros. **No es necesario entrar en los detalles internos de implementación** en esta etapa.

La arquitectura, entonces, representa de forma **concisa y comprensible** un sistema complejo. Eso sí, esta definición no evalúa si una arquitectura es buena o mala; eso requiere **análisis posterior**.

Principales usos de la descripción arquitectónica

1. Comprensión y comunicación

- Sirve como medio de **comunicación entre stakeholders**: usuarios, clientes, desarrolladores y arquitectos.
- Ofrece un **lenguaje común** para negociar y alinear expectativas.
- Reduce la complejidad mostrando el sistema como subsistemas y sus interacciones, ocultando los detalles de implementación.
- También puede documentar sistemas ya existentes, facilitando su entendimiento.

2. Reutilización (Reuse)

- La arquitectura es clave para decidir dónde y cómo **reutilizar componentes existentes**.
- Permite construir **familias de productos** (product lines), especificando qué partes son fijas y cuáles variables.
- La reutilización es más factible y eficaz a nivel arquitectónico que a nivel de detalle.

3. Construcción y evolución

- Una buena partición arquitectónica facilita que distintos equipos trabajen en paralelo en subsistemas relativamente independientes.
- Define **restricciones estructurales** que deben respetarse en el desarrollo.
- Durante la evolución del software, ayuda a decidir dónde agregar nuevas funcionalidades con el **mínimo impacto y complejidad**, y qué partes se verán afectadas por cambios.

4. Análisis

- Permite **predecir propiedades del sistema antes de construirlo**.

- Ejemplo: analizar confiabilidad, rendimiento o tiempos de respuesta de una arquitectura propuesta.
- Así se pueden explorar alternativas y elegir la que mejor satisfaga los requisitos de calidad.
- Por ejemplo, al diseñar un sitio de e-commerce, la arquitectura puede analizarse para estimar throughput y tiempos de respuesta según la carga esperada, y decidir si se necesita un servidor más potente o una arquitectura distinta.

No todos estos usos son igual de relevantes en todos los proyectos:

- En proyectos pequeños, la **comunicación** puede ser lo más importante.
- En sistemas grandes o críticos, el **análisis de rendimiento y confiabilidad** puede ser el foco central.

5 Planificación de un Proyecto de Software

6 Diseño Orientado a Funciones

7 Diseño Orientado a Objetos

8 Diseño Detallado

En capítulos anteriores se presentaron dos enfoques distintos para el **diseño de sistemas**.

En esa etapa del proceso, el objetivo principal es **definir los módulos** que componen el sistema y **cómo interactúan entre sí**.

Una vez que un módulo ha sido **especificado con precisión**, se puede pasar a determinar **la lógica interna** necesaria para implementar esas especificaciones. Ese es el tema central de este capítulo.

Aquí se explican los **métodos para desarrollar y describir el diseño detallado** de un módulo, junto con las **métricas** que pueden obtenerse a partir de dicho diseño.

8.1 Diseño Detallado y PDL

La mayoría de las técnicas de diseño, como el diseño estructurado, identifican los módulos principales y el flujo de datos más importante entre ellos. Los métodos utilizados para especificar el diseño del sistema suelen centrarse en las interfaces externas de los módulos y no pueden ampliarse fácilmente para describir su funcionamiento interno.

El *Process Design Language* (PDL) es una forma de comunicar el diseño de manera precisa y completa, con el nivel de detalle que el diseñador desee. Es decir, puede emplearse tanto para especificar el diseño del sistema como para extenderlo e incluir el diseño lógico.

El PDL resulta especialmente útil cuando se aplican técnicas de refinamiento descendente (*top-down refinement*) para diseñar un sistema o un módulo.

8.1.1 PDL

Una forma de **comunicar un diseño es especificarlo en un lenguaje natural, como el inglés**. Sin embargo, este enfoque **suele generar malentendidos y resulta impreciso**, lo que lo hace poco útil al momento de convertir el diseño en código. En el **extremo opuesto, se podría expresar el diseño en un lenguaje formal, como un lenguaje de programación**. Este tipo de representación **suele contener un nivel de detalle muy alto** —necesario para la implementación— pero no esencial para comunicar la estructura del diseño. Esos **detalles pueden dificultar la comprensión general**.

Lo ideal sería poder expresar el diseño en un lenguaje que sea lo más preciso y claro posible, pero sin caer en excesivos detalles, y que además pueda transformarse fácilmente en una implementación. Ese es el objetivo del *Process Design Language* (PDL).

El **PDL tiene una sintaxis externa similar a la de un lenguaje de programación estructurado, pero con un vocabulario de lenguaje natural** (en este caso, inglés). Puede entenderse como una especie de “inglés estructurado”. Dado que la estructura de un diseño expresado en PDL es formal, mediante el uso de construcciones típicas de lenguajes formales, es posible realizar cierto procesamiento automatizado sobre tales diseños.

Por ejemplo, supongamos que queremos encontrar el valor mínimo y máximo de un conjunto de números contenidos en un archivo y mostrar esos resultados. En PDL, el procedimiento podría describirse de manera completa en cuanto a su lógica, pero sin entrar en los detalles específicos de

implementación. Cada sentencia del PDL podría luego traducirse directamente a un lenguaje de programación concreto.

Otro ejemplo sería un programa que toma un texto en un archivo con una separación de un espacio entre palabras y lo formatea en líneas de 80 caracteres (excepto la última). Ninguna palabra debe dividirse entre dos líneas, y los espacios adicionales necesarios para completar la línea se añaden al final, sin usar más de dos espacios entre palabras. En PDL, este diseño se puede expresar mediante procedimientos que describen claramente el comportamiento deseado.

El PDL permite expresar el diseño con el nivel de detalle que se considere adecuado. Una forma común de usarlo es comenzar con un esquema general de la solución y luego ir refinándolo progresivamente, agregando más detalle conforme se avanza. Este enfoque de refinamiento sucesivo permite detectar errores de diseño temprano y facilita la verificación fase por fase, ayudando a obtener diseños más confiables. Además, su sintaxis estructurada fomenta el uso de construcciones de programación bien organizadas al implementar el diseño.

Las construcciones básicas del PDL son similares a las de un lenguaje estructurado. Por ejemplo, incluye el condicional IF, que se asemeja al `if-then-else` de Pascal, y estructuras de repetición flexibles que pueden adaptarse al problema sin necesidad de una formulación estricta. Algunos ejemplos válidos son:

DO WHILE there are characters in input file

DO UNTIL the end of file is reached

DO FOR each item in the list EXCEPT when item is zero

Además, en PDL pueden definirse y usarse diversas estructuras de datos, como listas, tablas, escalares o enteros. Existen varias versiones del PDL, y con el apoyo de herramientas automáticas, este lenguaje se utiliza ampliamente para comunicar y documentar diseños de software.

8.1.2 Diseño de lógica o de algoritmos

El objetivo principal del diseño detallado es definir la lógica de los distintos módulos que se especificaron durante el diseño del sistema. Especificar la lógica implica desarrollar un algoritmo que implemente esas especificaciones.

A continuación se presentan algunos principios para diseñar algoritmos o lógicas que cumplan con las especificaciones dadas.

El término *algoritmo* es bastante general y puede aplicarse a una amplia variedad de áreas. En esencia, un algoritmo es una secuencia de pasos que deben ejecutarse para resolver un problema determinado. El problema no tiene por qué ser necesariamente de programación. Por ejemplo, podemos diseñar algoritmos para actividades como cocinar un plato (una receta no es más que un algoritmo) o construir una mesa.

En el ciclo de desarrollo de software, solo nos interesan los algoritmos relacionados con el software. Para este contexto, un algoritmo se define como un procedimiento no ambiguo para resolver un problema. Un procedimiento es una secuencia finita de pasos u operaciones bien definidas, cada una de las cuales requiere una cantidad finita de memoria y tiempo para completarse. En esta definición

se asume que la terminación (es decir, que el proceso finaliza) es una propiedad esencial de todo procedimiento. A partir de aquí, usaremos los términos *procedimiento*, *algoritmo* y *lógica* como sinónimos.

El desarrollo de un algoritmo implica una serie de pasos. El primero es **definir el problema**. El problema para el cual se va a crear un algoritmo debe estar claramente expresado y bien entendido por la persona responsable de diseñarlo. En el diseño detallado, la definición del problema proviene del diseño del sistema, por lo tanto, ya se dispone de ella al comenzar esta etapa.

El siguiente paso es **desarrollar un modelo matemático del problema**. En esta fase se deben elegir las estructuras matemáticas más adecuadas para representarlo. Puede ser útil observar cómo se resolvieron problemas similares. En la mayoría de los casos, los modelos se construyen a partir de otros ya existentes, adaptándolos a las necesidades del problema actual.

Después se pasa al **diseño del algoritmo en sí**, donde se deciden las estructuras de datos y la estructura del programa. Una vez diseñado el algoritmo, es importante verificar su corrección.

No existe un procedimiento claro y universal para diseñar algoritmos. Tener uno equivaldría a automatizar completamente el proceso de desarrollo de algoritmos, algo que no es posible con los métodos actuales. Sin embargo, existen algunas heurísticas o métodos que pueden ayudar al diseñador.

El método más común para diseñar algoritmos o la lógica de un módulo es la **técnica de refinamiento paso a paso (stepwise refinement)**.

Esta técnica divide el problema de diseño lógico en una serie de pasos para poder desarrollarlo de manera gradual. El proceso comienza convirtiendo las especificaciones del módulo en una descripción abstracta del algoritmo que contiene unas pocas instrucciones generales. En cada paso, una o varias de esas instrucciones se descomponen en pasos más detallados. El refinamiento continúa hasta que todas las instrucciones son lo suficientemente precisas como para poder traducirse fácilmente a un lenguaje de programación.

Durante el refinamiento, tanto los datos como las instrucciones deben detallarse progresivamente. Una guía útil es que, en cada paso, la cantidad de descomposición sea manejable y represente una o dos decisiones de diseño.

El refinamiento paso a paso es un método **de arriba hacia abajo (top-down)** para desarrollar el diseño detallado. Como ya vimos, los métodos de arriba hacia abajo también se usan en el diseño de sistemas.

Para aplicar el refinamiento paso a paso se necesita un lenguaje que permita expresar la lógica de un módulo en distintos niveles de detalle, desde las especificaciones generales hasta la descripción detallada. Dicho lenguaje debe tener la flexibilidad suficiente para adaptarse a diferentes niveles de precisión. Los lenguajes de programación comunes no suelen ser adecuados para esto.

Por esa razón, se utiliza el **PDL (Process Design Language o Lenguaje de Diseño de Procesos)**, ya que tiene una **sintaxis externa formal** que asegura que el diseño desarrollado sea un “algoritmo computacional” que luego puede traducirse fácilmente a código, y una **sintaxis interna flexible basada en lenguaje natural**, lo que permite expresar las ideas con distintos grados de detalle y facilita el refinamiento.

Ejemplo:

Consideremos nuevamente el problema de contar las palabras distintas en un archivo de texto. Supongamos que en el diseño de alto nivel de un gran sistema de procesamiento de texto se ha especificado un módulo llamado COUNT, cuya tarea es determinar el número de palabras diferentes.

Durante el diseño detallado, debemos definir la lógica de este módulo para que cumpla con sus especificaciones. Usaremos el método de refinamiento paso a paso y, para especificar, el PDL con una sintaxis similar a C.

```
int count (file)
FILE file;
word_list wl;
{
    read file into wl
    sort (wl);
    count = different_words (wl);
    printf (count);
}
```

Figura 8.3: Estrategia para el primer paso en el refinamiento gradual.

Una estrategia sencilla para el primer paso se muestra en la Figura 8.3.

Esta estrategia es simple y fácil de entender; es la misma que se propuso en el diagrama de flujo de datos anterior. Las operaciones “primitivas” usadas en esta estrategia son de alto nivel y necesitan ser refinadas. En particular, tres operaciones requieren más detalle:

1. Leer el archivo y crear la lista de palabras.
2. Ordenar la lista de palabras en orden ascendente.
3. Contar las palabras distintas en una lista de palabras ordenada.

Hasta ahora, solo se ha definido una estructura de datos: la lista de palabras. A medida que se refine el diseño, podrían necesitarse más estructuras.

En el siguiente paso de refinamiento, elegimos una de las tres operaciones para desarrollarla con más detalle. Supongamos que refinamos el proceso de lectura. Una estrategia sería leer palabra por palabra y agregarlas a la lista de palabras (Figura 8.4).

```
read_from_file (file, wl)
FILE file;
word_list wl;
{
    initialize wl to empty;
    while not end-of-file {
        get_a_word from file
        add word to wl
    }
}
```

Figura 8.4: Refinamiento de la operación de lectura.

Esta estrategia es directa y puede completarse fácilmente en un paso. Otra opción sería leer grandes bloques de datos en un búfer y formar la lista de palabras desde allí, lo que podría ser más eficiente.

El siguiente paso podría ser refinar la función de conteo. Una estrategia posible para implementarla se muestra en la Figura 8.5.

```
int different_words (wl)
word_list wl;
{
    word last, cur;
    int cnt;

    last = first word in wl
    cnt = 1;
    while not end of list {
        cur = next word from wl
        if (cur <> last) {
            cnt = cnt + 1;
            last = cur;
        }
    }
    return (cnt)
}
```

Figura 8.5: Refinamiento de la función different_words.

De la misma forma, se puede refinar la función de ordenamiento. Una vez realizadas estas refinaciones, el diseño será suficientemente detallado y no requerirá más pasos.

En problemas más complejos, una sola operación puede necesitar muchos niveles de refinamiento. En estos casos, el diseño puede avanzar de dos maneras:

- **Profundidad primero (depth-first):** se termina completamente el refinamiento de una operación antes de pasar a otra.
- **Amplitud primero (breadth-first):** se refinan todas las operaciones una vez, y luego se continúa refinando cada una en sucesivas rondas.

También se puede combinar ambos enfoques.

Por último, vale la pena comparar la estructura de los programas PDL producidos con este método y la de los obtenidos mediante diseño estructurado. Las estructuras no son iguales. La diferencia principal es que, en el refinamiento paso a paso, la función `sort` depende directamente del módulo principal, mientras que en el diseño estructurado sería un submódulo del módulo de entrada.

Esta diferencia no es menor: refleja una distinción en los enfoques. En el refinamiento paso a paso, en cada etapa se especifican las operaciones necesarias (como al crear un diagrama de flujo de datos), mientras que en el diseño estructurado el foco está en dividir el problema en módulos de entrada, salida y transformación, lo que produce una estructura diferente.

8.1.3 Modelado de estado de las clases

En el diseño orientado a objetos, el enfoque recién discutido para obtener el diseño detallado puede no ser suficiente, ya que se centra en especificar la lógica o el algoritmo de los módulos

identificados en el diseño de alto nivel (orientado a funciones). Pero una clase no es una abstracción funcional y no puede verse como un algoritmo.

Un **método** de una clase sí puede considerarse un módulo funcional, y las técnicas mencionadas pueden aplicarse para especificar la lógica de dichos métodos. Sin embargo, para comprender mejor una **clase en su totalidad**, sin entrar en la lógica de cada método individual, se requiere un enfoque distinto del basado en refinamiento.

Un objeto de una clase tiene un **estado** y múltiples **operaciones** que pueden realizarse sobre él. Para entender una clase, es necesario comprender la relación entre su estado y las operaciones, así como el efecto que produce la interacción entre esas operaciones. Este es, de hecho, uno de los objetivos del **diseño detallado** en el desarrollo orientado a objetos. Una vez comprendida la clase en su conjunto, se pueden desarrollar los algoritmos para cada uno de sus métodos.

Cabe recordar que el **enfoque de especificación axiomática** de una clase, discutido anteriormente en este capítulo, también adopta esta perspectiva: en lugar de especificar la funcionalidad de cada operación, define —mediante axiomas— las interacciones entre diferentes operaciones.

Una forma de entender el comportamiento de una clase es modelarla como un **autómata finito (FSA, Finite State Automaton)**.

Un FSA está compuesto por **estados** y **transiciones** entre ellos, las cuales ocurren cuando se producen ciertos **eventos**.

Al modelar un objeto:

- El **estado** corresponde al valor de sus atributos.
- Un **evento** corresponde a la ejecución de una operación sobre el objeto.

Un **diagrama de estados** muestra la relación entre los eventos y los estados, indicando cómo cambia el estado cuando ocurre un evento.

Un diagrama de estados de un objeto suele incluir un **estado inicial**, desde el cual todos los demás estados son alcanzables (es decir, existe un camino desde el estado inicial a cualquier otro estado del autómata).

Sin embargo, el diagrama de estados **no representa todos los estados reales** del objeto, ya que estos pueden ser infinitos o muy numerosos. En su lugar, intenta representar solo los **estados lógicos** del objeto.

Un **estado lógico** es una agrupación de todos aquellos estados desde los cuales el comportamiento del objeto es similar para todos los eventos posibles. Dos estados lógicos son diferentes si difieren en su comportamiento para al menos un evento.

Por ejemplo, para un objeto que representa una **pila (stack)**:

- Todos los estados en los que la pila tiene entre 1 y un número máximo de elementos son equivalentes, ya que las operaciones (**push**, **pop**, **top**, etc.) se comportan de la misma manera en todos ellos.
- El estado que representa una **pila vacía** es distinto, ya que ahora las operaciones **top** y **pop** pueden generar un mensaje de error.
- Del mismo modo, el estado que representa una **pila llena** también es diferente.

El modelo de estados finitos para esta pila de tamaño limitado se muestra en la Figura 8.6.

El **modelado mediante autómatas finitos** de los objetos ayuda a comprender el efecto que las distintas operaciones definidas en una clase tienen sobre el estado del objeto. Una buena comprensión de esto puede facilitar el desarrollo de la lógica de cada operación.

Para desarrollar la lógica de las operaciones pueden utilizarse los métodos habituales de diseño de algoritmos. Además, el modelo de estados puede emplearse para **validar la corrección** de la lógica de una operación.

Como hemos visto, para una clase normalmente no se dispone de una especificación completa de entrada-salida para sus operaciones; por ello, el modelo FSA puede servir como **referencia de validación** para las distintas implementaciones de los métodos.

Más adelante, en el **Capítulo 10**, veremos que un modelo de estados también puede utilizarse para **generar casos de prueba** para la validación.

El **modelado de estado de clases** también ha sido propuesto como una técnica de análisis. Sin embargo, se considera que su utilidad durante la fase de análisis es **limitada** y que su papel es más apropiado durante el **diseño detallado**, cuando se necesita entender en profundidad el funcionamiento interno de una clase.

Incluso en esta etapa, su alcance sigue siendo restringido: resulta más útil cuando existe **una fuerte interacción entre los métodos a través del estado**, o cuando hay **muchos estados distintos** en los que los métodos deben comportarse de manera diferente.

8.2 Verificación

Existen algunas técnicas disponibles para **verificar que el diseño detallado sea consistente con el diseño del sistema**.

El enfoque de la verificación en la fase de diseño detallado se centra en **demostrar que el diseño detallado cumple con las especificaciones establecidas en el diseño del sistema**.

En esta etapa, **no se hace hincapié en validar** que el sistema, tal como ha sido diseñado, **sea consistente con los requisitos del sistema** (esa validación se realiza en fases anteriores).

Los tres métodos de verificación que se consideran son:

- **Revisiones de diseño (design walkthroughs)**
- **Revisiones críticas de diseño (critical design review)**
- **Comprobadores de consistencia (consistency checkers)**

8.2.1 Revisiones de diseño (Design Walkthroughs)

Una **revisión de diseño** o *walkthrough* es un **método manual de verificación**.

La forma en que se definen y utilizan los *walkthroughs* puede variar de una organización a otra. Aquí se describe un modelo general de este tipo de revisión.

Un *walkthrough* se realiza durante una **reunión informal** convocada por el diseñador o por el líder del grupo de diseño.

El grupo de revisión suele ser **pequeño** y está compuesto, además del diseñador, por el líder del grupo y/o otro diseñador del equipo.

A veces, el diseñador puede reunirse simplemente con un colega para realizar el *walkthrough*, o el líder puede solicitarle que lo haga con él.

Durante la sesión, **el diseñador explica paso a paso la lógica del diseño**, mientras que los demás participantes **hacen preguntas, señalan posibles errores o piden aclaraciones**.

Un efecto secundario positivo de este proceso es que, al **explicar y detallar su propio diseño**, el diseñador puede **descubrir errores por sí mismo**.

En esencia, los *walkthroughs* son una **forma de revisión entre pares** (*peer review*).

Sin embargo, debido a su **naturaleza informal**, suelen ser **menos efectivos que una revisión de diseño formal**.

8.2.2 Revisión Crítica del Diseño

El propósito de la revisión crítica del diseño es asegurar que el diseño detallado satisfaga las especificaciones establecidas durante el diseño del sistema. El proceso de revisión crítica del diseño es el mismo que el proceso de inspección, en el cual un grupo de personas se reúne para discutir el diseño con el objetivo de revelar errores de diseño o propiedades indeseables.

El grupo de revisión incluye, además del autor del diseño detallado, a un miembro del equipo de diseño del sistema, al programador responsable de codificar finalmente el(los) módulo(s) en revisión, y a un ingeniero independiente de calidad de software. Durante la revisión del diseño, se debe tener en cuenta que el objetivo es descubrir errores de diseño, no intentar corregirlos. La corrección se realiza más adelante.

El uso de listas de verificación, como en otras revisiones, se considera importante para el éxito de la revisión. La lista de verificación sirve como un medio para enfocar la discusión o la “búsqueda” de errores. Las listas pueden ser utilizadas por cada miembro durante el estudio privado del diseño y durante la reunión de revisión. Para obtener los mejores resultados, la lista de verificación debe adaptarse al proyecto en cuestión, para descubrir errores específicos del proyecto. A continuación se enumeran algunos elementos generales que pueden utilizarse para construir una lista de verificación para una revisión de diseño.

Ejemplo de Lista de Verificación

- ¿Existe cada uno de los módulos del diseño del sistema en el diseño detallado?
- ¿Hay análisis que demuestren que se pueden cumplir los requisitos de rendimiento?
- ¿Están todas las suposiciones explícitamente declaradas y son aceptables?
- ¿Se reflejan todos los aspectos relevantes del diseño del sistema en el diseño detallado?
- ¿Se han manejado las condiciones excepcionales?
- ¿Son todos los formatos de datos consistentes con el diseño del sistema?
- ¿Está el diseño estructurado y cumple con los estándares locales?
- ¿Se han estimado los tamaños de las estructuras de datos? ¿Se han tomado precauciones para evitar desbordamientos?
- ¿Está cada enunciado especificado en lenguaje natural fácilmente codificable?
- ¿Están correctamente especificadas las condiciones de terminación de los bucles?
- ¿Son correctas las condiciones en los bucles?
- ¿Son correctas las condiciones en las sentencias “if”?

- ¿Es adecuada la anidación?
- ¿Es la lógica del módulo demasiado compleja?
- ¿Son los módulos altamente cohesivos?

8.2.3 Verificadores de Consistencia

Las revisiones y recorridos del diseño son procesos manuales; las personas involucradas en ellos son quienes determinan los errores en el diseño. Si el diseño está especificado en un **PDL** (Lenguaje de Descripción de Programa) u otro lenguaje de diseño formalmente definido, es posible detectar algunos defectos de diseño utilizando **verificadores de consistencia**.

Los verificadores de consistencia son esencialmente compiladores que toman como entrada el diseño especificado en un lenguaje de diseño (PDL, en nuestro caso). Claramente, no pueden producir código ejecutable porque la sintaxis interna del PDL permite el uso de lenguaje natural y muchas actividades se especifican en lenguaje natural. Sin embargo, las especificaciones de las interfaces de los módulos (que pertenecen a la sintaxis externa) están especificadas formalmente.

Un verificador de consistencia puede asegurarse de que todos los módulos invocados o utilizados por un módulo dado realmente existan en el diseño, y que la interfaz usada por el llamador sea consistente con la definición de la interfaz del módulo llamado. También puede verificar si los elementos de datos globales utilizados están efectivamente definidos globalmente en el diseño.

Dependiendo de la precisión y la sintaxis del lenguaje de diseño, los verificadores de consistencia pueden generar otra información adicional. Además, estas herramientas pueden utilizarse para calcular la complejidad de los módulos y otras métricas, ya que dichas métricas se basan en construcciones alternativas y de bucles, las cuales tienen una sintaxis formal en PDL.

El compromiso aquí es que, cuanto más formal sea el lenguaje de diseño, más verificaciones podrán realizarse durante la etapa de diseño; pero el costo es que el lenguaje de diseño se vuelve menos flexible y tiende a parecerse más a un lenguaje de programación.

8.3 Métricas

Después del diseño detallado, la lógica del sistema y las estructuras de datos quedan en gran medida especificadas. Solo los detalles orientados a la implementación —que a menudo dependen del lenguaje de programación utilizado— necesitan definirse con mayor precisión.

Por lo tanto, muchas de las métricas que tradicionalmente se asocian con el código pueden utilizarse de manera efectiva después del diseño detallado. Durante esta etapa, todas las métricas tratadas en el diseño del sistema siguen siendo aplicables y útiles.

Con la lógica de los módulos ya disponible tras el diseño detallado, tiene sentido hablar de la **complejidad de un módulo**. Tradicionalmente, las métricas de complejidad se aplican al código, pero también pueden aplicarse fácilmente al diseño detallado. A continuación, se describen algunas métricas aplicables al diseño detallado.

8.3.1 Complejidad Ciclomática

Basado en la capacidad del cerebro humano y en la experiencia práctica, se reconoce generalmente que las condiciones y las sentencias de control agregan complejidad a un programa. Dado dos programas del mismo tamaño, el que tiene un mayor número de sentencias de decisión probablemente será más complejo.

La medida más simple de complejidad, entonces, es el número de construcciones que representan **ramificaciones en el flujo de control** del programa, como las sentencias `if-then-else`, `while-do`, `repeat-until` y `goto`.

Una medida más refinada es la **complejidad ciclomática** propuesta por *McCabe*, basada en conceptos de teoría de grafos. Para un grafo GGG con n nodos, e aristas y p componentes conectados, el número ciclomático $V(G)$ se define como:

$$V(G) = e - n + p$$

Para definir la complejidad ciclomática de un módulo, primero se dibuja el **grafo de flujo de control** GGG del módulo. Para construirlo, se divide el módulo en bloques delimitados por sentencias que afectan el flujo de control (como `if`, `while`, `repeat`, `goto`). Estos bloques forman los nodos del grafo. Si el control puede pasar de un bloque i a un bloque j , se dibuja un arco de i a j .

El grafo tiene un **nodo de entrada** y un **nodo de salida**, correspondientes al primer y último bloque de sentencias. En tal grafo, debe existir un camino desde la entrada hacia cualquier nodo y desde cualquier nodo hacia la salida (suponiendo que no haya código inalcanzable).

Si el código es una secuencia lineal sin estructuras de control, el número ciclomático puede ser 0. Sin embargo, se agrega un arco del nodo de salida al de entrada para hacer el grafo *fuertemente conexo*, garantizando así una complejidad distinta de cero incluso para programas simples. Por lo tanto, la **complejidad ciclomática de un módulo** se define como el número ciclomático de este grafo fuertemente conexo.

El número ciclomático de un módulo equivale al **número máximo de circuitos linealmente independientes** del grafo. Un conjunto de circuitos es linealmente independiente si ningún circuito está totalmente contenido en otro ni puede expresarse como combinación de otros. Por lo tanto, para calcular la complejidad ciclomática, se puede:

1. Dibujar el grafo,
2. Conectarlo agregando un arco de salida a entrada, y
3. Contar los circuitos o aplicar la fórmula $V(G) = e - n + p$.

Por ejemplo, en un grafo con 10 aristas, 7 nodos y un componente, se obtiene:

$$V(G') = 10 - 7 + 1 = 4$$

Los circuitos independientes serían:

- ckt 1: b c e b
- ckt 2: b c d e b

- ckt 3: a b f a
- ckt 4: a g a

También puede demostrarse que la **complejidad ciclomática** de un módulo es igual al **número de decisiones más uno**, donde una decisión corresponde a cualquier sentencia condicional en el módulo.

En el ejemplo dado, el módulo tiene tres decisiones (dos `while` y un `if`), por lo que la complejidad ciclomática es $3+1=4$.

El número ciclomático es una medida cuantitativa de la **complejidad de un módulo**. Aunque puede extenderse al programa completo, resulta más útil a nivel modular. McCabe propuso mantener la complejidad ciclomática de los módulos **por debajo de 10**.

Además, este número puede utilizarse como una **guía para determinar cuántos caminos deben probarse** durante las pruebas. La complejidad ciclomática es una de las métricas de complejidad más utilizadas.

Los experimentos muestran que la complejidad ciclomática está altamente correlacionada con el tamaño del módulo (en escala logarítmica), ya que más líneas de código suelen implicar más decisiones. También se ha encontrado correlación con el **número de errores** detectados en los módulos.

8.3.2 Vínculos de datos

Ya hemos visto que el **acoplamiento** y la **cohesión** son conceptos clave para evaluar un diseño. Sin embargo, para que esta evaluación sea realmente efectiva, es necesario contar con **métricas que permitan medir** el grado de acoplamiento entre módulos o la cohesión dentro de un módulo.

Durante el **diseño del sistema**, intentamos cuantificar el acoplamiento basándonos en el flujo de información entre módulos. Ahora que también conocemos la lógica interna de los módulos, podemos definir métricas que consideren tanto la estructura como el comportamiento. Una de estas métricas es el **vínculos de datos** (*data binding*), que busca capturar las interacciones de datos entre distintas partes del sistema de software. En otras palabras, los enlaces de datos miden **qué tan fuertemente acoplados están los diferentes módulos** del sistema. Existen distintos tipos de enlaces de datos.

Un **enlace de datos potencial** se define como un triplete (p, x, q) , donde p y q son módulos, y x es una variable que está dentro del **ámbito estático** de ambos. Esto refleja la posibilidad de que los módulos p y q puedan comunicarse mediante la variable compartida x . Este tipo de enlace no considera la lógica interna de los módulos para verificar si realmente acceden a x ; se basa únicamente en la declaración de la variable.

Un **enlace de datos usado** es un enlace potencial en el que tanto p como q efectivamente **usan la variable x** (ya sea para leer o asignar un valor). Este tipo de enlace es más difícil de determinar que el potencial, ya que requiere conocer más detalles sobre la lógica interna de los módulos.

Un **enlace de datos real** es un enlace usado con la condición adicional de que **el módulo p asigna un valor a x y el módulo q utiliza ese valor**. Este es el tipo más complejo de calcular, pues representa el flujo real de información desde el módulo p hacia el módulo q a través de la variable compartida x . Su cálculo requiere una descripción detallada de la lógica de ambos módulos.

Todos estos tipos de enlaces buscan **representar la fuerza de las interconexiones entre módulos**. Cuanto mayor sea el número de enlaces entre dos módulos, más estrechamente conectados estarán.

Para un tipo específico de enlace, puede construirse una **matriz** que contenga el número de enlaces entre los distintos módulos. Dicha matriz puede utilizarse luego en **análisis estadísticos** para evaluar el grado de interconexión del sistema o de alguno de sus subsistemas.

8.3.3 Métrica de Cohesión

Aquí se presenta un intento de **cuantificar la cohesión de un módulo**.

Para calcular el valor de la métrica de cohesión de un módulo M, se construye un **grafo de flujo** G para dicho módulo. Cada **vértice** en G representa una instrucción ejecutable de M. Para cada nodo, se registra también la **variable referenciada** en esa instrucción. Existe un **arco** desde un nodo S_i hacia otro nodo S_j si la instrucción S_j puede ejecutarse inmediatamente después de S_i en alguna ejecución del módulo.

Además, se agregan dos nodos especiales:

- un **nodo inicial** I, desde el cual comienza la ejecución del módulo, y
- un **nodo final** T, donde termina la ejecución.

Para las instrucciones de terminación (por ejemplo, `return`, `exit`), se dibuja un arco desde la instrucción correspondiente hacia T.

A partir de G, se construye un **grafo de flujo reducido** eliminando los nodos que **no hacen referencia a ninguna variable** (como los `goto` sin restricciones). Todos los arcos que llegaban al nodo eliminado se redirigen hacia su **sucesor** (estos nodos tienen solo un sucesor).

Supongamos que las variables están numeradas secuencialmente como 1, 2, ..., n.

Para una variable i:

- R_i es el **conjunto de referencia**, es decir, el conjunto de todas las instrucciones ejecutables que hacen referencia a la variable i.
La unión de todos los R_i constituye el conjunto de todos los nodos del grafo (excepto el nodo T, que no se ejecuta).

En esencia, esta métrica intenta medir la cohesión de un módulo observando **cuántos caminos independientes del módulo atraviesan las distintas instrucciones**.

La idea es que, si un módulo tiene **alta cohesión**, la mayoría de las variables serán utilizadas en la mayoría de los caminos.

Por lo tanto, en un módulo con alta cohesión, la cohesión de los conjuntos de referencia de cada variable será alta.

El **valor máximo posible** de cohesión ocurre cuando la dimensión de todos los conjuntos de referencia es igual al número total de caminos independientes, es decir, igual a la **complejidad ciclomática** del módulo.

En otras palabras, la cohesión máxima se alcanza cuando **todos los caminos independientes utilizan todas las variables** del módulo.

9 Codificación

El objetivo de la actividad de codificación o programación es **implementar el diseño de la mejor manera posible**.

La actividad de codificación influye profundamente tanto en las pruebas como en el mantenimiento. Como vimos antes, el tiempo dedicado a la codificación representa **un pequeño porcentaje del costo total del software**, mientras que las pruebas y el mantenimiento **consumen el mayor porcentaje**.

Por lo tanto, debe quedar claro que **el objetivo durante la codificación no debe ser reducir el costo de implementación**, sino **reducir el costo de las fases posteriores**, incluso si eso significa que el costo de esta fase debe aumentar. En otras palabras, **el objetivo durante esta fase no es simplificar el trabajo del programador**, sino **simplificar el trabajo del tester y del encargado del mantenimiento**.

Esta distinción es importante, ya que los programadores a menudo se preocupan por cómo terminar su trabajo rápidamente, sin tener en cuenta las fases posteriores. Durante la codificación, se debe tener presente que **los programas no deben construirse para que sean fáciles de escribir, sino para que sean fáciles de leer y entender**.

Un programa se lee con mucha más frecuencia y por muchas más personas en las fases posteriores.

Existen muchos criterios diferentes para juzgar un programa, entre ellos: **su legibilidad, tamaño, tiempo de ejecución y memoria requerida**.

Tener la **legibilidad y la comprensibilidad** como objetivos claros de la actividad de codificación puede, por sí mismo, **contribuir a producir software más mantenible**.

Un famoso experimento realizado por **Weinberg** mostró que si a los programadores se les especifica un objetivo claro para el programa, **suelen cumplirlo**. En el experimento, **cinco equipos diferentes** recibieron el mismo problema para desarrollar un programa, pero **a cada equipo se le indicó un objetivo distinto** que debía cumplir.

Los diferentes objetivos dados fueron:

- Minimizar el esfuerzo necesario para completar el programa.
- Minimizar el número de sentencias.
- Minimizar la memoria requerida.
- Maximizar la claridad del programa.
- Maximizar la claridad de la salida.

Se observó que, en la mayoría de los casos, **cada equipo obtuvo el mejor resultado en el objetivo que se le había asignado**.

La clasificación de los diferentes equipos para los distintos objetivos se muestra en la **Figura 9.1**.

	Resulting Rank (1 = Best)				
	O1	O2	O3	O4	O5
Minimize effort to complete (O1)	1	4	4	5	3
Minimize number of statements (O2)	2-3	1	2	3	5
Minimize memory required (O3)	5	2	1	4	4
Maximize program clarity (O4)	4	3	3	2	2
Maximize output clarity (O5)	2-3	5	5	1	1

Figure 9.1: El experimento de Weinberg.

El experimento muestra claramente que, **si los objetivos están bien definidos, los programadores tienden a alcanzarlos.**

Por lo tanto, si la **legibilidad** es un objetivo de la actividad de codificación, es probable que los programadores desarrollen programas **fáciles de entender.**

Para nuestros propósitos, la **facilidad de comprensión y de modificación** son los **objetivos básicos** de la actividad de programación.

En este capítulo, primero discutiremos **algunas prácticas y guías de programación**, en las que también mencionaremos algunos **errores comunes de codificación** para que los estudiantes tomen conciencia de ellos.

Luego discutiremos **algunos procesos que se siguen durante la codificación.**

A continuación, hablaremos sobre la **refactorización**, que se realiza durante la codificación, pero constituye una actividad distinta.

Después trataremos **algunos métodos de verificación**, seguidos por una **discusión sobre métricas.**

Finalmente, **concluiremos el capítulo** con una discusión sobre la **implementación de los estudios de caso.**

9.1 Principios y guías de programación

La tarea principal de un programador es **escribir código de calidad con pocos errores.**

La restricción adicional es hacerlo **rápidamente.**

Escribir código sólido es una **habilidad que solo puede adquirirse con la práctica.**

Sin embargo, basándose en la experiencia, se pueden establecer **algunas reglas y guías generales** para el programador.

La **buena programación** (es decir, producir programas **correctos y simples**) es una práctica **independiente del lenguaje de programación utilizado**, aunque los lenguajes de programación bien estructurados **facilitan el trabajo del programador.**

En esta sección, discutiremos **algunos conceptos y prácticas** que pueden ayudar al programador a **escribir código de mayor calidad.**

Dado que una de las tareas clave de un programador es **evitar errores en los programas**, comenzaremos analizando **algunos errores comunes de codificación.**

9.1.1 Errores comunes de codificación

Los errores de software (en esta sección utilizaremos los términos *errores*, *defectos* y *bugs* indistintamente; las definiciones precisas se dan en el siguiente capítulo) son una realidad con la que todos los programadores deben lidiar.

Gran parte del esfuerzo en el desarrollo de software se dedica a identificar y eliminar errores. Existen diversas prácticas que pueden reducir la aparición de bugs, pero sin importar las herramientas o métodos que usemos, los errores van a seguir apareciendo en los programas.

Aunque los errores pueden surgir de muchas maneras diferentes, ciertos tipos ocurren con mayor frecuencia.

A continuación, presentamos una lista de algunos de los errores más comunes.

El principal objetivo de discutirlos es educar a los programadores sobre estos errores, de modo que puedan evitarlos.

Fugas de memoria (Memory Leaks)

Una fuga de memoria ocurre cuando se asigna memoria a un programa pero luego no se libera correctamente.

Este error es una fuente común de fallos de software, y aparece con frecuencia en lenguajes que no tienen recolección automática de basura, como C o C++.

En programas cortos, las fugas de memoria tienen poco impacto, pero en sistemas que se ejecutan durante largos períodos pueden tener efectos catastróficos.

Un sistema de software con fugas de memoria consume cada vez más memoria, hasta que en algún momento el programa puede detenerse abruptamente debido a la falta de memoria libre.

Un ejemplo de este tipo de error es el siguiente:

```
char* foo(int s)
{
    char *output;
    if (s>0)
        output=(char*) malloc (size);
    if (s==1)
        return NULL; /* if s==1 then mem leaked */
    return(output);
}
```

Liberar un recurso ya liberado

En general, en los programas, los recursos primero se asignan y luego se liberan. Por ejemplo, la memoria se asigna primero y después se desasigna.

Este error ocurre cuando el programador intenta liberar un recurso que ya fue liberado previamente.

El impacto de este error común puede ser catastrófico.

Un ejemplo de este tipo de error es el siguiente:

```

main()
{
    char *str;
    str=(char *)malloc(10);
    if (global==0)
        free(str);
    free(str); /* str is already freed
}

```

El impacto de este error puede ser aún más grave si existe alguna instrucción `malloc` entre las dos llamadas a `free`.

En ese caso, existe la posibilidad de que la ubicación de memoria liberada por primera vez sea reasignada a una nueva variable, y que la segunda llamada a `free` libere esa nueva variable por error.

Desreferenciación de NULL

Este error ocurre cuando se intenta acceder al contenido de una ubicación de memoria que apunta a **NULL**.

Es un error muy común que puede hacer fallar completamente un sistema de software. Además, es difícil de detectar, ya que la desreferenciación de **NULL** puede ocurrir solo en ciertos caminos de ejecución y bajo determinadas condiciones.

A menudo, este problema surge debido a una inicialización incorrecta en distintos caminos del programa, lo que lleva a que una variable no se asigne correctamente antes de ser utilizada.

También puede deberse al uso de alias, es decir, cuando dos variables apuntan al mismo objeto: si una de ellas se libera y luego se intenta desreferenciar la otra, se produce el error.

El siguiente fragmento de código muestra dos casos de desreferenciación de **NULL**.

```

char *ch=NULL;
if (x>0)
{
    ch='c';
}
printf("\%c", *ch); /* ch may be NULL
*ch=malloc(size);
ch = 'c'; /* ch will be NULL if malloc returns NULL

```

Similar al error de **desreferenciación de NULL** está el error de **acceso a memoria no inicializada**. Este problema ocurre con frecuencia cuando los datos **se inicializan en la mayoría de los casos**, pero **algunas situaciones no previstas** quedan **sin cubrir** y, por lo tanto, **la variable no se inicializa antes de ser usada**.

Un ejemplo de este error es el siguiente:

```

switch( i )
{
    case 0: s=OBJECT_1; break;
    case 1: s=OBJECT_2; break;
}
return (s); /* s not initialized for values
            other than 0 or 1 */

```

Falta de direcciones únicas

El **aliasing** genera muchos problemas, y uno de ellos es la **violación de la unicidad de direcciones** cuando se espera que sean diferentes.

Por ejemplo, en una función de **concatenación de cadenas**, se espera que las **direcciones de origen y destino sean distintas**.

Si esto no ocurre —como en el segmento de código que se muestra a continuación— puede provocar **errores en tiempo de ejecución**.

```
strcat(src, destn);  
/* In above function, if src is aliased to destn,  
 * then we may get a runtime error */
```

Errores de sincronización

En un programa paralelo, donde hay múltiples hilos que pueden acceder a recursos compartidos, pueden ocurrir errores de sincronización. Estos errores son muy difíciles de detectar, ya que no se manifiestan fácilmente. Sin embargo, cuando aparecen, pueden causar daños serios al sistema.

Existen diferentes categorías de errores de sincronización, entre las cuales se encuentran:

1. Interbloqueos (**Deadlocks**)
2. Condiciones de carrera (**Race conditions**)
3. Sincronización inconsistente
 - Un **interbloqueo(dl)** ocurre cuando **uno o más hilos se bloquean mutuamente**. La causa más frecuente es la secuencia de bloqueo inconsistente, donde los hilos en interbloqueo esperan recursos que, a su vez, están bloqueados por otro hilo.
 - Las **condiciones de carrera** ocurren cuando **dos hilos intentan acceder al mismo recurso** y el **resultado de la ejecución depende del orden en que se ejecutan los hilos**.
 - La sincronización inconsistente es un error común que ocurre cuando hay una mezcla de accesos bloqueados y no bloqueados a algunas variables compartidas, especialmente si los accesos implican actualizaciones.

9.1.2 Programación Estructurada

Como se mencionó anteriormente, el objetivo básico de la actividad de codificación es producir **programas que sean fáciles de entender**. Muchos han argumentado que la práctica de la **programación estructurada** ayuda a desarrollar programas más comprensibles. El movimiento de programación estructurada comenzó en la década de 1970, y se ha hablado y escrito mucho al respecto. Hoy en día, el concepto está tan difundido que se acepta generalmente —incluso se da por sentado— que la programación debe ser estructurada.

Aunque se ha hecho mucho hincapié en la programación estructurada, el concepto y la motivación detrás de ella a menudo no se entienden completamente. A menudo se considera que la programación estructurada es programación "sin goto". Aunque el uso extensivo de **goto** ciertamente no es deseable, **se pueden escribir programas estructurados usando goto**. Aquí ofrecemos una breve explicación de qué es la programación estructurada.

Un programa tiene una **estructura estática** y una **estructura dinámica**.

- La **estructura estática** es la **organización del texto del programa**, que generalmente es una organización lineal de las sentencias del programa.
- La **estructura dinámica** es la **secuencia de sentencias que se ejecutan durante la ejecución del programa**.

En otras palabras, tanto la estructura estática como el comportamiento dinámico son secuencias de sentencias; mientras que la secuencia que representa la estructura estática es fija, la secuencia de sentencias que se ejecuta puede cambiar de una ejecución a otra.

La noción general de corrección de un programa significa que, cuando el programa se ejecuta, produce el comportamiento deseado. Para demostrar que un programa es correcto, necesitamos mostrar que **su comportamiento durante la ejecución es el esperado**. Por lo tanto, al analizar un programa, ya sea formalmente para probar su corrección o informalmente para depurarlo o convencernos de que funciona, **estudiamos la estructura estática del programa (su código) pero tratamos de razonar sobre su comportamiento dinámico**.

En otras palabras, **gran parte de la actividad de entender un programa consiste en entender su comportamiento dinámico a partir del texto del programa**. Será más fácil entender el comportamiento dinámico si este se asemeja a la estructura estática. Cuanto más cercana sea la correspondencia entre la ejecución y la estructura del texto, más fácil será entender el programa; mientras más diferente sea la estructura durante la ejecución, más difícil será razonar sobre el comportamiento a partir del código.

El **objetivo de la programación estructurada es garantizar que la estructura estática y la dinámica sean las mismas**. Es decir, se busca escribir programas de manera que la secuencia de sentencias ejecutadas durante la ejecución coincida con la secuencia de sentencias en el texto del programa. Dado que las sentencias en un texto de programa están organizadas linealmente, el objetivo de la programación estructurada se convierte en desarrollar programas cuyo flujo de control durante la ejecución sea lineal y siga la organización lineal del texto.

Claramente, ningún programa significativo puede escribirse como una secuencia de sentencias simples sin ningún desvío o repetición (lo cual también implica desvío). Entonces, ¿cómo se logra linearizar el flujo de control?

Mediante el uso de constructos estructurados.

En la programación estructurada, una sentencia no es solo una sentencia de asignación simple, sino una **sentencia estructurada**. La **propiedad clave** de una sentencia estructurada es que tiene **una sola entrada y una sola salida**. Es decir, durante la ejecución, la sentencia comienza desde un punto definido y termina en un punto definido.

Con sentencias de **entrada y salida única**, podemos ver un programa como una **secuencia de sentencias estructuradas**. Y si todas las sentencias son estructuradas, entonces durante la ejecución, la secuencia de ejecución será la misma que la secuencia en el texto del programa. Por lo tanto, usando sentencias de entrada y salida única, se puede lograr la correspondencia entre la estructura estática y la dinámica. Las sentencias de una sola entrada y una sola salida más comúnmente utilizadas son:

Selection: if B then S1 else S2

if B then S1

Iteration: While B do S

repeat S until B

Sequencing: S1; S2; S3;...

Se puede demostrar que estas tres construcciones básicas son suficientes para programar cualquier algoritmo concebible.

Los lenguajes modernos tienen otras construcciones similares que ayudan a linearizar el flujo de control de un programa, lo cual, en términos generales, facilita su comprensión.

Por lo tanto, los programas deberían escribirse de modo que, en la medida de lo posible, se utilicen estructuras de control de una sola entrada y una sola salida.

El objetivo principal, como hemos tratado de enfatizar, es hacer que la lógica del programa sea fácil de entender.

No se puede formular una regla estricta que sea aplicable en todas las circunstancias.

La práctica de la programación estructurada constituye una buena base y una guía útil para escribir programas de forma clara.

Debe señalarse que la razón principal por la cual se promovió la programación estructurada es la verificación formal de programas.

Como veremos más adelante en este capítulo, durante la verificación, un programa se considera una secuencia de sentencias ejecutables, y la verificación avanza paso a paso, considerando una sentencia de la lista (el programa) a la vez.

En estos métodos de verificación se asume que, durante la ejecución, las sentencias se ejecutarán en el mismo orden en que aparecen en el texto del programa.

Si esta suposición se cumple, la tarea de verificación se simplifica.

Por lo tanto, incluso desde el punto de vista de la verificación, es importante que la secuencia de ejecución de las sentencias coincida con su orden en el texto.

Una nota final sobre las construcciones estructuradas:

no cualquier fragmento de código con una sola entrada y una sola salida puede considerarse una construcción estructurada.

Si así fuera, uno siempre podría definir unidades apropiadas en cualquier programa para que pareciera una secuencia de dichas unidades (en el peor de los casos, todo el programa podría definirse como una sola unidad).

El objetivo fundamental de utilizar construcciones estructuradas es linearizar el flujo de control para que el comportamiento en tiempo de ejecución sea más fácil de entender y razonar sobre él.

En un flujo de control linealizado, si comprendemos adecuadamente el comportamiento de cada una de las construcciones básicas, entonces el comportamiento del programa puede considerarse una composición de los comportamientos de las distintas sentencias.

Para que este enfoque funcione, se implica que podamos entender con claridad el comportamiento de cada construcción.

Esto requiere que seamos capaces de describir de manera concisa el comportamiento de cada construcción.

A menos que podamos hacer esto, no será posible componerlas.

Claramente, para una estructura arbitraria no podemos hacer esto simplemente porque tenga una sola entrada y una sola salida.

Desde este punto de vista, las estructuras mencionadas anteriormente son las que se eligen como sentencias estructuradas, ya que existen reglas bien definidas que especifican cómo se comportan durante la ejecución, lo que nos permite razonar sobre programas más grandes.

En general, puede decirse que la programación estructurada conduce a programas más fáciles de entender que los no estructurados, y que dichos programas son también más fáciles (relativamente

hablando) de verificar formalmente.

Sin embargo, debe tenerse en cuenta que la programación estructurada no es un fin en sí misma.

Nuestro objetivo básico es que el programa sea fácil de comprender, y la programación estructurada es una manera segura de lograr este objetivo.

Aun así, existen algunas prácticas comunes de programación actualmente bien entendidas que hacen uso de construcciones no estructuradas (por ejemplo, las sentencias `break` o `continue`).

Aunque se debe procurar evitar el uso de sentencias que violen efectivamente la propiedad de una sola entrada y una sola salida, si el uso de tales sentencias es la forma más simple de organizar el programa, entonces, desde el punto de vista de la legibilidad, deberían utilizarse.

El punto principal es que una construcción no estructurada solo debería usarse si la alternativa estructurada es más difícil de entender.

Esta visión puede adoptarse únicamente porque estamos poniendo el foco en la legibilidad.

Si el objetivo fuera la verificabilidad formal, la programación estructurada sería probablemente necesaria.

9.1.3 Ocultamiento de Información

Una solución de software a un problema siempre contiene estructuras de datos que están destinadas a representar la información del dominio del problema.

Es decir, cuando se desarrolla un software para resolver un problema, este utiliza ciertas estructuras de datos para capturar la información presente en dicho dominio.

En general, solo ciertas operaciones se realizan sobre una determinada información.

Es decir, una pieza de información en el dominio del problema se usa solo de un número limitado de maneras.

Por ejemplo, un libro contable en una oficina de contabilidad tiene usos muy definidos: debitación, acreditación, consulta del saldo actual, etc.

Una operación como “multiplicar todos los débitos entre sí y luego dividirlos por la suma de todos los créditos” no suele realizarse.

Por lo tanto, cualquier tipo de información en el dominio del problema normalmente tiene un pequeño conjunto de operaciones definidas que se aplican sobre ella.

Cuando esa información se representa mediante estructuras de datos, debe aplicarse el mismo principio:

solo deben realizarse operaciones definidas sobre esas estructuras de datos.

Esto, esencialmente, es el principio del ocultamiento de información.

La información capturada en las estructuras de datos debe ocultarse del resto del sistema, y solo las funciones de acceso que representen las operaciones válidas sobre esa información deben ser visibles.

En otras palabras, cuando la información del dominio del problema se captura en estructuras de datos, para cada operación que se pueda realizar sobre dicha información, se debe proporcionar una función de acceso.

Y dado que, en el dominio del problema, el resto del sistema solo realiza esas operaciones definidas sobre la información, los demás módulos del software deben usar únicamente esas funciones de acceso para manipular las estructuras de datos.

El ocultamiento de información puede reducir el acoplamiento entre módulos y hacer que el sistema sea más fácil de mantener.

También es una herramienta eficaz para manejar la complejidad del desarrollo de software: al utilizar el ocultamiento de información, se separa la preocupación de manejar los datos de la preocupación de usarlos para producir los resultados deseados.

Muchos de los lenguajes más antiguos, como Pascal, C y FORTRAN, no proporcionan mecanismos para soportar la abstracción de datos.

Con esos lenguajes, el ocultamiento de información solo puede lograrse mediante un uso disciplinado del lenguaje, es decir, imponiendo manualmente restricciones de acceso por parte de los programadores, ya que el lenguaje no las proporciona.

En cambio, la mayoría de los lenguajes modernos orientados a objetos sí ofrecen mecanismos lingüísticos que permiten implementar el ocultamiento de información.

9.2 Proceso de Codificación

La actividad de codificación comienza cuando ya se ha realizado alguna forma de diseño y se dispone de las especificaciones de los módulos que deben desarrollarse.

Con el diseño en mano, los módulos suelen asignarse a desarrolladores individuales para su implementación.

En una implementación de arriba hacia abajo (top-down), se comienza asignando los módulos que están en la parte superior de la jerarquía y luego se avanza hacia los niveles inferiores.

En cambio, en una implementación de abajo hacia arriba (bottom-up), el desarrollo empieza por implementar los módulos de nivel más bajo de la jerarquía y luego se avanza hacia los superiores. La forma en que se elija proceder tiene un impacto directo en la integración y las pruebas.

Una vez que los módulos se asignan a los desarrolladores, estos utilizan algún proceso para desarrollar el código.

A continuación, se presentan algunos de los procesos que los desarrolladores suelen usar durante la codificación, o que han sido propuestos para guiar esta etapa.

9.2.1 Un Proceso de Codificación Incremental

El proceso que muchos desarrolladores siguen consiste en escribir el código del módulo que tienen asignado, y cuando lo terminan, realizar pruebas unitarias sobre él y corregir los errores que se encuentren. Luego, el código se sube al repositorio del proyecto para ponerlo a disposición de los demás integrantes del equipo.

(Explicaremos más adelante el proceso de *check-in* o subida del código).

Un proceso mejor para la codificación —que suelen seguir los desarrolladores con más experiencia— es desarrollar el código de manera incremental.

Esto significa escribir código que implemente solo una parte de la funcionalidad del módulo.

Ese código se compila y se prueba rápidamente con algunos tests simples, para verificar que la parte escrita funcione correctamente.

Una vez que el código pasa esas pruebas, el desarrollador agrega más funcionalidad, y vuelve a probar.

En otras palabras, el código se construye de manera incremental, probándolo a medida que se

desarrolla.

Este proceso de codificación se muestra en la Figura 9.2.

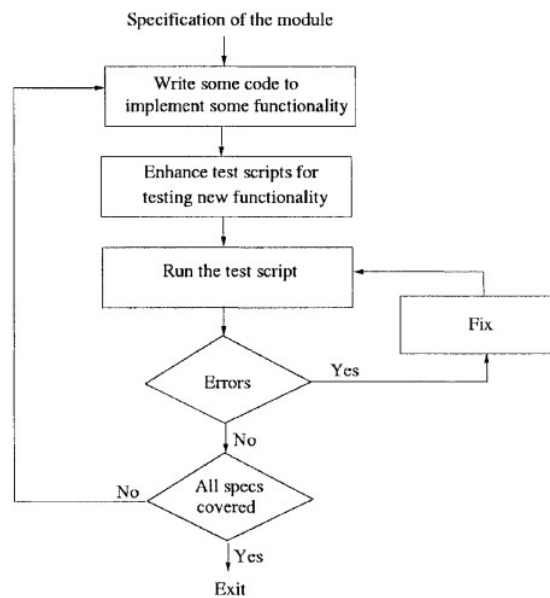


Figure 9.2: Un proceso de codificación incremental.

La ventaja principal de desarrollar código de forma incremental, realizando pruebas después de cada ronda de codificación, es que facilita la depuración (debugging):

si se encuentra un error durante las pruebas, se puede atribuir con seguridad al código que se agregó desde la última prueba exitosa.

Para seguir este proceso, es fundamental disponer de scripts de prueba automatizados, que permitan ejecutar los casos de prueba con solo presionar un botón.

Con estos scripts, las pruebas pueden realizarse tan frecuentemente como se desee, y además es fácil agregar nuevos casos de prueba.

Estos scripts de prueba son de gran ayuda cuando, en el futuro, se mejora o modifica el código debido a cambios en los requisitos: gracias a ellos se puede verificar rápidamente que la funcionalidad anterior sigue funcionando correctamente.

Incluso, estos scripts pueden reutilizarse —con algunas mejoras— para realizar las pruebas unitarias finales, que suelen hacerse antes de subir el módulo al repositorio.

9.2.2 Desarrollo Guiado por Pruebas (Test Driven Development)

El Desarrollo Guiado por Pruebas (TDD) es un proceso de codificación que invierte el enfoque tradicional de desarrollo.

En TDD, el programador primero escribe los scripts de prueba, y luego escribe el código necesario para que esas pruebas pasen exitosamente.

Todo el proceso se realiza de manera incremental, escribiendo las pruebas a partir de las especificaciones, y desarrollando el código para satisfacerlas.

El proceso de TDD se muestra en la Figura 9.3.

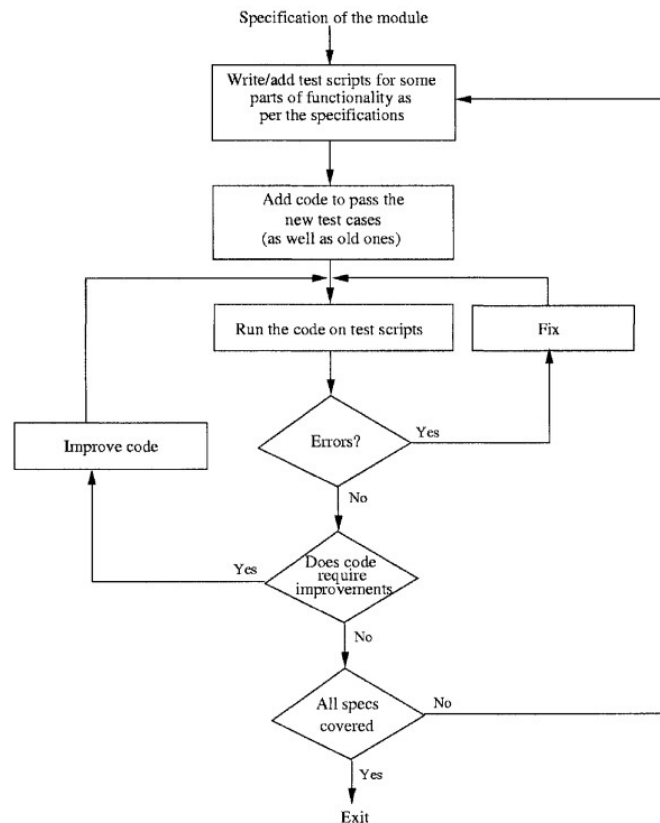


Figure 9.3: Proceso de desarrollo guiado por pruebas.

Este es un enfoque relativamente nuevo, que ha sido adoptado dentro de la metodología de Programación Extrema (Extreme Programming, XP).

Sin embargo, el concepto de TDD es general y no está limitado a ninguna metodología en particular.

Hay algunos puntos importantes que destacar sobre el TDD:

Primero, este enfoque establece que solo se debe escribir el código necesario para pasar las pruebas. Siguiendo esta idea, el código siempre se mantiene sincronizado con las pruebas.

Esto no siempre sucede con el enfoque tradicional de “código primero”, donde es común escribir una gran cantidad de código y luego crear solo unas pocas pruebas que cubren una parte del mismo.

Al promover que el código se escriba únicamente para pasar los tests, la responsabilidad de garantizar que se implemente toda la funcionalidad requerida recae en la etapa de escritura de los casos de prueba.

Es decir, son los casos de prueba los que deben verificar que el código desarrollado cubra toda la funcionalidad necesaria.

El hecho de escribir los casos de prueba antes del código hace que el desarrollo sea guiado por el uso (*usage-driven*).

Primero, se define cómo se espera que el código sea utilizado, lo cual se extrae de las especificaciones, y las interfaces de uso se definen con precisión al escribir los casos de prueba. Esto ayuda a asegurar que las interfaces se diseñen desde la perspectiva del usuario del código, y que los escenarios clave de uso se consideren antes de comenzar a programar.

El enfoque se centra en los usuarios del código, y el código se escribe para satisfacer sus necesidades, lo que puede reducir los errores de interfaz.

En TDD, también ocurre de forma natural cierta priorización del desarrollo.

Es muy probable que las primeras pruebas se enfoquen en la funcionalidad principal, mientras que las pruebas de características secundarias o de menor prioridad se escriban más adelante.

Como resultado, el código de las funcionalidades más importantes se desarrolla primero, y las características de menor prioridad se implementan después.

Esto tiene la ventaja de que las funciones más críticas se completan antes, pero también el inconveniente de que algunos casos especiales o características de baja prioridad pueden quedar sin manejar, si no se escriben pruebas para ellos.

Dado que el código se escribe para satisfacer los casos de prueba, la completitud del código depende directamente de la exhaustividad de las pruebas.

Sin embargo, escribir casos de prueba para todos los escenarios posibles o condiciones especiales puede resultar difícil y tedioso, y es poco probable que el desarrollador los cubra todos.

En TDD, dado que el objetivo es escribir solo el código necesario para pasar las pruebas existentes, algunos casos especiales podrían no ser tratados.

Además, como en cada paso el código se escribe principalmente para pasar los tests actuales, puede ocurrir que con el tiempo se descubra que algunos algoritmos implementados previamente no eran los más adecuados.

En ese caso, el código debería mejorarse o refactorizarse antes de agregar nueva funcionalidad, tal como se ilustra en la Figura 9.3.

9.2.3 Programación en Pareja (Pair Programming)

La programación en pareja es también un proceso de codificación que se ha propuesto como una técnica clave en la metodología de programación extrema (XP). En la programación en pareja, el código no es escrito por programadores individuales, sino por una pareja de programadores. Es decir, el trabajo de codificación no se asigna a un individuo, sino a una pareja de personas. Esta pareja escribe el código conjuntamente.

El proceso previsto es que una persona teclee el programa mientras la otra participa activamente y revisa constantemente lo que se está escribiendo. Cuando se notan errores, se señalan y corrigen. Cuando es necesario, la pareja discute los algoritmos, estructuras de datos o estrategias que se utilizarán en el código. Los roles se rotan con frecuencia, haciendo que ambos sean socios iguales y desempeñen roles similares.

La motivación básica para la programación en pareja es que, dado que la lectura y revisión de código han demostrado ser muy efectivas para detectar defectos, al hacer que una pareja realice la programación tenemos la situación en la que el código se revisa a medida que se escribe. Es decir, en lugar de escribir el código y luego hacer que otro programador lo revise, tenemos un programador que revisa constantemente el código que se está escribiendo. Al igual que en el desarrollo e implementación incremental de pruebas, ahora tenemos una revisión incremental en curso.

Además de la revisión continua del código, el hecho de que dos programadores se dediquen juntos a la tarea de programación probablemente dé lugar a mejores decisiones sobre las estructuras de

datos, algoritmos, interfaces, lógica, etc. Las condiciones especiales, que con frecuencia resultan en errores, también es más probable que se manejen de una mejor manera.

El posible inconveniente de la programación en pareja es que puede resultar en una pérdida de productividad al asignar dos personas a una misma tarea de programación. Está claro que una pareja producirá un código de mejor calidad en comparación con el desarrollado por un solo programador. La pregunta abierta es si este aumento en la productividad debido a la mejora en la calidad del código compensa la pérdida ocasionada por asignar dos personas a una tarea. También existen cuestiones de responsabilidad y propiedad del código, particularmente cuando las parejas no son fijas y rotan (como se propone en XP). El impacto de la programación en pareja es un área activa de investigación, especialmente dentro de la ingeniería de software experimental.

9.2.4 Control del Código Fuente y Construcción

En un proyecto, muchas personas diferentes desarrollan código fuente. Cada programador crea distintos archivos fuente, los cuales finalmente se combinan para crear los ejecutables. Los programadores van modificando sus archivos fuente a medida que el código evoluciona, como hemos visto en los procesos mencionados anteriormente, y con frecuencia también realizan cambios en otros archivos fuente. Para mantener el control sobre los archivos fuente y su evolución, casi siempre se utiliza un sistema de control de versiones en los proyectos, empleando herramientas como CVS en UNIX (www.cvshome.org) o Visual SourceSafe (VSS) en Windows (msdn.microsoft.com/vstudio/previous/ssafe). A continuación, se ofrece una breve descripción de cómo se utilizan estas herramientas en el proceso de codificación.

En el Capítulo 2 discutimos los conceptos generales de un proceso de gestión de configuración (CM). Nuestra explicación aquí se basa en CVS.

Un sistema moderno de control de código fuente contiene un repositorio, que es esencialmente una estructura de directorios controlada que mantiene el historial completo de revisiones de todos los archivos. Por eficiencia, el historial de un archivo generalmente se guarda como diferencias o incrementos respecto al archivo base. Esto permite recrear cualquier versión anterior del archivo, proporcionando así la flexibilidad de descartar fácilmente un cambio si es necesario. El repositorio es también la fuente “oficial” de todos los archivos.

Para un proyecto, debe configurarse un repositorio con permisos adecuados para las distintas personas del equipo. También se especifican los archivos que contendrá el repositorio —es decir, los archivos cuya evolución se desea mantener. Los programadores utilizan el repositorio tanto para hacer disponibles los cambios en sus archivos fuente como para obtener los archivos fuente de otros. Algunos de los comandos típicos que un programador ejecuta son:

Obtener una copia local. Un programador trabaja en una copia local de los archivos. Existen comandos para crear una copia local desde el repositorio. Obtener una copia local generalmente se denomina “checkout”. Un ejemplo de comando es `cvsv checkout <module>`, que copia en la máquina local el conjunto de archivos pertenecientes al módulo indicado. El usuario obtendrá la versión más reciente del archivo. Sin embargo, si lo desea, puede obtener cualquier versión anterior del archivo, ya que el repositorio mantiene el historial completo. Varios usuarios pueden realizar el checkout de un mismo archivo.

Realizar cambios en los archivos. Los cambios que un programador hace en su copia local permanecen locales hasta que se “confirman” (commit) en el repositorio. Al confirmar (por

ejemplo, con `cv commit <file>`), los cambios realizados localmente se actualizan en el repositorio y se vuelven disponibles para los demás. Esta operación también se conoce como “check in”.

Actualizar una copia local. Los cambios que otros miembros del proyecto confirman en el repositorio no se reflejan automáticamente en las copias locales que se obtuvieron antes. Para obtener estos cambios, las copias locales deben actualizarse (por ejemplo, con el comando `cv update`). Con esta operación, todos los cambios realizados en los archivos se reflejan en la copia local.

Obtener reportes. Las herramientas de control de versiones ofrecen una variedad de comandos para generar informes sobre la evolución de los archivos. Estos incluyen reportes que muestran las diferencias entre la versión local y la versión más reciente, o listan todos los cambios realizados en un archivo junto con las fechas y las razones de dichos cambios (que suelen proporcionarse al confirmar una modificación).

Cabe destacar que, una vez que los cambios se confirman, se vuelven disponibles para todos los miembros del equipo, quienes deben usar los archivos del repositorio al verificar sus propios programas. Por lo tanto, es esencial que un programador confirme un archivo fuente solo cuando esté en un estado utilizable por otros. En un estado estable, el comportamiento típico de un miembro del proyecto será el siguiente: obtener (checkout) la versión más reciente de los archivos que planea modificar, realizar los cambios planeados, verificar que los cambios tengan el efecto deseado (para lo cual puede copiar todos los archivos y probar el sistema localmente), y luego confirmar los cambios en el repositorio.

Está claro que si dos personas obtienen el mismo archivo y luego hacen cambios, puede surgir un conflicto —es decir, diferentes modificaciones en las mismas partes del archivo. Todas las herramientas detectarán el conflicto cuando la segunda persona intente confirmar los cambios y notificarán al usuario. Este debe resolver manualmente el conflicto, es decir, modificar el archivo de manera que las modificaciones no se contradigan con los cambios existentes, y luego confirmar el archivo. Los conflictos son poco frecuentes, ya que solo ocurren cuando se realizan cambios en las mismas líneas de un archivo.

Con un sistema de control de código fuente, un programador no necesita mantener todas las versiones por su cuenta —en cualquier momento, si es necesario deshacer algún cambio, las versiones anteriores pueden recuperarse fácilmente. Los repositorios siempre se respaldan, lo que también protege contra pérdidas accidentales. Además, se mantiene un registro detallado de los cambios —quién los hizo y cuándo, por qué se realizaron, qué se modificó exactamente, etc. Lo más importante es que el repositorio proporciona un lugar centralizado para almacenar los archivos más recientes y oficiales del proyecto. Esto es invaluable para productos con una vida larga y que evolucionan durante muchos años.

Además de utilizar el repositorio para mantener diferentes versiones, también se usa para construir el sistema de software a partir de los archivos fuente —una actividad que suele llamarse “build”. El proceso de build obtiene la versión más reciente (o una versión específica) de los archivos fuente del repositorio y genera los ejecutables a partir de ellos.

La construcción de los ejecutables finales a partir de los archivos fuente suele realizarse mediante herramientas como *Makefile* [62], que especifican las dependencias entre archivos y cómo deben

combinarse los archivos fuente para producir los ejecutables finales. Estas herramientas pueden detectar cuándo un archivo ha cambiado y recompilar automáticamente los necesarios para crear los ejecutables. Con el control de código fuente, estas herramientas generalmente obtienen la copia más reciente del repositorio antes de generar los ejecutables.

Este es uno de los enfoques más simples para el control de código fuente y el proceso de construcción. Sin embargo, cuando se construyen sistemas grandes, se requieren métodos más elaborados para el control del código y la compilación. Estos métodos suelen incluir una jerarquía de áreas controladas, cada una con distintos niveles de control y fuentes diferentes, donde la parte superior de la jerarquía contiene todos los archivos necesarios para construir el sistema “oficial”. Los niveles inferiores pueden ser utilizados por diferentes grupos para crear versiones “locales” destinadas a pruebas u otros propósitos. En tales sistemas, se requiere integración directa e inversa (forward y reverse integration) para transferir los cambios entre las distintas áreas controladas de la jerarquía. Una herramienta avanzada como *ClearCase* ofrece estas capacidades.

9.3 Refactorización

Hemos visto que la codificación a menudo implica realizar cambios en código existente. El código también cambia cuando cambian los requisitos o cuando se agrega nueva funcionalidad. Debido a las modificaciones que se realizan en los módulos, incluso si comenzamos con un buen diseño, con el tiempo solemos terminar con un código cuyo diseño ya no es tan bueno como podría ser. Y una vez que el diseño incorporado en el código se vuelve complejo, mejorar el código para adaptarlo a los cambios necesarios se vuelve más difícil, lleva más tiempo y es más propenso a errores. En otras palabras, la productividad y la calidad comienzan a disminuir.

La **refactorización** es la técnica utilizada para mejorar el código existente y evitar que el diseño se deteriore con el tiempo. La refactorización forma parte de la actividad de codificación, ya que se realiza durante ella, pero no es lo mismo que programar normalmente. La refactorización ha sido practicada por programadores desde hace tiempo, pero recientemente ha tomado una forma más concreta y se ha propuesto como un paso clave en la metodología de **Programación Extrema (XP)**.

La refactorización también cumple un papel importante en el desarrollo guiado por pruebas (**TDD**), ya que el paso de mejora del código en el proceso de TDD consiste, en realidad, en realizar refactorización. En esta sección discutiremos algunos conceptos clave y métodos para llevar a cabo la refactorización.

9.3.1 Conceptos Básicos

La refactorización se define como un cambio realizado en la estructura interna del software para hacerlo más fácil de entender y más barato de modificar, sin cambiar su comportamiento observable. Un punto clave aquí es que el cambio se realiza sobre el diseño incorporado en el código fuente (es decir, su estructura interna) con el único propósito de **mejorarlo**.

El objetivo principal de la refactorización es **mejorar el diseño**. Sin embargo, es importante notar que no se trata de mejorar un diseño durante las etapas de diseño para crear un modelo que luego se implementará (que es el objetivo de las metodologías de diseño), sino de **mejorar el diseño del código que ya existe**. En otras palabras, aunque la refactorización se realice sobre el código fuente,

su propósito es optimizar el diseño que dicho código implementa. Por lo tanto, los principios fundamentales del diseño guían el proceso de refactorización.

Como resultado, una refactorización suele producir uno o más de los siguientes efectos:

1. **Reducción del acoplamiento**
2. **Aumento de la cohesión**
3. **Mejor adherencia al principio abierto/cerrado** (en sistemas orientados a objetos)

La refactorización implica modificar el código para mejorar alguna propiedad del diseño, **manteniendo el mismo comportamiento externo**. A menudo, se inicia cuando es necesario realizar ciertos cambios en el código. Si se requiere agregar una mejora o modificación, y se percibe que la estructura actual del código dificulta ese cambio, ese es el momento adecuado para refactorizar y mejorar su estructura.

Aunque la refactorización suele surgir a partir de la necesidad de cambiar el software (y su comportamiento externo), **no debe confundirse ni mezclarse con la implementación de nuevas funcionalidades**. Lo ideal es mantener ambos tipos de cambios separados. Así, cuando un programador detecta la necesidad de refactorizar, **debe detener el desarrollo de nueva funcionalidad, realizar primero la refactorización y recién después continuar con la nueva implementación**.

El principal riesgo de refactorizar es que el código existente —que funciona correctamente— **puede romperse** debido a los cambios realizados. Esta es la razón más común por la que la refactorización no se hace con frecuencia. (La otra razón es que suele considerarse un costo adicional innecesario). Para mitigar este riesgo, existen dos **reglas de oro**:

1. **Refactorizar en pasos pequeños.**
2. **Contar con scripts de prueba que verifiquen la funcionalidad existente.**

Si se dispone de una buena batería de pruebas automatizadas, es fácil comprobar si la refactorización mantiene el comportamiento anterior. Sin una suite de pruebas, determinar si el comportamiento externo se ha alterado sería costoso y poco práctico. Al refactorizar en pequeños pasos y ejecutar pruebas tras cada uno, los errores pueden detectarse y corregirse fácilmente. Cada refactorización realiza un cambio pequeño, pero una **serie de refactorizaciones puede transformar significativamente la estructura de un programa**.

Gracias a la refactorización, el código puede mejorar continuamente. En lugar de degradarse con el tiempo, **el diseño evoluciona y se fortalece**. Esto mejora la calidad del código, facilita la incorporación de cambios y la detección de errores. El costo adicional de refactorizar se compensa luego con los ahorros obtenidos en menor tiempo de prueba y depuración, mayor calidad y menor esfuerzo para futuras modificaciones.

Si la refactorización se aplica de forma sistemática, también puede **simplificar la tarea de diseño** durante las etapas iniciales. A menudo, los diseñadores dedican mucho esfuerzo a tratar de crear un diseño “perfecto” o lo bastante flexible como para soportar todo tipo de cambios futuros. Esto puede volver el diseño demasiado complejo. Con la refactorización, el diseñador puede centrarse en crear un diseño **simple y correcto**, sabiendo que si surgen nuevas necesidades o limitaciones, el diseño podrá ajustarse más adelante mediante refactorización.

Finalmente, es importante aclarar que **la refactorización no es una técnica para corregir errores ni para reparar código en mal estado**. Se aplica a código que ya funciona correctamente. Su propósito es **prolongar la vida útil del software** manteniendo su estructura saludable. En lugar de dejar que el código se deteriore con el tiempo, la refactorización busca mantenerlo constantemente en buena forma.

9.3.2 Un ejemplo

9.3.3 Malos olores

Ahora hablaremos de las **señales en el código que pueden indicarnos que podría ser necesario aplicar refactorización**. A estas señales a veces se las llama “malos olores” (*bad smells*).

Básicamente, son señales fáciles de detectar en el código que suelen indicar que algunas de las **propiedades deseables del diseño pueden estar siendo violadas, o que existe la posibilidad de mejorar el diseño**. En otras palabras, si “huelo” uno de estos “malos olores”, puede ser una señal de que se necesita refactorizar. Por supuesto, si la refactorización es realmente necesaria deberá decidirse caso por caso, observando el código y las oportunidades de mejora que puedan existir. A continuación se presentan algunos de estos malos olores tomados de [65].

1. **Código duplicado.** Esto es bastante común. Una razón puede ser que una pequeña funcionalidad se ejecuta en varios lugares (por ejemplo, calcular la edad a partir de la fecha de nacimiento en cada sitio donde se necesita). Otra razón frecuente es que, cuando hay múltiples subclases de una clase, cada subclase termina haciendo algo similar. El código duplicado implica que, si se necesita cambiar esa lógica o función, habrá que modificarla en todos los lugares donde aparece, lo que hace los cambios mucho más difíciles y costosos.
2. **Método largo.** Si un método es grande, a menudo representa una situación en la que intenta hacer demasiadas cosas y, por lo tanto, carece de cohesión.
3. **Clase larga.** De manera similar, una clase grande puede indicar que está encapsulando múltiples conceptos, haciendo que la clase no sea cohesionada.
4. **Lista de parámetros larga.** Las interfaces complejas claramente no son deseables: hacen que el código sea más difícil de entender. A menudo, esta complejidad no es intrínseca, sino una señal de un diseño inadecuado.
5. **Sentencias switch.** En los programas orientados a objetos, si el polimorfismo no se usa adecuadamente, es probable que aparezcan sentencias *switch* en cada lugar donde el comportamiento dependa de alguna propiedad. La presencia de sentencias *switch* similares en diferentes partes del código indica que se está utilizando una estructura condicional en lugar de una jerarquía de clases. La presencia de una sentencia *switch* hace mucho más difícil extender el código: si se agrega una nueva categoría, será necesario modificar todas las sentencias *switch*.
6. **Generalidad especulativa.** Algunas jerarquías de clases pueden existir porque los objetos de las subclases parecen diferentes. Sin embargo, si el comportamiento de los objetos de las diferentes subclases es el mismo y no hay una razón inmediata para pensar que estos comportamientos puedan cambiar, se trata de un caso de complejidad innecesaria.

7. **Demasiada comunicación entre objetos.** Si los métodos de una clase hacen muchas llamadas a métodos de otro objeto para averiguar su estado, esto es un signo de acoplamiento fuerte. Es posible que esto sea innecesario, por lo que tales situaciones deberían analizarse para una posible refactorización.
8. **Encadenamiento de mensajes.** Un método llama a otro método, que simplemente pasa la llamada a otro objeto, y así sucesivamente. Esta cadena puede generar un acoplamiento innecesario.

En general, estos malos olores son indicativos de un mal diseño. Podemos notar que muchos de estos olores estaban presentes en nuestro ejemplo anterior.

9.3.4 Refactorizaciones comunes

Está claro que existen infinitas posibilidades sobre cómo se puede refactorizar el código para mejorar su diseño. Un catálogo de refactorizaciones comunes y los pasos para realizar cada una de ellas se presenta en. Nuevas refactorizaciones se siguen listando continuamente en www.refactoring.com. Como se mencionó antes, una refactorización debe ayudar a que el código sea más fácil de entender y modificar.

Para lograr este objetivo, muchas refactorizaciones se enfocan en mejorar los métodos, las clases o las jerarquías de clases. Aquí discutimos brevemente algunas de las refactorizaciones sugeridas. Existen otras refactorizaciones que tratan con la reducción de cadenas de mensajes (*message chains*), pero no las discutiremos aquí.

El proceso general de refactorización es el mismo para todas ellas: se realiza un paso de refactorización y luego se prueba el nuevo código (con pruebas automatizadas) para asegurarse de que la refactorización no haya alterado el comportamiento y que las pruebas anteriores sigan pasando. Si se van a aplicar múltiples refactorizaciones, deben aplicarse una a la vez, y una nueva solo debe realizarse después de que la anterior haya sido probada con éxito.

Mejorando los métodos

Ya vimos antes que un método puede no ser cohesivo y puede realizar muchas funciones diferentes. Un objetivo principal de la refactorización para mejorar métodos es simplificarlos y hacerlos más cohesivos. El nivel de acoplamiento de un método depende en gran medida de su interfaz, y al simplificar la interfaz, el acoplamiento puede reducirse.

1. Extraer método (Extract Method).

Esta refactorización se realiza con frecuencia cuando un método es demasiado largo, lo que indica que puede no ser cohesivo y estar realizando múltiples funciones. El objetivo es tener métodos cortos cuyas firmas den a los usuarios una idea bastante precisa de lo que hacen. Durante esta refactorización, un fragmento de código dentro de un método se extrae y se convierte en un nuevo método. Las variables a las que hace referencia ese fragmento y que están en el alcance del método original se convierten en los parámetros del nuevo método. Cualquier variable declarada en ese fragmento pero usada en otra parte deberá definirse en el método original. En el método original, el código extraído se reemplaza por una llamada al nuevo método.

A veces, el nuevo método puede ser una función que devuelve un valor. Un ejemplo de esta refactorización fue mostrado anteriormente.

De manera similar, si existe un método que devuelve un valor pero también cambia el estado de algunos objetos, entonces este método debe dividirse en dos: uno que devuelva el valor deseado y otro que realice el cambio de estado necesario. Tener un método que solo devuelve un valor y no tiene efectos secundarios generalmente implica una fuerte cohesión funcional. (Esta refactorización se llama **separar consulta de modificador** —*separate query from modifier*.)

2. Agregar o eliminar parámetros (Add/Remove Parameter).

Si un método necesita más información de su invocador, quizás porque el alcance de lo que debe hacer se ha ampliado, entonces se deben agregar nuevos parámetros. Sin embargo, esto debe hacerse solo si los parámetros existentes no pueden proporcionar la información necesaria.

El caso opuesto es la eliminación de parámetros. A veces, por razones de flexibilidad o futuras extensiones, se piden parámetros que finalmente no se utilizan. Por simplicidad, los parámetros que no se usan deberían eliminarse. Pero si la clase forma parte de una jerarquía, esto debe hacerse con cuidado para asegurar que las relaciones jerárquicas no se vean afectadas. O bien, el cambio tendrá que realizarse también en los niveles superiores o inferiores de la jerarquía.

Mejorando las clases

La mayoría de las refactorizaciones en esta categoría se centran en mejorar la cohesión de las clases o en reducir el acoplamiento entre ellas. Aumentar la cohesión de las clases a menudo implica mover campos y métodos de una clase a otra, de manera que los datos y comportamientos relacionados se encuentren encapsulados en un mismo objeto. La reducción del acoplamiento, en cambio, suele requerir cambios en las clases para disminuir el grado de interacción entre ellas.

1. Mover método (Move Method).

En esta refactorización, un método se mueve de una clase a otra. Es una de las más importantes y se realiza generalmente cuando un método en la clase A interactúa demasiado con un objeto de la clase B, lo que sugiere que el lugar más natural para el método es B y no A.

A veces no está claro desde el principio a qué clase debería pertenecer un método, por lo que el diseñador puede colocarlo inicialmente en A. Sin embargo, si más adelante se observa que su ubicación lógica es B, entonces debe moverse.

Inicialmente, es recomendable dejar el método original como un método *delegado* que solo invoque al nuevo método. De esta forma, el cambio afecta únicamente a esas dos clases. No obstante, siempre que sea posible, las referencias o llamadas al método deben redirigirse al nuevo, y el método original debe eliminarse. Si el método original no era privado, todas las clases que lo utilizaban deberán actualizarse.

2. Mover campo (Move Field).

Si un campo en la clase A es utilizado con mayor frecuencia por métodos de la clase B, ese campo debería trasladarse a B. Esto reducirá el acoplamiento entre A y B y mejorará la encapsulación de ambas clases.

Esta refactorización es similar a la anterior: en *Mover Método* se reasigna comportamiento, y en *Mover Campo* se reasigna estado. Si el campo era privado, después de moverlo, todas las

referencias a él en los métodos de A deberán reemplazarse por llamadas a métodos apropiados de B para obtener o modificar ese estado.

3. **Extraer clase (Extract Class).**

A menudo un diseñador comienza con una clase que, con el tiempo y la incorporación de nuevas funcionalidades, se amplía tanto que termina teniendo múltiples responsabilidades o conceptos mezclados. Si una clase grande parece carecer de una abstracción clara o está asumiendo demasiadas responsabilidades, esta refactorización es apropiada.

Se crea una nueva clase y se mueven a ella los campos y métodos relacionados. La refactorización se justifica si ambas clases resultantes tienen una abstracción más clara y responsabilidades bien definidas. Sin embargo, si el resultado genera un acoplamiento excesivo entre ambas clases, eso indica que la división fue artificial.

Una manera práctica de aplicar esta refactorización es primero crear la nueva clase y establecer un enlace desde la clase original. Luego se aplican las refactorizaciones *Mover Campo* y *Mover Método* repetidamente para transferir los elementos que correspondan a la nueva clase.

4. **Reemplazar un valor de datos con un objeto (Replace Data Value with Object).**

Esta refactorización es similar a *Extraer Clase*. A menudo, ciertos datos se tratan inicialmente como campos simples de una clase. Sin embargo, a medida que el desarrollo avanza, esos datos pueden adquirir mayor significado y asociarse con múltiples operaciones. Ejemplos típicos son fechas, números de teléfono, direcciones o números de seguridad social.

Si se realizan múltiples operaciones sobre un mismo dato dentro de la clase, puede tener sentido convertirlo en un objeto propio. Esto mejora la organización del código, facilita la reutilización y refuerza la cohesión.

Mejorando las jerarquías

La jerarquía de clases es un concepto clave en la programación orientada a objetos y constituye la base del *principio abierto/cerrado*. Para cumplir con este principio, es fundamental utilizar adecuadamente el polimorfismo. El objetivo de las refactorizaciones en esta categoría es aprovechar el polimorfismo para facilitar la extensión futura del código y crear diseños que representen el problema de manera más natural.

1. **Reemplazar condicional con polimorfismo (Replace Conditional with Polymorphism).**

Si una clase contiene métodos cuyo comportamiento depende del valor de algún tipo o código de estado (*type-code*), se está utilizando un enfoque más cercano al estilo funcional que al orientado a objetos. En estos casos, el polimorfismo puede y debe emplearse para representar mejor la situación.

La presencia de una sentencia `switch`, `case` o una serie de condicionales `if/else` dentro de un método, o de variables que actúan como códigos de tipo dentro de la clase, son señales claras de que esta refactorización puede ser necesaria.

Por ejemplo, en el método `printReport()` de una clase `GradReport`, si el comportamiento varía según si el estudiante es de posgrado (PG) o de grado (UG), una mejor solución es crear una jerarquía de clases donde `PGStudent` y `UGStudent` hereden de una clase base `Student`.

Las funcionalidades comunes se ubican en la clase base, mientras que los comportamientos

específicos se implementan en las subclases. Así, un objeto del tipo `Student` puede apuntar dinámicamente a cualquiera de las clases derivadas, eliminando la necesidad de sentencias condicionales y capturando las diferencias de comportamiento mediante el polimorfismo.

2. Elevar campo/método (**Pull Up Field/Method**).

Un buen diseño de jerarquía requiere que los elementos comunes se ubiquen en las superclases y los elementos variables en las subclases.

Si se detecta que varias subclases tienen el mismo campo, dicho campo debería moverse hacia arriba en la jerarquía, es decir, a la superclase.

De igual manera, si existen métodos en distintas subclases que realizan esencialmente la misma función (aunque con implementaciones distintas), se está duplicando funcionalidad, lo que dificulta el mantenimiento y la evolución del código. En este caso, el método debe trasladarse a la superclase.

Incluso los constructores pueden refactorizarse de esta forma: si varias subclases comparten constructores similares, estos pueden elevarse a la superclase y ser invocados desde los constructores de las subclases.

La operación inversa se conoce como **empujar hacia abajo (Push Down Field/Method)**.

Si una superclase contiene un campo que solo es utilizado por una o pocas subclases, es mejor mover ese campo hacia la subclase correspondiente.

Del mismo modo, si un método en la superclase solo tiene relevancia para ciertas subclases, debería desplazarse hacia abajo, es decir, a aquellas clases donde realmente se utiliza.

Estas refactorizaciones ayudan a mantener jerarquías limpias, lógicas y fáciles de extender, reduciendo duplicación de código y mejorando la cohesión entre clases.

9.4 Verificación

Una vez que un programador ha escrito el código para un módulo, este debe ser verificado antes de que sea utilizado por otros. Hasta ahora hemos asumido que la verificación se realiza mediante pruebas. Aunque las pruebas son el método más común de verificación, existen otras técnicas efectivas. Aquí discutiremos algunas de las más comunes.

Es importante aclarar que por verificación no nos referimos a la **demostración formal de la corrección de programas**, que para nuestros propósitos es solo un método dentro del conjunto de técnicas de verificación.

Nos centraremos en las técnicas que se utilizan ampliamente en la práctica, como las **inspecciones** (incluyendo la lectura de código), las **pruebas unitarias** y la **verificación automática de programas (program checking)**. También hablaremos brevemente de la **verificación formal**, aunque esta se usa con menos frecuencia y principalmente en situaciones especiales.

Aunque aquí nos enfocamos en la verificación de programas individuales escritos por los programadores, algunas técnicas —como la verificación automática— también pueden aplicarse a nivel de **sistema completo**.

9.4.1 Inspección de código (Code Inspections)

La **inspección** es una técnica general de verificación que puede aplicarse a cualquier tipo de documento y se ha utilizado ampliamente para detectar defectos. Originalmente se desarrolló para encontrar errores en el código, pero luego se aplicó también al diseño, los requisitos, los planes, etc. El proceso general de inspección ya fue discutido anteriormente, y para la inspección de código se mantiene básicamente igual.

Las **inspecciones de código** suelen realizarse una vez que el código ha sido compilado correctamente y después de aplicar otras herramientas estáticas. Esto se hace para ahorrar tiempo y esfuerzo humano, evitando que las personas busquen errores que ya podrían ser detectados por un compilador o un analizador estático.

La documentación que se entrega al equipo de inspección incluye el **código a revisar** y el **documento de diseño**. El equipo debe incluir al programador, al diseñador y al tester. El objetivo principal de la inspección es **detectar defectos en el código**, aunque también se revisan **aspectos de calidad** como la eficiencia o el cumplimiento de estándares de codificación.

Con frecuencia, las inspecciones se guían por una **lista de verificación (checklist)** que contiene los tipos de defectos en los que deben centrarse los inspectores. Algunos ejemplos de ítems que puede incluir una checklist son:

- ¿Las definiciones de datos aprovechan las capacidades de tipado del lenguaje?
- ¿Todos los punteros apuntan a algún objeto (no hay punteros colgantes)?
- ¿Se asignan los punteros a NULL cuando corresponde?
- ¿Se verifica si un puntero es NULL antes de usarse?
- ¿Los índices de los arreglos están dentro de los límites?
- ¿Los índices están correctamente inicializados?
- ¿Las condiciones de las ramas son correctas (ni demasiado débiles ni demasiado fuertes)?
- ¿Los bucles siempre terminan (no hay bucles infinitos)?
- ¿La condición de terminación de los bucles es correcta?
- ¿El número de ejecuciones del bucle no tiene un error de “uno de más o uno de menos”?
- ¿Se verifica que los divisores no sean cero?
- ¿Se valida la entrada de datos externos?
- ¿Los parámetros reales y formales coinciden?
- ¿Todas las variables se utilizan y todas las variables de salida se asignan?
- ¿Existen sentencias dentro de bucles que podrían moverse fuera de ellos?
- ¿Hay etiquetas sin referencia?
- ¿Se cumplen los requisitos de tiempo de ejecución?
- ¿Se siguen los estándares locales de codificación?

Las inspecciones son **muy efectivas** para detectar defectos y se utilizan ampliamente en muchas organizaciones comerciales. Sin embargo, también **son costosas**, ya que requieren el tiempo de varias personas. Por eso, en algunos casos el costo puede no estar justificado, y en lugar de una inspección grupal puede realizarse una **revisión individual**.

Una técnica común para esto es la **lectura estructurada de código (code reading)**. Esta consiste en leer cuidadosamente el código para detectar discrepancias entre la especificación de diseño y la

implementación real. El lector debe identificar la **abstracción del módulo** y luego compararla con su especificación.

El proceso de lectura es el inverso del diseño:

- En el diseño se parte de una abstracción y se avanza hacia los detalles.
- En la lectura de código se parte de los detalles del programa y se asciende hacia una descripción abstracta.

Lo ideal es leer el código **desde adentro hacia afuera**, empezando por la estructura más interna del módulo. Primero se determina su comportamiento abstracto, luego se reemplaza en la estructura de nivel superior, y se repite hasta llegar al módulo completo. Finalmente, el comportamiento abstracto obtenido se compara con las especificaciones para detectar discrepancias.

La **lectura de código** es muy útil y puede detectar errores que no aparecen en las pruebas. Además, fomenta que los programadores escriban código estructurado de manera clara, lo que mejora la calidad general. Este proceso también se conoce como **revisión de escritorio (desk review)**.